ჩემი პირადი ტექნიკური სახელმძღვანელო — NAPR QC პლატფორმა

> **ვისთვის:** ჩემთვის (ლუკა) და სალომესთვის: IT-სთან საუბრისთვის საჭირო ცნებების სწავლა, ჩვენს პროექტზე, მაგალითებით. **მდგომარეობა:** 2026-10-22. ფაქტები ემყარება კოდის აუდიტს; რა რას ნიშნავს — 01 — Handoff, §0.1. **მიზანი:** არა დამახსოვრება, არამედ გაგება. თითოეული ცნება ისე უნდა გავიგო, რომ საკუთარი სიტყვებით ავხსნა.

როგორ არის აწყობილი ყოველი თემა

ყოველი მნიშვნელოვანი ცნება ერთი და იმავე რვა ნაწილით არის ახსნილი:

1. **ტექნიკური განმარტება** — როგორც გამოცდილი ინჟინერი იტყოდა.
2. **მარტივად რომ ავხსნათ** — ყოველდღიური ენით.
3. **ჩვენს პროექტში** — სად და როგორ გვხვდება.
4. **რატომ გვჭირდება** — პრაქტიკული აზრი.
5. **როგორ მუშაობს კულისებში** — მექანიზმი.
6. **რა შეიძლება აირიოს** — ტიპური პრობლემები.
7. **რას იკითხავს IT** — სავარაუდო კითხვები.
8. **როგორ ვუპასუხო** — მოკლე, გულწრფელი პასუხი.

სტატუსის ნიშნები (✅ 🅿️ 💡 ❓ ⚠️) იგივეა, რაც ყველა დოკუმენტში.

სარჩევი

- **A. სერვერები და ჰოსტინგი** — A.1–A.16
- **B. SQL და მონაცემთა ბაზები** — B.1–B.18
- **C. API და გარე მონაცემი** — C.1–C.13
- **D. უსაფრთხოება და ავთენტიფიკაცია** — D.1–D.16
- **E. აპლიკაციის არქიტექტურა** — E.1–E.7
- **F. მონაცემის მიგრაცია და არსებული სისტემები** — F.1–F.9

A. სერვერები და ჰოსტინგი

A.1 რა არის სერვერი

1. ტექნიკური განმარტება. სერვერი არის კომპიუტერი (ფიზიკური ან ვირტუალური), რომელზეც მუდმივად გაშვებული პროგრამები ქსელში შემოსულ მოთხოვნებს ემსახურება. „სერვერი“ ორ რამეს ნიშნავს: მანქანას (host, VM) და პროგრამას (სერვისს), რომელიც ამ მანქანაზე კონკრეტულ **პორტზე** „უსმენს“, მაგალითად ვებ-სერვერი 443-ზე ან PostgreSQL 5432-ზე. დღეს სერვერების უმეტესობა **ვირტუალური მანქანაა (VM)**: ერთ ფიზიკურ მანქანაზე ჰიპერვიზორი (VMware, Hyper-V, KVM) რამდენიმე იზოლირებულ „კომპიუტერს“ უშვებს, საკუთარი CPU-ს, RAM-ის, დისკისა და IP-ის წილით.

2. მარტივად რომ ავხსნათ. სერვერი არის კომპიუტერი, რომელიც არასოდეს ითიშება და სხვებს ემსახურება. მე ჩემს კომპიუტერთან ვზივარ; სერვერთან არავინ ზის. ის სადღაც სერვერების ოთახშია და დღე-ღამე ელოდება, როდის ვინმე „დააკაკუნებს“ (გახსნის საიტს).

3. ჩვენს პროექტში. ✅ დღეს „სერვერი“ ჩემი კომპიუტერია: `run.bat` უშვებს პროგრამას uvicorn, რომელიც პორტ 8200-ზე უსმენს (`scripts/run_local.py`). 💡 საწარმოოში ეს იგივე პროგრამა სააგენტოს VM-ზე იმუშავებს.

4. რატომ გვჭირდება. ≈ 300 თანამშრომელმა სხვადასხვა ადგილიდან (ოფისი, ველი, ტელეფონი) ერთსა და იმავე მონაცემთან უნდა იმუშაოს, ნებისმიერ დროს. ამისთვის საჭიროა ერთი ადგილი, რომელიც ყოველთვის ჩართულია და ყველასთვის ხელმისაწვდომია.

5. როგორ მუშაობს კულისებში. ბრაუზერი DNS-ით ადგენს სერვერის IP-ს (მაგ. `qc.<დომენი>` → `10.x.x.x`), შემდეგ ამ IP-ის კონკრეტულ პორტთან TCP კავშირს ხსნის. სერვერის ოპერაციული სისტემა კავშირს იმ პროგრამას გადასცემს, რომელიც ამ პორტზე უსმენს. პროგრამა მოთხოვნას ამუშავებს და პასუხს აბრუნებს.

6. რა შეიძლება აირიოს.

- სერვერი გადაიტვირთა და პროგრამა თავისით არ ჩაირთო (სერვისად არ არის დაყენებული);
- დისკი გაივსო;
- Firewall-ი პორტს ბლოკავს;
- VM-ს რესურსი არ ჰყოფნის.

7. რას იკითხავს IT. „ფიზიკური სერვერი გჭირდებათ თუ VM? რა OS? რამდენი რესურსი?“

8. როგორ ვუპასუხო. „ფიზიკური არ გვჭირდება. ერთი VM საკმარისია, Linux ან Windows Server — თქვენი სტანდარტით. რესურსის შეფასება მოგვაქვს (დოკუმენტი 04, R1), მაგრამ ის შეფასებაა და პილოტზე გავზომავთ.“

A.2 რატომ არ შეიძლება ყველაფერი VS Code-იდან ან ერთი თანამშრომლის კომპიუტერიდან

1. ტექნიკური განმარტება. VS Code კოდის რედაქტორია (IDE) და არა გაშვების გარემო (runtime). ის კოდს წერს და შეუძლია მისი დროებით გაშვება დეველოპმენტისთვის. სამუშაო კომპიუტერი კი საწარმოო ჰოსტინგისთვის არ გამოდგება:

- არ აქვს მაღალი ხელმისაწვდომობა (ითიშება, იძინებს, ახლდება);
- მისი IP დინამიკურია;
- firewall და ანტივირუსი მის „სერვერობას“ ზღუდავს;
- არ აქვს ბექაპი, მონიტორინგი და ფიზიკური დაცვა;
- მომხმარებლის სესიაზეა დამოკიდებული (ვინც გამოვა სისტემიდან, პროგრამაც ჩერდება);
- უსაფრთხოების პოლიტიკით სამუშაო სადგური ქსელში სერვისს არ უნდა აწვდიდეს.

2. მარტივად რომ ავხსნათ. VS Code ჩემი „სახელოსნოა“, სადაც საიტს ვაწყობ. ჩემი კომპიუტერი კი „საცხოვრებელი სახლია“ და არა „მაღაზია“. როცა სახლში არ ვარ ან შუქი ქრება, მაღაზია იკეტება. ამიტომ საიტი ისეთ ადგილას უნდა იდგეს, რომელიც ყოველთვის ღიაა და დაცული.

3. ჩვენს პროექტში.

- ✅ `run_lan.bat` ტელეფონზე ჩვენებისთვის ჩემს კომპიუტერს ქსელში ხსნის, ოღონდ PIN-ით და დროებით. ფანჯარაში თავად წერია: „მხოლოდ მოკლე ჩვენებისთვის“ (`scripts/serve_lan.py`).
- ✅ ძველი განაწილების პროგრამა სწორედ ამ პრობლემის მაგალითია: ის ერთ კომპიუტერზეა და იმ კომპიუტერის გარეშე განაწილება ვერ ხდება.

4. რატომ გვჭირდება ეს გაგება. IT-მ შეიძლება იკითხოს: „დღეს ხომ მუშაობს, რატომ გჭირდებათ სერვერი?“ პასუხი: დღეს მუშაობს ერთი ადამიანისთვის, დემოსთვის, და არა 300 ადამიანისთვის, ყოველდღიურად.

5. კულისებში. კომპიუტერის ჩაძინება ან გადატვირთვა პროცესს კლავს. VPN-ის გათიშვა ქსელურ დისკს (`Z:`) აქრობს. Windows-ის განახლება ღამით გადატვირთავს. ჩემი პაროლის შეცვლა კი გაშვებულ სკრიპტებს წვდომას დაუკარგავს.

6. რა შეიძლება აირიოს. თუ ვინმე „დროებით“ ჩემი კომპიუტერიდან დაიწყებს მუშაობას:

- მონაცემი ერთ დისკზე იქნება, ბექაპის გარეშე;
- ჩემი შვებულება სისტემის შვებულება გახდება.

7. IT იკითხავს. „რატომ არ შეიძლება, უბრალოდ თქვენს კომპიუტერზე ეშვებოდეს და ქსელში გავხსნათ?“

8. პასუხი. „სამუშაო სადგური ხელმისაწვდომობას, ბექაპსა და უსაფრთხოებას ვერ უზრუნველყოფს. მასზე ჩვენ სისტემას ვაწყობთ და ვამოწმებთ; სამუშაო ვერსია თქვენს მართულ სერვერზე უნდა იდგეს.“

A.3 localhost, შიდა სერვერი, ღრუბელი და on-premises — რა განსხვავებაა

1. ტექნიკური განმარტება.

ტერმინი	რას ნიშნავს	ვინ მიწვდება
localhost (`127.0.0.1`)	„ეს კომპიუტერი“. ამ მისამართზე გაშვებული სერვისი მხოლოდ იმავე მანქანიდან იხსნება	მხოლოდ მე
LAN / შიდა მისამართი (`10.x` , `192.168.x` , `0.0.0.0` -ზე მოსმენა)	სერვისი ლოკალური ქსელის ინტერფეისზე უსმენს	იმავე შიდა ქსელში მყოფი ყველა
შიდა (on-premises) სერვერი	ორგანიზაციის საკუთარ მონაცემთა ცენტრში მდგარი, მისი IT-ის მიერ მართული მანქანა	შიდა ქსელი; გარედან — VPN-ით ან გამოქვეყნებით (reverse proxy / DMZ)
ღრუბელი (Render, AWS, Azure...)	სხვა კომპანიის მონაცემთა ცენტრში ქირით აღებული რესურსი	ინტერნეტიდან, თუ ასეა კონფიგურირებული
ჰიბრიდი	ნაწილი შიდაა, ნაწილი ღრუბელში	—

2. მარტივად რომ ავხსნათ.

- localhost ჩემი ოთახია: მხოლოდ მე ვარ შიგნით.
- LAN სართულია: ვინც ამ სართულზეა, შემოვა.
- on-premises სააგენტოს საკუთარი შენობაა, საკუთარი დაცვით.
- ღრუბელი ნაქირავები ოფისია სხვის შენობაში: მოსახერხებელია, მაგრამ გასაღები სხვასაც აქვს და შენობა სხვა ქვეყანაში შეიძლება იყოს.

3. ჩვენს პროექტში.

- ✅ `run.bat` → `127.0.0.1:8200` (localhost);
- ✅ `run_lan.bat` → `0.0.0.0:8200` (LAN, PIN-ით);
- ✅ ძველი სტატისტიკის პანელის `DEPLOY.md` აღწერს Render-სა და Neon-ს — ეს ღრუბელია, Frankfurt-ში;
- 💡 ახალი პლატფორმისთვის მიზანია on-premises, სააგენტოს სერვერი (`OPEN_QUESTIONS.md` #5).

4. რატომ არის მნიშვნელოვანი. QC მონაცემი შეიცავს თანამშრომლების სახელებს, QC კომენტარებს (ზოგჯერ მოქალაქეების სახელებით) და პირად ნომრებს. სახელმწიფო უწყებისთვის მონაცემის ადგილმდებარეობა და კონტროლი პოლიტიკისა და კანონის საკითხია.

5. კულისებში. იმისთვის, რომ ველიდან ტელეფონით შიდა სერვერზე შევიდეთ, ორი გზა არსებობს:

- **(ა) VPN** — ტელეფონი შიდა ქსელის ნაწილი ხდება;
- **(ბ) გამოქვეყნება** — reverse proxy ან სააგენტოს load balancer საჯარო HTTPS მისამართზე იღებს მოთხოვნას და შიდა სერვერს გადასცემს. ჩვეულებრივ, ეს DMZ-ში ხდება — ქსელის ნაწილში, რომელიც გარედანაც ჩანს და შიდა ქსელისგან გამოყოფილია.

6. რა შეიძლება აირიოს.

- მხოლოდ შიდა წვდომა → ველი ტელეფონიდან ვერ შედის;

- საჯარო წვდომა ზედმეტი დაცვის გარეშე → თავდასხმის ზედაპირი იზრდება.

7. IT იკითხავს. „საიტი გარედან უნდა იხსნებოდეს?“

8. პასუხი. „ველი ტელეფონით და პლანშეტით მუშაობს, ამიტომ გარედან წვდომა გვჭირდება: ან საჯარო HTTPS, ან VPN. რომელი, თქვენი პოლიტიკით გადაწყვიტეთ. აპლიკაცია ორივეზე მუშაობს (❓ IT-ის გადასაწყვეტია).“

A.4 IP, პორტი, პროცესი — როგორ „ესმის“ პროგრამას

1. ტექნიკური განმარტება.

- **IP მისამართი** მანქანის ქსელური მისამართია.
- **პორტი** (0–65535) კონკრეტული სერვისის „კარია“ ამ მანქანაზე. სტანდარტული პორტები: HTTP 80, HTTPS 443, PostgreSQL 5432, SQL Server 1433, SMTP 25/587, SSH 22, RDP 3389.
- **პროცესი** გაშვებული პროგრამაა. ის OS-ს სთხოვს, რომ კონკრეტულ IP:პორტზე „მოუსმინოს“ (bind/listen).
- `127.0.0.1`-ზე მოსმენა მხოლოდ ლოკალურ კავშირს იღებს, `0.0.0.0`-ზე მოსმენა კი ყველა ინტერფეისიდან.

2. მარტივად რომ ავხსნათ. IP შენობის მისამართია, პორტი ბინის ნომერი. ვებ-საიტი 443-ე ბინაშია, ბაზა 5432-ეში. თუ კარი (firewall) დაკეტილია, მისამართის ცოდნა არ გიშველის.

3. ჩვენს პროექტში.

- ✅ აპლიკაცია პორტ 8200-ზე უსმენს (`QCP_PORT`);
- 💡 საწარმოოში მომხმარებელი 443-ზე შევა (proxy), proxy კი 8200-ს მიმართავს შიგნით;
- 💡 PostgreSQL 5432-ზე იქნება და მას მხოლოდ აპლიკაციის სერვერი უნდა მიწვდეს.

4. რატომ. IT-ის firewall-ის წესები სწორედ „ვის → რომელ IP:პორტზე“ ფორმატით იწერება. ეს უნდა ვიცოდეთ, რომ მოთხოვნა სწორად ჩამოვაყალიბოთ.

5. კულისებში. Firewall პაკეტს წესების მიხედვით ატარებს ან აჩერებს. ეს წესები შეიძლება იყოს VM-ზე (Windows Firewall, `iptables` / `nftables`) და ქსელურ მოწყობილობაზეც.

6. რა შეიძლება აირიოს.

- „კავშირი ვერ დამყარდა“ (connection refused) — სერვისი არ უსმენს ან პორტი არასწორია;
- „timeout“ — firewall ჩუმად აგდებს პაკეტს.

7. IT იკითხავს. „რომელი პორტები უნდა გავხსნათ?“

8. პასუხი.

- „შემომავალი: მხოლოდ 443 proxy-ზე (და 80, HTTPS-ზე გადამისამართებისთვის).
- შიგნით: proxy → აპლიკაცია 8200, აპლიკაცია → PostgreSQL 5432.
- გამავალი: Google-ის OAuth-ი 443-ზე და SMTP relay (587 ან თქვენი პორტი).
- წყაროები: იმის მიხედვით, რა გზას აირჩევთ (view → 5432 ან 1433; ფაილი → SMB 445 ან SFTP 22).“

A.5 აპლიკაციის სერვერი (uvicorn) და worker-ები

1. ტექნიკური განმარტება. აპლიკაციის სერვერი არის პროცესი, რომელიც HTTP-ს იღებს და აპლიკაციის კოდს იძახებს. Python-ში ვებ-აპლიკაციები სტანდარტული ინტერფეისით ერთვება: WSGI (სინქრონული — Flask) ან **ASGI** (ასინქრონული — FastAPI). **uvicorn** ASGI სერვერია. **worker** აპლიკაციის ერთი ასლი-პროცესია. რამდენიმე worker პარალელურად ემსახურება მოთხოვნებს და რამდენიმე CPU ბირთვს იყენებს. ამის საშუალებას იძლევა `uvicorn --workers N` ან gunicorn + `uvicorn.workers.UvicornWorker` (gunicorn მხოლოდ Linux-ზე მუშაობს).

2. მარტივად რომ ავხსნათ. uvicorn რესტორნის „მიმტანია“: შეკვეთას იღებს, სამზარეულოს (ჩვენს კოდს) გადასცემს და კერძს მოაქვს. worker-ები მიმტანების რაოდენობაა. ერთი მიმტანი ბევრ მაგიდას ემსახურება, ოთხი კი პიკის საათზე უკეთ გაართმევს თავს.

3. ჩვენს პროექტში.

- ✅ დღეს ერთი uvicorn პროცესი ეშვება, `--reload`-ის გარეშე (`scripts/run_local.py`);
- ⚠️ დღეს რამდენიმე worker **ვერ** ჩაირთვება: განაწილების გადახედვა ერთი პროცესის მეხსიერებაში ინახება (Handoff G6). ჯერ ეს უნდა გასწორდეს.

4. რატომ. worker-ების რაოდენობა ნაწილობრივ განსაზღვრავს, რამდენ ერთდროულ მოთხოვნას გაუძლებს სისტემა.

5. კულისებში.

- ჩვენი მარშრუტების უმეტესობა სინქრონული ფუნქციაა (`def`). FastAPI ასეთ ფუნქციებს thread pool-ში უშვებს, ამიტომ ერთი worker-იც რამდენიმე მოთხოვნას ერთდროულად ამუშავებს;
- Python-ის GIL ერთ პროცესში CPU-ს ინტენსიურ სამუშაოს ზღუდავს. ბაზის მოლოდინი კი GIL-ს ათავისუფლებს;
- თითო worker ცალკე იტვირთება მეხსიერებაში. რამდენი MB სჭირდება თითოს, პილოტზე უნდა გავზომოთ.

6. რა შეიძლება აირიოს.

- ზედმეტად ბევრი worker → ბაზის კავშირები ამოიწურება;
- ზედმეტად ცოტა → რიგი იქმნება;
- მდგომარეობის მეხსიერებაში შენახვა (G6) → worker-ებს შორის აირევა.

7. IT იკითხავს. „რამდენ პროცესს გაუშვებთ? როგორ ირთვება გადატვირთვის შემდეგ?“

8. პასუხი. „საწყისად 2–4 worker-ს ვგეგმავთ, რაც პილოტზე დაზუსტდება. პროცესი სისტემურ სერვისად იქნება დაყენებული (Linux-ზე systemd, Windows-ზე სერვისი ან კონტეინერი), ავტომატური რესტარტით. რამდენიმე worker-ისთვის ერთ კომპონენტს ვასწორებთ, ესაა ჩვენი სამუშაო პილოტამდე.“

A.6 Reverse proxy

1. ტექნიკური განმარტება. Reverse proxy (nginx, Apache, IIS + ARR, HAProxy, ან სააგენტოს load balancer) არის სერვერი, რომელიც კლიენტის მოთხოვნას იღებს და შიდა აპლიკაციის სერვერს გადასცემს. მისი ფუნქციები:

- **TLS termination** — HTTPS-ის დაშიფვრა და გაშიფვრა;
- სტატიკური ფაილების გაცემა;
- მოთხოვნის ზომის და დროის ლიმიტები;
- წვდომის ჟურნალი;
- ბალანსირება რამდენიმე ასლს შორის;
- IP-ის შეზღუდვა;
- `X-Forwarded-For` / `X-Forwarded-Proto` თავსართების დამატება, რომ აპლიკაციამ იცოდეს კლიენტის ნამდვილი IP და პროტოკოლი.

2. მარტივად რომ ავხსნათ. reverse proxy შენობის მიმღებია. სტუმარი მას ხვდება; მიმღები ამოწმებს და სწორ კაბინეტში უშვებს. თავად კაბინეტი (ჩვენი აპლიკაცია) ქუჩიდან არ ჩანს.

3. ჩვენს პროექტში.

- ✅ დღეს proxy არ არის;
- 💡 საწარმოოში საჭიროა: HTTPS-ისთვის, ფოტოს ატვირთვის ზომის ლიმიტისთვის (აპლიკაცია ≤ 10 MB-ს ამოწმებს, proxy-ზეც ეს ზღვარი უნდა იყოს) და ლოგებისთვის;
- ⚠️ აპლიკაციამ proxy-ს თავსართები უნდა მიიღოს, თორემ cookie-ს `Secure` ნიშანი და აუდიტის IP არასწორი იქნება (Handoff G4).

4. რატომ. Python-ის აპლიკაციის სერვერს პირდაპირ ინტერნეტზე არ გამოაჩენენ: TLS-ის მართვა, ნელი კლიენტებისგან დაცვა და ლოგები proxy-ს საქმეა. IT-ს ხშირად ერთი სტანდარტული proxy აქვს ყველა საიტისთვის.

5. კულისებში.

- კლიენტი → `https://qc.<დომენი>:443` → proxy შიფრს ხსნის → `http://127.0.0.1:8200` → აპლიკაცია;
- proxy ამატებს `X-Forwarded-Proto: https`-ს და `X-Forwarded-For: <კლიენტის IP>`-ს;
- uvicorn ამ თავსართებს ენდობა მხოლოდ მაშინ, თუ გაშვებულია `--proxy-headers --forwarded-allow-ips=<proxy-ს IP>`-ით.

6. რა შეიძლება აირიოს.

- Google redirect URI-ში `http` აღმოჩნდება `https`-ის ნაცვლად → Google შესვლას უარყოფს (`redirect_uri_mismatch`). ამიტომ `QCP_BASE_URL` ცხადად უნდა მიეთითოს;
- ატვირთვა ჩავარდება 413-ით (proxy-ს ლიმიტი ნაკლებია 10 MB-ზე).

7. IT იკითხავს. „თქვენ დააყენებთ nginx-ს, თუ ჩვენს load balancer-ს გამოიყენებთ?“

8. პასუხი. „თქვენი სტანდარტი გვირჩევნია. ჩვენ გვჭირდება: HTTPS, მოთხოვნის ზომა ≥ 10 MB, `X-Forwarded-*` თავსართები და მათი IP, რომ აპლიკაციაში ნდობის სიაში ჩავწეროთ.“

A.7 ბაზის სერვერი

1. ტექნიკური განმარტება. მონაცემთა ბაზის მართვის სისტემა (DBMS, მაგ. PostgreSQL) ცალკე სერვერული პროცესია. ის მონაცემს დისკზე ინახავს და კლიენტებს ქსელით (TCP 5432) ემსახურება. მისი ფუნქციებია ტრანზაქციები, ერთდროული წვდომა, უფლებები და ბექაპი. **SQLite** ამის საპირისპიროდ **ჩაშენებული** ბიბლიოთეკაა: ბაზა ერთი ფაილია, რომელსაც აპლიკაციის პროცესი პირდაპირ კითხულობს. ცალკე სერვერი იქ არ არსებობს.

2. მარტივად რომ ავხსნათ.

- SQLite რვეულია, რომელიც ჩემს მაგიდაზე დევს: სწრაფია და მარტივი, მაგრამ ერთდროულად ერთი წერს.
- PostgreSQL არქივია, თავისი არქივარიუსით: ბევრი ადამიანი ერთდროულად ითხოვს და აბრუნებს საქაღალდეებს, არქივარიუსი კი წესრიგს იცავს.

3. ჩვენს პროექტში.

- ✅ დღეს SQLite: `data/platform.db`, WAL რეჟიმი, `foreign_keys=ON` (`app/db.py`);
- 💡 საწარმოოში PostgreSQL 16: ან იმავე VM-ზე, ან სააგენტოს DB სერვერზე.

4. რატომ. ერთდროული ჩაწერა, ბექაპის ინსტრუმენტები, უფლებების მართვა და მონიტორინგი სერვერულ DBMS-ს აქვს. დაწვრილებით: B.11.

5. კულისებში. აპლიკაცია PostgreSQL-ს connection string-ით უკავშირდება:

```
postgresql+psycopg://qc_app:<პაროლი>@db-host:5432/qc
```

SQLAlchemy კავშირების „აუზს“ (pool) ინახავს, რომ ყოველ მოთხოვნაზე ახალი კავშირი არ გაიხსნას.

6. რა შეიძლება აირიოს.

- კავშირების ლიმიტი (`max_connections`, ნაგულისხმევად 100) ამოიწურება, თუ worker-ები × pool ზედმეტია;
- ქსელის შეფერხება აპლიკაციასა და ბაზას შორის ყველა გვერდს ანელებს.

7. IT იკითხავს. „ბაზა ჩვენს არსებულ კლასტერზე იქნება, თუ ცალკე?“

8. პასუხი. „ორივე გამოდგება. ჩვენთვის მთავარია PostgreSQL 14+ (სასურველია 16), ცალკე ბაზა და ორი მომხმარებელი — აპლიკაციის და მიგრაციის — და ბექაპი PITR-ით. თუ თქვენი სტანდარტი SQL Server-ია, ვისაუბროთ (B.11).“

A.8 ფონური worker და განრიგით შესრულებადი ამოცანები

1. ტექნიკური განმარტება. ფონური worker ვებ-მოთხოვნისგან დამოუკიდებლად ასრულებს ხანგრძლივ ან პერიოდულ ამოცანებს: ღამის იმპორტს, მეილის რიგის დამუშავებას, გასუფთავებას, ანგარიშების მომზადებას. განრიგისთვის გამოიყენება cron/systemd timer (Linux), Task Scheduler (Windows), ან აპლიკაციის შიგნით scheduler (APScheduler და მისთ.). რიგისთვის — ბაზის ცხრილი (outbox), Redis ან სხვა message broker.

2. მარტივად რომ ავხსნათ. ვებ-გვერდი მყისიერ შეკვეთებს ემსახურება. ფონური worker ღამის ცვლაა: როცა ყველას სძინავს, ის ახალ მონაცემს ჩამოიტანს, ძველს დაალაგებს და დაგროვილ წერილებს გაგზავნის, ისე რომ დღისით ვინმე ლოდინში არ დატოვოს.

3. ჩვენს პროექტში.

- ✅ დღეს ფონური worker არ არის:
  - მონაცემი ხელით, სკრიპტებით მოდის (`fetch_sheets.py` , `build_db.py`);
  - მეილი მოთხოვნის შიგნით იგზავნება (`notify._send`);
  - ძველი presence ჩანაწერები ყოველი ping-ის დროს იშლება.
- 💡 საწარმოოში საჭიროა:
  - **(ა)** ღამის იმპორტი REGFIELDALL-იდან;
  - **(ბ)** ყოველდღიური სპორადულის იმპორტი, თუ view-ით მოვა;
  - **(გ)** მეილის რიგი;
  - **(დ)** ჟურნალების გასუფთავება.

4. რატომ. მომხმარებელს არ უნდა ელოდოს 2-მილიონიანი ცხრილის წაკითხვა ან SMTP-ის პასუხი. იმპორტის ჩავარდნა გვერდის ჩავარდნა არ უნდა გახდეს.

5. კულისებში. ამოცანა:

1. იწყება განრიგით (მაგ. 02:00);
2. `import_runs` -ში წერს „დაიწყო“-ს;
3. წყაროს კითხულობს, ბაზაში upsert-ს (არსებულის განახლება ან ახლის ჩამატება) აკეთებს;
4. წერს „დასრულდა: N ჩანაწერი“ ან „შეცდომა: ...“;
5. შეცდომისას ადმინს აცნობებს.

6. რა შეიძლება აირიოს.

- ორი ასლი ერთდროულად ეშვება → დუბლიკატი (საჭიროა ბლოკი: advisory lock);
- ამოცანა „გაიჭედა“ და შემდეგი ვერ იწყება (საჭიროა timeout; ძველ პანელში ეს მოხდა — `SYNC_STALE_AFTER`);
- ჩუმი ჩავარდნა: არავინ იცის, რომ მონაცემი ძველია.

7. IT იკითხავს. „რა ეშვება ღამით? რამდენ ხანს? რა დატვირთვას აძლევს წყაროს?“

8. პასუხი. „ერთხელ ღამით, read-only შეკითხვა საჭირო ველებზე (REGFIELDALL ≈ 2.3 მლნ ჩანაწერია; ჩვენ მხოლოდ 7 ველი გვჭირდება), ან უკვე მზა ფაილის წაკითხვა. ყოველი გაშვება ჟურნალში იწერება. დრო და დატვირთვა პილოტზე გაიზომება; თქვენთვის მისაღებ საათს თქვენ გვეტყვით.“

A.9 რა ხდება, როცა ბევრი თანამშრომელი ერთდროულად მუშაობს

1. ტექნიკური განმარტება. უნდა განვასხვავოთ:

- **მომხმარებლების საერთო რაოდენობა** (≈ 300);
- **ერთდროულად აქტიური მომხმარებელი** (შეფასება ≈ 40);
- **ერთდროული მოთხოვნები** (წამში რამდენი HTTP მოთხოვნა მოდის).

ადამიანი დროის უმეტეს ნაწილს კითხულობს ან წერს, მოთხოვნა კი მილიწამებში სრულდება. ამიტომ 40 აქტიური მომხმარებელი, ჩვეულებრივ, წამში რამდენიმე მოთხოვნას ნიშნავს და არა 40-ს. ერთდროულობის ორი რეალური საკითხია:

- **(ა) გამტარუნარიანობა** — worker-ები, ბაზის კავშირები, ნელი გვერდები, მაგ. სტატისტიკა;
- **(ბ) მონაცემის მთლიანობა** — ორი ადამიანი ერთ ჩანაწერს ცვლის. ამისთვის საჭიროა ტრანზაქციები, უნიკალურობის შეზღუდვები და ვერსია.

2. მარტივად რომ ავხსნათ. 300 ადამიანი ერთდროულად ღილაკს არ აჭერს. ისინი კითხულობენ, ფიქრობენ, წერენ. სერვერისთვის რთული ის მომენტებია, როცა ორი ადამიანი ზუსტად ერთსა და იმავე ოქმს ცვლის, ან როცა ბევრი ერთად ხსნის მძიმე გვერდს. სტატისტიკაა ასეთი.

3. ჩვენს პროექტში.

- ✅ ერთ ოქმზე ორი პასუხის წინააღმდეგ: `version` + „ახლა ხსნის“ ნიშანი;
- ✅ ერთი ოქმის ორჯერ განაწილების წინააღმდეგ: `uq_item_protocol_cad`;
- ✅ სტატისტიკა SQLite-ზე 2–3 წამს იტვირთება;
- ❓ დატვირთვის ტესტი არ ჩატარებულა.

4. რატომ. IT-ის პირველი კითხვა ხშირად ესაა: „100 ადამიანი რომ შევიდეს, არ ჩამოვარდება?“

5. კულისებში.

- თითო მოთხოვნა ცალკე DB სესიას იღებს (`db.get_db`) და ბოლოს ხურავს;
- PostgreSQL MVCC-ით მუშაობს: მკითხველები მწერლებს არ ბლოკავენ;
- SQLite-ში ერთდროულად ერთი მწერალია (WAL-ში მკითხველები არ ბლოკდებიან). ჩვენი ჩაწერები ხანმოკლეა, ამიტომ დღეს ეს საკმარისია, მაგრამ 300 მომხმარებლისთვის არა.

6. რა შეიძლება აირიოს.

- სტატისტიკის მძიმე შეკითხვები ერთდროულად → ბაზის CPU იზრდება;
- ბაზის კავშირების ამოწურვა;
- G5: ორი ერთდროული ცვლილება PostgreSQL-ზე, თუ ვერსიის შემოწმება არ გამკაცრდა.

7. IT იკითხავს. „100 ერთდროული მომხმარებელი?“

8. პასუხი. „100 ერთდროული მომხმარებელი, დიდი ალბათობით, წამში ათეულობით მოთხოვნაა, რაც ასეთი აპლიკაციისთვის ჩვეულებრივია. მაგრამ ამას ციფრით ვერ დავადასტურებ, სანამ დატვირთვის ტესტს არ გავაკეთებთ. პილოტამდე ტესტს ჩავატარებთ PostgreSQL-ზე, განსაკუთრებით სტატისტიკის გვერდზე, და შედეგს გაგიზიარებთ. მონაცემის მთლიანობას ცალკე მექანიზმები იცავს: უნიკალურობის შეზღუდვა, ტრანზაქცია და ვერსია.“

A.10 ცალკე ფიზიკური სერვერები ნამდვილად საჭიროა?

1. ტექნიკური განმარტება. აპლიკაციის სერვერი, ბაზის სერვერი, proxy და worker **ლოგიკური როლებია** და არა აუცილებლად ცალკე მანქანები. მცირე და საშუალო სისტემისთვის (ასეულობით მომხმარებელი) ყველა როლი ხშირად ერთ VM-ზეა, ან ორზე: აპლიკაცია + ბაზა. გაყოფის მიზეზებია:

- უსაფრთხოების ზონები (ბაზა უფრო ღრმა ქსელში);
- დამოუკიდებელი მასშტაბირება;
- სააგენტოს სტანდარტი, მაგ. ცენტრალიზებული DB კლასტერი;
- ხელმისაწვდომობა (რეპლიკა).

2. მარტივად რომ ავხსნათ. „მიმღები“, „სამზარეულო“ და „არქივი“ სამი სხვადასხვა ოთახი შეიძლება იყოს ერთ სახლში. სამი სახლი მხოლოდ მაშინ გვჭირდება, თუ სახლი ვიწროა ან უსაფრთხოების წესი ითხოვს.

3. ჩვენს პროექტში. 💡 ჩვენი მინიმალური მოთხოვნა ერთი VM-ია: აპლიკაცია + worker + (proxy) + PostgreSQL. სასურველი ვარიანტი: აპლიკაციის VM + სააგენტოს მართული PostgreSQL.

4. რატომ. IT-ს რესურსის დაზოგვა სურს; ზედმეტი მოთხოვნა ნდობას ამცირებს.

5. კულისებში. ერთ VM-ზე კომპონენტები ერთმანეთს `localhost`-ით უკავშირდება: უფრო სწრაფია და უფრო მარტივი. ცალკე VM-ზე საჭიროა firewall-ის წესი 5432-ზე და TLS ბაზასთან.

6. რა შეიძლება აირიოს. ერთ VM-ზე, თუ VM ჩავარდა, ყველაფერი ჩავარდება. ამიტომ ბექაპი VM-ის გარეთ უნდა ინახებოდეს.

7. IT იკითხავს. „რამდენი სერვერი?“

8. პასუხი. „მინიმუმ ერთი VM. თუ ბაზას თქვენს კლასტერზე მოგვცემთ, უკეთესია. ბექაპი აუცილებლად სხვა მედიაზე ან ლოკაციაზე უნდა იყოს.“

A.11 ოპერაციული სისტემა, runtime, Docker, სერვისი

1. ტექნიკური განმარტება.

- **OS** — Linux (Ubuntu LTS, RHEL/Rocky) ან Windows Server.
- **Runtime** — Python ≥ 3.12 და ბიბლიოთეკები (`requirements.txt`), იზოლირებულ ვირტუალურ გარემოში (`.venv`).
- **Docker / კონტეინერი** — აპლიკაცია და მისი runtime ერთ „image“-ში იფუთება, რომ ყველა სერვერზე ერთნაირად იმუშაოს.
- **სერვისი** — OS-ის მექანიზმი, რომელიც პროცესს ჩატვირთვისას უშვებს და ჩავარდნისას აღადგენს (systemd, Windows Service, Docker `restart: always`).

2. მარტივად რომ ავხსნათ. OS სახლის საძირკველია. Python ხელსაწყოების ყუთია. Docker მზა „კონტეინერია“, სადაც საიტი ყველა ხელსაწყოსთან ერთად დევს, ისე რომ ნებისმიერ საძირკველზე ერთნაირად დადგეს. სერვისი კი „ავტომატური ჩამრთველია“: სერვერი ჩაირთო და საიტიც თავისით ჩაირთო.

3. ჩვენს პროექტში.

- ✅ დღეს: Windows, `.venv` (uv-ით ან pip-ით), `run.bat`, Python 3.14.5;
- ✅ კოდი OS-ზე არ არის მიბმული: ბილიკები `pathlib`-ით და გარემოს ცვლადებით იწერება;
- ⚠️ თუმცა წყაროების ნაგულისხმევი ბილიკები ჩემს კომპიუტერზეა (`config.py`: `DISTRIBUTOR_DB`, `QC_WORKBOOK`) და `truststore` Windows-ის სერტიფიკატების საცავს იყენებს. Linux-ზე სისტემის საცავს გამოიყენებს, ამიტომ სააგენტოს შიდა CA იქ უნდა დაემატოს;
- 💡 Dockerfile ჯერ არ არის. მისი მომზადება ჩვენი სამუშაოა, თუ IT კონტეინერს აირჩევს.

4. რატომ. IT თავის სტანდარტს მოითხოვს; ჩვენ ორივეზე უნდა შეგვეძლოს მუშაობა.

5. კულისებში. Linux-ზე systemd unit-ის მაგალითი (ილუსტრაციაა და არა მზა ფაილი):

```
[Service]
User=qcapp
WorkingDirectory=/opt/qc-platforma
EnvironmentFile=/etc/qc-platforma/env      # QCP_* ცვლადები, საიდუმლოები — 600 უფლებით
ExecStart=/opt/qc-platforma/.venv/bin/uvicorn app.main:app --host 127.0.0.1 --port 8200 --workers 3 --proxy-headers --forwarded-
allow-ips=127.0.0.1
Restart=always
```

6. რა შეიძლება აირიოს.

- Python-ის ვერსია განსხვავდება და რომელიმე ბიბლიოთეკას იმ ვერსიისთვის build არ აქვს. ეს რეალურად მოხდა ძველ პანელზე: Render-მა Python 3.14 აიღო, `psycopg-binary`-ს კი build არ ჰქონდა (`DEPLOY.md`). 💡 ვერსიების ზუსტი ფიქსაცია (lock ფაილი) სასურველია;
- სერვერს ინტერნეტი არ აქვს და `pip install` ვერ ხერხდება. საჭიროა შიდა სარკე (mirror) ან მზა wheel-ები / Docker image.

7. IT იკითხავს. „Linux თუ Windows? Docker? ინტერნეტი სჭირდება ინსტალაციას?“

8. პასუხი. „ორივე OS-ზე იმუშავებს; Linux + Docker ყველაზე მარტივია მხარდაჭერისთვის, მაგრამ თქვენს სტანდარტს მივყვებით. ინსტალაციისას ბიბლიოთეკები უნდა ჩამოვიტვირთოთ: ან დროებითი წვდომა PyPI-ზე, ან შიდა სარკე, ან წინასწარ აწყობილ image-ს მოგაწვდით. მუშაობის დროს ინტერნეტი მხოლოდ Google-ით შესვლას სჭირდება.“

A.12 რესურსები: CPU, RAM, დისკი — როგორ შევაფასოთ

1. ტექნიკური განმარტება. რესურსის საჭიროება დატვირთვის პროფილით განისაზღვრება: მოთხოვნების სიხშირე, თითოს CPU დრო, მეხსიერების მოცულობა თითო worker-ზე, ბაზის მოცულობა და ზრდის ტემპი, ინდექსები, ბექაპის მოცულობა და შენახვის ვადა. საიმედო რიცხვი მხოლოდ **გაზომვით** მიიღება: დატვირთვის ტესტით (locust, k6) და მონიტორინგით.

2. მარტივად რომ ავხსნათ. ზუსტად რამდენი „ოთახი“ გვჭირდება, მაშინ გავიგებთ, როცა ხალხს შევუშვებთ და დავითვლით. ახლა შეგვიძლია გონივრული შეფასება ვთქვათ, მარაგით, და პილოტზე შევამოწმოთ.

3. ჩვენს პროექტში — რაც ✅ ვიცით:

გაზომილი	მნიშვნელობა
პლატფორმის ბაზა (SQLite), 44 159 შემთხვევა + 128 690 ადამიანის კავშირი + 45 765 კოდი + ისტორია	≈ 94 MB + WAL ≈ 4 MB
REGFIELDALL-ის ლოკალური ასლი (11 ველი, 2 331 017 ჩანაწერი)	≈ 1 GB (SQLite)
სტატისტიკის გვერდი	2–3 წმ (SQLite, ერთი მომხმარებელი)
ტესტების სრული გაშვება	≈ 6 წმ

შეფასება 💡 (არ არის გაზომილი, მხოლოდ გონივრული ვარაუდი):

- ახალ სისტემაში თითო შემთხვევას ისტორიის მეტი ჩანაწერი ექნება (5–10 მოვლენა, ახლა 1-ია). თუ წელიწადში 50–100 ათასი შემთხვევა დაემატება, აპლიკაციის ბაზა წელიწადში **≈ 0.5–2 GB** გაიზრდება. PostgreSQL ხშირად SQLite-ზე მეტ ადგილს იკავებს, ამიტომ ფართო დიაპაზონი დავტოვეთ;
- REGFIELDALL-ის ასლი, თუ ჩვენს ბაზაში ინახება: **≈ 1–2 GB**;
- ფოტოები: `IT_REQUEST.md`-ის შეფასებით წელიწადში **1–3 GB**;
- ამიტომ 100 GB დისკი რამდენიმე წელს ეყოფა ბექაპის გარეშე. ბექაპი ცალკე საცავშია.

4. რატომ. IT-ს ზუსტი ციფრი უნდა. გამოგონილი ციფრი ნდობას გვაკარგვინებს; შეფასება, მიზეზით, კი ნდობას გვმატებს.

5. კულისებში. CPU-ს ძირითადად სტატისტიკის აგრეგაცია და HTML-ის აწყობა მოიხმარს, RAM-ს — worker-ები და PostgreSQL-ის ქეში (`shared_buffers`). დისკის I/O ბაზის ჩაწერებითა და ბექაპით იტვირთება.

6. რა შეიძლება აირიოს. ბაზის ზრდა ისტორიისა და ჟურნალების გამო. ამისთვის საჭიროა `audit_log`-ისა და `presence`-ის შენახვის წესი, ასევე ბაზის „ბერვის“ (bloat) მართვა — PostgreSQL-ში ამას autovacuum აკეთებს.

7. IT იკითხავს. „რამდენი დისკი? რამდენი RAM?“

8. პასუხი. „საწყისი შეფასებაა 4 vCPU, 8 GB RAM და 100 GB SSD ერთ VM-ზე, ბაზის ჩათვლით. ეს შეფასებაა და არა გაზომვა. დღევანდელი ბაზა 44 ათას შემთხვევაზე 94 MB-ია, ამიტომ დისკი დიდი მარაგით გვაქვს მოთხოვნილი. პილოტზე გავზომავთ და, თუ ნაკლები გვეყოფა, შევამცირებთ.“

A.13 ჩავარდნა, კავშირის დაკარგვა, გადატვირთვა

1. ტექნიკური განმარტება.

სცენარი	ქცევა
პროცესის ჩავარდნა	სერვისის მენეჯერი (systemd / Docker) ხელახლა უშვებს. მიმდინარე მოთხოვნები იკარგება; დაუსრულებელი ტრანზაქციები ბაზა თავად აბრუნებს (rollback)
VM-ის გადატვირთვა	სერვისები ჩატვირთვისას ირთვება. PostgreSQL ჩამოწერილ WAL-ს „თავიდან ატარებს“ (crash recovery), ამიტომ დადასტურებული (commit) მონაცემი არ იკარგება
ქსელის დაკარგვა წყაროსთან	იმპორტი ჩავარდება და ჟურნალში ჩაიწერება; საიტი ბოლო ჩატვირთულ მონაცემზე მუშაობს
ქსელის დაკარგვა მომხმარებელთან	გვერდი ვერ იხსნება; გაუგზავნელი ფორმა იკარგება (ბრაუზერის მხარეს)

2. მარტივად რომ ავხსნათ. თუ სერვერი ჩაქრა და ისევ ჩაირთო, ყველაფერი, რაც „შენახვა“ ღილაკით უკვე შეინახა, ადგილზე დახვდება. იკარგება მხოლოდ ის, რაც ზუსტად იმ წამს იგზავნებოდა. მომხმარებელს ხელახლა მოუწევს ღილაკზე დაჭერა.

3. ჩვენს პროექტში.

- ✅ ყოველი მოქმედება ერთ ტრანზაქციაშია (`db.commit()` , შეცდომისას `rollback`) — `routes/items.action` ;
- ✅ სესიები ბაზაში ინახება, ამიტომ სერვერის გადატვირთვის შემდეგ მომხმარებელი შესული რჩება (12 სთ-მდე);
- ⚠️ განაწილების გადახედვა მეხსიერებაშია და რესტარტზე იკარგება. QC მენეჯერს გადახედვის ხელახლა გაშვება მოუწევს (დადასტურებული განაწილება კი ბაზაშია და არ იკარგება);
- ✅ ძველი Distributor-იც ქსელის მიუწვდომლობისას ბოლო ჩატვირთულ მონაცემს იყენებს — იგივე პრინციპი.

4. რატომ. IT-ს აინტერესებს, რა დაიკარგება ინციდენტის დროს (RPO) და რამდენ ხანში აღდგება (RTO). იხ. A.16.

5. კულისებში. PostgreSQL ჯერ WAL-ში (write-ahead log) წერს და მერე მონაცემის ფაილებში. commit დაბრუნდება მხოლოდ მაშინ, როცა WAL დისკზე ჩაიწერა (`fsync`). ამიტომ commit-ის შემდეგ ჩავარდნაც კი მონაცემს არ კარგავს.

6. რა შეიძლება აირიოს.

- დისკის ფიზიკური დაზიანება — სწორედ ამისთვის არის ბექაპი;
- მეილი, რომელიც „იგზავნებოდა“, როცა პროცესი ჩავარდა (G7) — outbox-ით გამოსწორდება.

7. IT იკითხავს. „სერვერი რომ ჩავარდეს, რა იკარგება?“

8. პასუხი. „დადასტურებული ცვლილება არ იკარგება: ის ბაზაშია და ბაზა ტრანზაქციულია. იკარგება მხოლოდ ის მოთხოვნა, რომელიც იმ მომენტში მუშავდებოდა. დისკის დაზიანებისთვის ბექაპი და PITR გვჭირდება, ამიტომ ვითხოვთ.“

A.14 განთავსება და განახლება (deployment)

1. ტექნიკური განმარტება. Deployment არის კოდის ახალი ვერსიის სერვერზე გადატანა და გაშვება. ტიპური ნაბიჯებია:

1. ვერსიის მომზადება (git tag ან image);
2. ტესტები;
3. ბაზის ბექაპი;
4. მიგრაციის გაშვება (Alembic);
5. ახალი კოდის გაშვება;
6. შემოწმება (health check, smoke test);
7. საჭიროებისას დაბრუნება (rollback).

გარემოები: **dev** (ჩემი კომპიუტერი) → **test/staging** (სატესტო სერვერი, ასლი ან ანონიმიზებული მონაცემი) → **production**.

შეფერხების გარეშე განახლების ხერხები:

- **rolling restart** — worker-ების მორიგეობით გადატვირთვა;
- **blue/green** — ორი ასლი, proxy გადართვა;
- მიგრაციები, რომლებიც ძველსა და ახალ კოდთან ერთდროულად თავსებადია (expand → migrate → contract).

2. მარტივად რომ ავხსნათ. ახალი ვერსია ჯერ „სარეპეტიციო დარბაზში“ მოწმდება. მერე, ღამით ან შესვენებისას, სამუშაო სერვერზე იცვლება. ძველი ვერსია მზად გვაქვს, რომ რამე თუ არ გამოვა, უკან დავაბრუნოთ. პატარა განახლებას მომხმარებელი ვერც შეამჩნევს.

3. ჩვენს პროექტში.

- ✅ დღეს deployment არ არსებობს: კოდი ჩემს კომპიუტერზეა;
- ⚠️ პროექტის საქაღალდე ჯერ git-ის რეპოზიტორია არ არის (აუდიტისას: `not a git repository`). `.gitignore` მზადაა და `data/`-ს გამორიცხავს;
- 💡 საჭიროა:
  - **(ა)** ვერსიის კონტროლი (სააგენტოს GitLab/GitHub/Azure DevOps ან სხვა);
  - **(ბ)** Alembic (G2);
  - **(გ)** სატესტო გარემო;
  - **(დ)** განახლების წერილობითი პროცედურა.

4. რატომ. IT ცვლილებების მართვის (change management) წესს ითხოვს: ვინ, როდის, რას აყენებს და როგორ ბრუნდება უკან.

5. კულისებში. ჩვენი სისტემა მცირე შეფერხებას იტანს: სესია ბაზაშია, ამიტომ რესტარტი მომხმარებელს არ „აგდებს“. რამდენიმე worker-ით rolling restart უკვე შეუმჩნეველი იქნება.

6. რა შეიძლება აირიოს.

- მიგრაცია წარუმატებელია → საჭიროა ბექაპი და დაბრუნების გეგმა;
- ახალი კოდი ძველ სქემას ელოდება;
- „სერვერზე ხელით შეცვლილი ფაილი“ — ასეთი ცვლილება არასოდეს უნდა ხდებოდეს.

7. IT იკითხავს. „განახლებას ვინ აყენებს — თქვენ თუ ჩვენ? შეფერხება იქნება?“

8. პასუხი. „გვინდა წერილობითი პროცედურა, რომელიც ორივე მხარეს შეეძლება. ჩვენ ვამზადებთ ვერსიას და მიგრაციას, თქვენი წესით ვაყენებთ, ჯერ სატესტო გარემოზე. ჩვეულებრივი განახლება წამებს გრძელდება და სესიები არ იკარგება. დიდი ცვლილება შეთანხმებულ ფანჯარაში ჩატარდება.“

A.15 მონიტორინგი და ლოგები

1. ტექნიკური განმარტება.

- **მონიტორინგი** — მეტრიკების მუდმივი შეგროვება და განგაში: CPU, RAM, დისკი, პასუხის დრო, შეცდომების სიხშირე, health check, ბაზის კავშირები, ბექაპის წარმატება, იმპორტის წარმატება.
- **ლოგები** — მოვლენების ჩანაწერები: წვდომის ლოგი (proxy), აპლიკაციის ლოგი (შეცდომები, stack trace), აუდიტის ლოგი (ვინ რა მოქმედება გააკეთა — ჩვენთან ბაზაშია).
- ცენტრალიზებული სისტემები: Zabbix, Prometheus + Grafana, ELK/OpenSearch, Graylog, Windows Event Log.

2. მარტივად რომ ავხსნათ. მონიტორინგი მანქანის დაფაა: სიჩქარე, საწვავი, ძრავის ტემპერატურა. ლოგი კი „შავი ყუთია“: როცა რამე მოხდა, ვკითხულობთ, რა და როდის მოხდა.

3. ჩვენს პროექტში.

- ✅ `GET /api/health` → `{"status": "ok"}` (ან `"empty"`, თუ ბაზა ცარიელია);

- ✅ `audit_log` ცხრილი ჩანს ადმინისტრირება → აუდიტში; `import_runs` — იმპორტის ანგარიშებისთვის;
- ⚠️ აპლიკაციის ლოგი მინიმალურია (`log_level="warning"`), სტრუქტურირებული ლოგი არ არის (G16);
- ⚠️ `health` ბაზის მიუწვდომლობისას შეცდომას (500) დააბრუნებს. Pre-QC hosted-ში ეს შეგნებულად `200 + degraded` -ით არის გადაწყვეტილი, რომ health-check-მა აპლიკაცია ზედმეტად არ გადატვირთოს. ჩვენთვის 💡 გასათვალისწინებელია.

4. რატომ. პრობლემა მომხმარებელზე ადრე უნდა დავინახოთ. „საიტი ნელია“ ციფრით უნდა შევამოწმოთ.

5. კულისებში. მონიტორინგის სისტემა ყოველ წუთს `/api/health` -ს მიმართავს. პასუხი თუ არ მოვიდა, მორიგეს ატყობინებს. ლოგები ფაილში იწერება ბრუნვით (rotation), ან ცენტრალურ სისტემას ეგზავნება.

6. რა შეიძლება აირიოს.

- ლოგში საიდუმლო ან პირადი მონაცემი ჩაიწერება. მაგალითად, OAuth-ის `code` query string-ში — Pre-QC hosted-მა ეს რეალურად აღმოაჩინა და `_RedactQueryString` -ით მოაგვარა. ჩვენთვისაც ეს წესი უნდა იყოს;
- ლოგები დისკს ავსებს.

7. IT იკითხავს. „თქვენ რა მონიტორინგს გვთავაზობთ? ჩვენს სისტემაში ჩაერთვება?“

8. პასუხი. „თქვენს არსებულ სისტემაში ჩართვა გვირჩევნია. ჩვენ მოგცემთ health endpoint-ს, ლოგებს ფაილში ან stdout-ში და აპლიკაციის მეტრიკებს. ლოგში პაროლი, token და პირადი ნომერი არ ჩაიწერება.“

A.16 აღდგენა (recovery): RPO და RTO

1. ტექნიკური განმარტება.

- **RPO** (Recovery Point Objective) — რა მაქსიმალური პერიოდის მონაცემის დაკარგვაა დასაშვები.
- **RTO** (Recovery Time Objective) — რამდენ ხანში უნდა აღდგეს სისტემა.

ღამის `pg_dump` RPO-ს ≤ 24 საათამდე ამცირებს. PITR (base backup + WAL არქივი) RPO-ს წუთებამდე ამცირებს. აღდგენის გეგმა უნდა იყოს დოკუმენტირებული და **პერიოდულად გამოცდილი** (restore test).

2. მარტივად რომ ავხსნათ.

- RPO ნიშნავს: „რამდენი სამუშაოს დაკარგვას ვიტანთ?“ — ერთი დღის? ერთი საათის?
- RTO ნიშნავს: „რამდენ ხანს შეგვიძლია უსისტემოდ ვიყოთ?“ — ერთი საათი? ერთი დღე?

რაც უფრო მცირეა ეს ციფრები, მით მეტი ღირს. ამიტომ ციფრები ხელმძღვანელობამ და IT-მ ერთად უნდა გადაწყვიტონ.

3. ჩვენს პროექტში.

- ✅ დღეს ბექაპი არ კეთდება;
- ✅ ერთადერთი „აღდგენა“ `build_db.py` -ია, რომელიც ძველი ცხრილიდან ხელახლა აწყობს ბაზას. საწარმოოში ეს აღდგენა არ არის, რადგან ახალ სისტემაში შექმნილ სამუშაოს ძველი ცხრილი აღარ შეიცავს;
- 💡 შეთავაზება: RPO ≤ 15 წთ (PITR), RTO ≤ 4 სთ სამუშაო დღეს. ❓ დასამტკიცებელია.

4. რატომ. QC-ის ისტორია სამუშაოს მტკიცებულებაა: ვინ რა გადაწყვიტა. მისი დაკარგვა დღევანდელი ცხრილის დაკარგვაზე უარესია.

5. კულისებში. აღდგენისას:

1. base backup აღდგება;
2. WAL „ითამაშება“ საჭირო წუთამდე;
3. აპლიკაცია ახალ ბაზას უკავშირდება;
4. ფოტოები ცალკე, ფაილების ბექაპიდან აღდგება.

6. რა შეიძლება აირიოს.

- ბექაპი კეთდებოდა, მაგრამ არასოდეს შემოწმებულა და აღდგენისას აღმოჩნდა დაზიანებული;
- ბაზა აღდგა, ფოტოები კი არა.

7. IT იკითხავს. „რა RPO/RTO გჭირდებათ?“

8. პასუხი. „ვთავაზობთ RPO 15 წუთს და RTO 4 საათს. თუ თქვენი სტანდარტული ვარიანტი ღამის ბექაპია (RPO 24 სთ), პილოტისთვის მისაღებია, მაგრამ სრული გაშვებისთვის PITR გვინდა. აღდგენის ტესტი წელიწადში ერთხელ მაინც უნდა ჩატარდეს.“

B. SQL და მონაცემთა ბაზები

B.1 რატომ სჭირდება პლატფორმას ბაზა

1. ტექნიკური განმარტება. მონაცემთა ბაზა (DBMS) მონაცემის მუდმივ (persistent) შენახვას, ერთდროულ უსაფრთხო წვდომას, მთლიანობის წესებს (constraints), ტრანზაქციებს, სწრაფ ძებნას (ინდექსები), უფლებებს და აღდგენას უზრუნველყოფს. ვებ-აპლიკაცია თავად „უმეხსიერო“ (stateless) უნდა იყოს: მისი პროცესი ნებისმიერ დროს შეიძლება გადაიტვირთოს, მდგომარეობა კი ბაზაში ცხოვრობს.

2. მარტივად რომ ავხსნათ. PostgreSQL არის ცენტრალური, დაცული ადგილი, სადაც საიტი ყველა ინფორმაციას ინახავს და ალაგებს. საიტი თავად მხოლოდ „ფანჯარაა“; ყველაფერი, რაც უნდა დარჩეს, ბაზაში იწერება. საიტი რომ გადაიტვირთოს, ინფორმაცია ადგილზე რჩება.

3. ჩვენს პროექტში. ბაზაში ინახება:

- ოქმები (`qc_items`) და მათი ამზომველები და მენეჯერები (`item_people`);
- ხარვეზის კოდები (`qc_item_codes`);
- ყოველი ნაბიჯის ისტორია (`qc_events`);
- მომხმარებლები და როლები (`users` , `user_roles`);
- გუნდები (`teams` , `team_members`);
- კომენტარები: QC-ის `qc_comment` , ველის `field_comment` , ისტორიაში `qc_events.comment` ;
- სტატუსის მოთხოვნები (`verdict_requests`);
- შეფასებები (`qc_reviews`);
- მეილები (`emails`);
- განაწილების ისტორია;
- სესიები და აუდიტი.

4. რატომ გვჭირდება. ≈ 300 ადამიანი ერთსა და იმავე ჩანაწერს ხედავს და ცვლის. ყველამ ერთი და იგივე, ბოლო მდგომარეობა უნდა დაინახოს. ცვლილება არ უნდა დაიკარგოს და ორმა ადამიანმა ერთმანეთის სამუშაო არ უნდა „გადაწეროს“.

5. კულისებში. იხ. B.7 (ტრანზაქცია) და B.8 (ერთდროულობა).

6. რა შეიძლება აირიოს.

- ბაზის გარეშე, მაგალითად ფაილებში ან ცხრილში: ერთდროული ჩაწერისას მონაცემი იკარგება, ძებნა ნელია, წესს ვერავინ იცავს;
- ბაზით, მაგრამ ბექაპის გარეშე: დისკის ერთი დაზიანება ყველაფერს წაშლის.

7. IT იკითხავს. „რატომ არ გამოიყენებთ ჩვენს არსებულ ბაზას?“

8. პასუხი. „არსებულ სისტემებს მხოლოდ ვკითხულობთ და მათში არაფერს ვწერთ — ეს პრინციპია. ჩვენს სამუშაო მონაცემს (სტატუსები, ისტორია, კომენტარები) ცალკე ბაზა სჭირდება, რომ თქვენს სისტემებზე არ ვიმოქმედოთ. თუ თქვენს DB სერვერზე ცალკე ბაზას ან სქემას მოგვცემთ, ესეც კარგი ვარიანტია.“

B.2 ცხრილი, მწკრივი, სვეტი — რელაციური მოდელი

1. ტექნიკური განმარტება. რელაციური ბაზა მონაცემს **ცხრილებად** (relation) ინახავს:

- **სვეტი** (column, attribute) — ტიპიზებული ველი, მაგ. `INTEGER`, `VARCHAR(32)`, `TIMESTAMP`, `BOOLEAN`, `JSON`;
- **მწკრივი** (row, tuple) — ერთი ჩანაწერი.

ცხრილები ერთმანეთს გასაღებებით უკავშირდება. მონაცემი **ნორმალიზებულია**: ერთი ფაქტი ერთ ადგილას ინახება. შეკითხვები SQL ენით იწერება.

2. მარტივად რომ ავხსნათ. ცხრილი Excel-ის ფურცელს ჰგავს, მაგრამ უფრო მკაცრია. ყოველ სვეტს წინასწარ აქვს განსაზღვრული, რა შეიძლება ჩაიწეროს: თარიღის სვეტში თარიღი, რიცხვის სვეტში რიცხვი. და Excel-ისგან განსხვავებით, სახელი ასჯერ კი არ იწერება, არამედ ერთხელ, თავის ცხრილში. სხვაგან მხოლოდ მისი ნომერი დგას.

3. ჩვენს პროექტში. ძველ ცხრილში F სვეტში ამზომველის სახელი ოთხი სხვადასხვა ფორმით იწერებოდა: მხოლოდ სახელი; სახელი + ტაბულაცია + პირადი ნომერი; სახელი + რამდენიმე ჰარი + პირადი ნომერი; სახელი + ერთი ჰარი + პირადი ნომერი (`qc-dashboard-main/HANDOFF.md`, §6). შედეგად 661 ნედლი მნიშვნელობიდან 276 რეალური ადამიანი გამოვიდა. ✅ ახალ ბაზაში ადამიანი ერთხელ წერია `users`-ში, ოქმზე კი მხოლოდ `user_id` ინახება (`item_people.user_id`).

4. რატომ. სახელის შეცდომა ერთ ადგილას სწორდება და ყველგან აისახება. სტატისტიკა „ერთი ადამიანი = ერთი ჩანაწერი“ პრინციპით ითვლება.

5. კულისებში. `app/models.py`-ში თითო კლასი ერთი ცხრილია:

```
class QcItem(Base):
    __tablename__ = "qc_items"
    id = mapped_column(Integer, primary_key=True)
    protocol_number = mapped_column(String(32))
    cadcode = mapped_column(String(64))
    state = mapped_column(String(24), default="pending", index=True)
    ...
```

SQLAlchemy ამ აღწერიდან `CREATE TABLE`-ს აგენერირებს.

6. რა შეიძლება აირიოს. არასწორი ტიპი. მაგალითად, ოქმის ნომერი წყაროში `float`-ად მოდის (`6150001.0`). ✅ ამიტომ კოდი მას ტექსტად გარდაქმნის: `str(int(round(float(pid))))` (`source.py`).

7. IT იკითხავს. „სქემა გაქვთ? ER დიაგრამა?“

8. პასუხი. „სქემა `app/models.py`-შია, 27 ცხრილი. დიაგრამა და ცხრილების აღწერა Handoff-ის §4.4-შია. ბაზიდან ავტომატურად აგებული ER დიაგრამის მოწოდებაც შეგვიძლია.“

B.3 Primary key (პირველადი გასაღები)

1. ტექნიკური განმარტება. Primary key არის სვეტი ან სვეტების კომბინაცია, რომელიც მწკრივს ცალსახად განსაზღვრავს: უნიკალური და `NOT NULL`. ჩვეულებრივ, ეს ხელოვნური (surrogate) მთელი რიცხვია (`id`), რომელიც ბაზა ავტომატურად ზრდის (PostgreSQL-ში `IDENTITY` / `SERIAL`). შეიძლება იყოს **ბუნებრივი** (natural) გასაღებიც, მაგ. `defect_codes.code = 'ATR-04'`, ან **შედგენილი** (composite), მაგ. `user_roles (user_id, role)`.

2. მარტივად რომ ავხსნათ. Primary key ჩანაწერის „პირადი ნომერია“. ორ ჩანაწერს ერთი ნომერი ვერ ექნება, და ამ ნომრით ნებისმიერი ჩანაწერი მყისიერად მოიძებნება.

3. ჩვენს პროექტში. ✅

- `qc_items.id` — შემთხვევის ნომერი, რომელიც მისამართშიც ჩანს: `/items/4512`;
- `user_roles` — შედგენილი გასაღები `(user_id, role)`: ერთ ადამიანს ერთი როლი ორჯერ ვერ ექნება;

- `qc_reviews.item_id` — primary key-ც არის და foreign key-ც. ამიტომ ერთ შემთხვევაზე მხოლოდ ერთი მიმდინარე შეფასება არსებობს;
- `sessions.sid` — შემთხვევითი ტექსტია (`secrets.token_urlsafe(32)`), რომ გამოცნობა შეუძლებელი იყოს.

4. რატომ. ყველა კავშირი და ყველა ბმული ამ ნომერს ეყრდნობა.

5. კულისებში. primary key-ზე ბაზა ავტომატურად უნიკალურ ინდექსს ქმნის.

6. რა შეიძლება აირიოს. ბიზნეს-მნიშვნელობის primary key-დ გამოყენება. მაგალითად, ოქმის ნომერი ერთი შემთხვევისთვის საკმარისი **არ არის**: ერთ ოქმში რამდენიმე საკადასტრო კოდი შეიძლება იყოს. ამიტომ ბიზნეს-უნიკალურობა ცალკე შეზღუდვაა (B.6).

7. IT იკითხავს. „რა არის შემთხვევის ბიზნეს-გასაღები?“

8. პასუხი. „ოქმის ნომერი + საკადასტრო კოდი. ბაზაში ეს უნიკალურობის შეზღუდვაა `uq_item_protocol_cad`, ხოლო ტექნიკური გასაღები `id`-ია.“

B.4 Foreign key და კავშირები

1. ტექნიკური განმარტება. Foreign key (FK) არის სვეტი, რომელიც სხვა ცხრილის primary key-ზე მიუთითებს, და ბაზა ამ კავშირის მთლიანობას იცავს (referential integrity): არარსებულ ჩანაწერზე მითითება შეუძლებელია. `ON DELETE CASCADE` მშობლის წაშლისას შვილსაც შლის, `SET NULL` მითითებას ასუფთავებს, ნაგულისხმევი `RESTRICT` / `NO ACTION` კი წაშლას კრძალავს. კავშირების ტიპები:

- **1:N** — ერთს ბევრი;
- **N:M** — ბევრი-ბევრთან, შუალედური ცხრილით;
- **1:1**.

2. მარტივად რომ ავხსნათ. foreign key არის „ბმული“ ერთი ცხრილიდან მეორეზე. ოქმზე წერია „ოპერატორი №17“, და ბაზა არ დაუშვებს ოპერატორ №999-ის ჩაწერას, თუ ასეთი ადამიანი არ არსებობს.

3. ჩვენს პროექტში. ✅

კავშირი	ტიპი	როგორ
შემთხვევა → ოპერატორი	N:1	`qc_items.operator_id → users.id`
შემთხვევა → ისტორია	1:N	`qc_events.item_id → qc_items.id`
შემთხვევა ↔ ამზომველები/ მენეჯერები	**N:M**	შუალედური ცხრილი `item_people (item_id, user_id, kind, is_primary)`
მომხმარებელი ↔ როლი	1:N	`user_roles`
მომხმარებელი ↔ გუნდი	N:M თარიღებით	`team_members (team_id, user_id, valid_from, valid_to)`
შემთხვევა → შეფასება	1:1	`qc_reviews.item_id` (PK = FK)

რატომ N:M ამზომველზე: ოქმს წყვილი ზომავს. ძველ ცხრილში ოქმების 93.6% წყვილით იყო (`qc-dashboard-main/HANDOFF.md`), Pre-QC-ის 31 რეალურ ოქმზე 61%-ს ორი შემსრულებელი ჰყავდა (`hosted/README.md`). ერთი `surveyor_id` სვეტი ამას ვერ აღწერდა.

4. რატომ. „ობოლი“ ჩანაწერი (ისტორია არარსებულ ოქმზე) ბაზის დონეზე შეუძლებელი ხდება.

5. კულისებში. SQLite-ში FK-ის შემოწმება ნაგულისხმევად **გამორთულია**. ✅ ამიტომ `app/db.py` თითო კავშირზე `PRAGMA foreign_keys=ON`-ს უშვებს. PostgreSQL-ში ეს ყოველთვის ჩართულია.

6. რა შეიძლება აირიოს.

- `CASCADE` არასწორ ადგილას: მომხმარებლის წაშლა მთელ ისტორიას წაშლიდა. ✅ ამიტომ `qc_events.actor_id`-ზე cascade არ არის;
- 💡 თანამშრომელს არასოდეს ვშლით, მხოლოდ `active = false`-ს ვუსვამთ.

7. IT იკითხავს. „მომხმარებლის წაშლისას რა ხდება მის ჩანაწერებთან?“

8. პასუხი. „მომხმარებელს არ ვშლით, ვთიშავთ (`active = false`). შესვლა მაშინვე ეკეტება, ისტორიაში კი მისი სახელი რჩება — ეს აუდიტისთვის აუცილებელია.“

B.5 ინდექსი

1. ტექნიკური განმარტება. ინდექსი არის დამატებითი მონაცემთა სტრუქტურა (ჩვეულებრივ B-tree), რომელიც სვეტის მნიშვნელობით მწკრივის პოვნას ლოგარითმულ დროში უზრუნველყოფს და ამიტომ მთელი ცხრილის წაკითხვა (sequential scan) არ სჭირდება. ფასი: ყოველი ჩაწერა ინდექსსაც ანახლებს, და ინდექსი დისკზე ადგილს იკავებს. შედგენილი ინდექსი `(a, b)` ეხმარება ფილტრს `a`-ზე და `a + b`-ზე. რომელ ინდექსს იყენებს შეკითხვა, `EXPLAIN` / `EXPLAIN ANALYZE` აჩვენებს.

2. მარტივად რომ ავხსნათ. ინდექსი წიგნის ბოლოს არსებული საძიებელია. „ATR-04 — გვ. 37“: წიგნს თავიდან ბოლომდე აღარ ვფურცლავთ.

3. ჩვენს პროექტში. ✅ ბაზაში 39 ინდექსია, მათ შორის:

```
-- ოპერატორის „გასავლელი": operator_id = ? AND state = 'pending'
CREATE INDEX ix_item_operator_state ON qc_items (operator_id, state);
-- მენეჯერის სია
CREATE INDEX ix_item_manager_state  ON qc_items (field_manager_id, state);
-- ძებნა ოქმით და კოდით
CREATE INDEX ix_item_protocol ON qc_items (protocol_number);
CREATE INDEX ix_item_cadcode  ON qc_items (cadcode);
-- ამზომველი/მენეჯერი ოქმებში
CREATE INDEX ix_people_user_kind ON item_people (user_id, kind);
```

4. რატომ. ოპერატორის სია 44 ათას შემთხვევაში მხოლოდ მისას პოულობს, მილიწამებში.

5. კულისებში. შეკითხვის გეგმის ნახვა (PostgreSQL):

```
EXPLAIN ANALYZE
SELECT id, protocol_number FROM qc_items
WHERE operator_id = 17 AND state = 'pending';
```

მარტივად: ეს ბრძანება ბაზას ეკითხება: „როგორ აპირებ ამის მოძებნას და რამდენ ხანს მოუნდი?“ კარგ პასუხში წერია `Index Scan using ix_item_operator_state`, ცუდში — `Seq Scan` (მთელი ცხრილის წაკითხვა).

6. რა შეიძლება აირიოს. სტატისტიკის შეკითხვებს ბევრი სხვადასხვა ფილტრი აქვს, და ყველა კომბინაციისთვის ინდექსს ვერ შევქმნით. ⚠️ სტატისტიკის 2–3 წამი (G14) PostgreSQL-ზე `EXPLAIN`-ით უნდა გავაანალიზოთ.

7. IT იკითხავს. „ინდექსები გაქვთ? რომელ შეკითხვებზე?“

8. პასუხი. „კი, მთავარ სიებზე. სტატისტიკის შეკითხვებს PostgreSQL-ზე `EXPLAIN`-ით შევამოწმებთ და, საჭიროებისას, ინდექსს ან წინასწარ დათვლილ შეჯამებას დავამატებთ.“

B.6 უნიკალურობის და შემოწმების შეზღუდვები — დუბლიკატების წინააღმდეგ

1. ტექნიკური განმარტება.

- `UNIQUE` შეზღუდვა კრძალავს ორ მწკრივს ერთნაირი მნიშვნელობით (ან კომბინაციით);
- `CHECK` შეზღუდვა მნიშვნელობის წესს ამოწმებს;

- `NOT NULL` ცარიელ მნიშვნელობას კრძალავს.

შეზღუდვას ბაზა თავად ამოწმებს, ტრანზაქციის შიგნით. ამიტომ ის მაშინაც მუშაობს, როცა ორი პროცესი ერთდროულად ცდილობს ჩაწერას. აპლიკაციის დონის „ჯერ შევამოწმოთ, მერე ჩავწეროთ“ ამას ვერ უზრუნველყოფს.

2. მარტივად რომ ავხსნათ. ეს ბაზის „დაცვის თანამშრომელია“. თუ ორი ადამიანი ერთსა და იმავე წამს ერთი ოქმის განაწილებას სცდის, ერთი გაივლის, მეორეს კარი დახვდება დაკეტილი. აპლიკაციამ რომც შეცდეს, ბაზა მაინც არ დაუშვებს.

3. ჩვენს პროექტში. ✅

```
-- ერთი ოქმი + კოდი მხოლოდ ერთხელ
CONSTRAINT uq_item_protocol_cad UNIQUE (protocol_number, cadcode)
-- ნაშთშიც ერთხელ
CONSTRAINT uq_backlog_item UNIQUE (protocol_number, cadcode)
-- მდგომარეობა მხოლოდ შვიდიდან ერთ-ერთი
CONSTRAINT ck_item_state CHECK (state IN ('pending','awaiting_field','awaiting_manager',
                                          'awaiting_qcm','returned_to_manager','awaiting_qc','closed'))
-- ვერდიქტი მხოლოდ ოთხიდან ერთ-ერთი, ან ცარიელი
CONSTRAINT ck_item_verdict CHECK (verdict IS NULL OR verdict IN
                                  ('positive','negative','recommendation','clarification'))
-- ელფოსტა უნიკალურია (Google-ით შესვლა)
email VARCHAR(320) UNIQUE
```

4. რატომ. დუბლიკატი ჩვენი მთავარი ძველი პრობლემაა. ძველ ცხრილში 379 დუბლიკატი აღმოჩნდა.

5. კულისებში. შეზღუდვის დარღვევისას ბაზა შეცდომას აბრუნებს (`IntegrityError`) და ტრანზაქცია უკან ბრუნდება. ✅ განაწილების დადასტურება ამასთან ერთად წინასწარაც ამოწმებს არსებულ წყვილებს (`service.confirm`) და დუბლიკატს გამოტოვებს, ანგარიშში კი წერს `skipped` -ს.

6. რა შეიძლება აირიოს. ⚠️ კოდის ჩაწერის ფორმა: „202.43.12.42.095.042“ და იგივე ბოლოში ჰარით ბაზისთვის სხვადასხვაა. ✅ ამიტომ შენახვამდე ნორმალიზაცია ხდება (`_norm` , `strip`). 💡 ახალ წყაროებზეც იგივე წესი უნდა გავრცელდეს.

7. IT იკითხავს. „როგორ გამორიცხავთ დუბლიკატებს?“

8. პასუხი. „ბაზის დონეზე: ოქმის ნომერი + საკადასტრო კოდი უნიკალურია. ამას ემატება ნორმალიზაცია შენახვამდე, ხოლო ძველ მონაცემზე — დედუპლიკაციის წესი: ერთი ჩანაწერი რჩება, თუ სპორადულშიცაა — სპორადული.“

B.7 ტრანზაქცია და ACID

1. ტექნიკური განმარტება. ტრანზაქცია ოპერაციების ჯგუფია, რომელიც ACID თვისებებს აკმაყოფილებს:

- **Atomicity** — ან ყველა ოპერაცია სრულდება, ან არცერთი;
- **Consistency** — ტრანზაქციის შემდეგ ყველა შეზღუდვა დაცულია;
- **Isolation** — ერთდროული ტრანზაქციები ერთმანეთის შუალედურ მდგომარეობას ვერ ხედავენ (დონეები: READ COMMITTED, REPEATABLE READ, SERIALIZABLE; PostgreSQL-ის ნაგულისხმევია READ COMMITTED);
- **Durability** — `COMMIT` -ის შემდეგ მონაცემი ჩავარდნასაც უძლებს.

2. მარტივად რომ ავხსნათ. ტრანზაქცია არის „ყველაფერი ან არაფერი“. ბანკში ფულის გადარიცხვისას ერთ ანგარიშზე ფული მცირდება და მეორეზე იზრდება. თუ შუაში შუქი ჩაქრა, ფული არ უნდა გაქრეს. ორივე ნაბიჯი ერთად ხდება, ან არცერთი.

3. ჩვენს პროექტში. ✅ ოპერატორი „გაგზავნას“ აჭერს. ერთ ტრანზაქციაში ხდება:

```
BEGIN;
  UPDATE qc_items
     SET verdict = 'negative', qc_comment = '…', state = 'awaiting_field',
         checked_at = COALESCE(checked_at, now()), first_sent_at = COALESCE(first_sent_at, now()),
         version = version + 1
   WHERE id = 4512;
  DELETE FROM qc_item_codes WHERE item_id = 4512;
  INSERT INTO qc_item_codes (item_id, code, source) VALUES (4512, 'GEO-02', 'auto');
  INSERT INTO qc_events (item_id, at, actor_id, actor_role, action, from_state, to_state, comment)
       VALUES (4512, now(), 17, 'qc_operator', 'sent', 'pending', 'awaiting_field', '…');
  INSERT INTO attachments (…) VALUES (…);           -- თუ ფოტოა
COMMIT;
```

(ილუსტრაციაა. რეალურ SQL-ს SQLAlchemy აგენერირებს `workflow.apply()`-დან; ჩვენი კოდი დროს `datetime.now()`-ით აყენებს და არა `now()`-ით.)

მარტივად: ეს ბრძანებები ერთად ცვლის ოქმის სტატუსს, ხარვეზის კოდებს და წერს ისტორიაში, „ვინ და როდის გაგზავნა". თუ რომელიმე ნაბიჯი ჩავარდა, არცერთი ცვლილება არ შეინახება, და ოქმი „ნახევრად გაგზავნილი" ვერ დარჩება.

4. რატომ. ისტორია და მდგომარეობა არასოდეს უნდა დაშორდეს ერთმანეთს. სტატუსი შეიცვალა, ისტორიაში კი არ ჩაიწერა — ასეთი რამ დაუშვებელია.

5. კულისებში. SQLAlchemy-ის `Session` ტრანზაქციას პირველ შეკითხვაზე ხსნის. `db.commit()` მას ადასტურებს, შეცდომისას `db.rollback()` კი აუქმებს (`routes/items.action`). ✅ განაწილების დადასტურებაც ასეა: ათასობით ოქმი ერთ ტრანზაქციაში იქმნება (`service.confirm`).

6. რა შეიძლება აირიოს. ძალიან გრძელი ტრანზაქცია ბლოკავს სხვებს. 💡 დიდი განაწილება PostgreSQL-ზე უნდა გაიზომოს.

7. IT იკითხავს. „არათანმიმდევრული მდგომარეობა როგორ გამოირიცხება?"

8. პასუხი. „ყოველი მოქმედება ერთი ტრანზაქციაა: მდგომარეობის ცვლილება, კოდები, ისტორია და ფაილის ჩანაწერი ერთად ინახება ან ერთად უქმდება. ამას ემატება CHECK შეზღუდვები დასაშვებ მდგომარეობებზე."

B.8 ერთდროული ცვლილება: ბლოკირება და ვერსია

1. ტექნიკური განმარტება. ერთი ჩანაწერის ერთდროული ცვლილების კლასიკური პრობლემაა „lost update": ორივე კითხულობს ვერსია 7-ს, ორივე წერს, და მეორე პირველის ცვლილებას „გადაწერს". გამოსავალი ორია:

- **პესიმისტური ბლოკირება** — `SELECT … FOR UPDATE` მწკრივს ტრანზაქციის ბოლომდე კეტავს;
- **ოპტიმისტური ბლოკირება** — ჩანაწერს `version` სვეტი აქვს, და განახლება მხოლოდ მაშინ სრულდება, თუ ვერსია არ შეცვლილა:

```
UPDATE qc_items SET …, version = version + 1
 WHERE id = 4512 AND version = 7;
-- თუ განახლდა 0 მწკრივი → ვიღაცამ უკვე შეცვალა → მომხმარებელს ვეუბნებით
```

2. მარტივად რომ ავხსნათ. თითო ოქმს „მრიცხველი" აქვს. როცა ბარათს ხსნი, გახსოვს, რომ მრიცხველი 7-ზეა. როცა „გაგზავნას" აჭერ, სისტემა ამოწმებს, ისევ 7-ზეა თუ არა. თუ ამასობაში ვინმემ უკვე შეცვალა და 8 გახდა, შენი ცვლილება არ შეინახება და გეტყვის: „ჩანაწერი სხვამ შეცვალა, გადახედე".

3. ჩვენს პროექტში.

- ✅ `qc_items.version`. ფორმა მას ატარებს, `workflow.apply()` კი ადარებს: `if str(v) != str(item.version): raise ConflictError`;
- ✅ ველზე ორი ამზომველის ერთდროული პასუხისას მეორე იღებს შეტყობინებას: „ამ ოქმზე პასუხი უკვე გაგზავნა X-მა (დრო) — ერთ კოდზე ერთი პასუხი იგზავნება" (`routes/items.action`);

- ✅ დამატებით, „ახლა ხსნის“ ნიშანი ადამიანებს წინასწარ აფრთხილებს.
- ⚠️ **G5:** შედარება აპლიკაციაში ხდება, ხოლო `UPDATE WHERE version = ?` -ის გარეშე იგზავნება. SQLite ჩაწერებს რიგრიგობით ასრულებს, ამიტომ პრაქტიკული რისკი მცირეა. PostgreSQL-ზე რამდენიმე worker-ით ორი ზუსტად ერთდროული მოთხოვნა ორივე გაივლის. 💡 გამოსავალი: SQLAlchemy `version_id_col` (ის ზუსტად `WHERE version = ?` -ს ამატებს) ან `SELECT … FOR UPDATE` .

4. რატომ. ბიზნეს-წესია: „ერთ კოდზე ერთი პასუხი“, და „პირველი გაგზავნილი ითვლება“.

5. კულისებში. PostgreSQL-ის MVCC-ში ყოველი ტრანზაქცია მონაცემის „სნეპშოტს“ ხედავს. ერთი მწკრივის ერთდროულ `UPDATE` -ს ბაზა რიგში აყენებს: მეორე პირველის დასრულებას ელოდება, მერე კი `WHERE version = 7` პირობას თავიდან ამოწმებს. ის აღარ სრულდება, და შედეგად 0 მწკრივი განახლდება.

6. რა შეიძლება აირიოს. ფორმამ `version` არ გამოაგზავნა (მაგ. API კლიენტმა). ⚠️ დღეს ასეთ შემთხვევაში შემოწმება გამოტოვდება. 💡 სავალდებულო უნდა გახდეს.

7. IT იკითხავს. „ორი ადამიანი ერთ ჩანაწერს რომ ცვლიდეს?“

8. პასუხი. „ოპტიმისტური ბლოკირება გვაქვს ვერსიის ნომრით: მეორე ცვლილება უარყოფილია და ადამიანი ხედავს, ვინ და როდის გააკეთა პირველი. PostgreSQL-ზე გადასვლისას შემოწმებას ბაზის დონეზე გადავიტანთ, `UPDATE … WHERE version = ?` -ით.“

B.9 View (წარმოდგენა)

1. ტექნიკური განმარტება. View არის შენახული `SELECT` შეკითხვა, რომელიც ვირტუალური ცხრილივით გამოიყენება. მისი გამოყენებით:

- მომხმარებელს მხოლოდ საჭირო სვეტები და მწკრივები ეჩვენება (უსაფრთხოება: `GRANT SELECT` მხოლოდ view-ზე);
- რთული join-ები იმალება;
- შიდა სტრუქტურის ცვლილება მომხმარებელს არ ეხება, თუ view უცვლელია (სტაბილური ინტერფეისი).

Materialized view შედეგს ფიზიკურად ინახავს და `REFRESH` -ით ახლდება. ის სწრაფია, მაგრამ შეიძლება მოძველდეს.

2. მარტივად რომ ავხსნათ. view არის „ფანჯარა“ დიდ ცხრილზე: ჩანს მხოლოდ ის, რისი დანახვაც გვინდა. IT-ს თავისი სისტემის „კარის გაღება“ არ სჭირდება, მხოლოდ პატარა ფანჯარას გვიჭრის, საიდანაც ჩვენთვის საჭირო 7 სვეტი ჩანს.

3. ჩვენს პროექტში. 💡 IT-სთვის ჩვენი პირველი რეკომენდაცია REGFIELDALL-სა და სპორადულისთვის სწორედ view-ია (`docs/IT_REQUEST.md` §3). მაგალითი (ილუსტრაცია; ცხრილისა და სქემის სახელები IT-ის სისტემაში ❓ უცნობია):

```
-- IT-ის მხარეს, მათ ბაზაში
CREATE VIEW qc_export.v_regfield_for_qc AS
SELECT fld_paper_id, cadcode, mox_create_date, fld_status_name,
       oqmi_creator, moxaz_creator, is_last
  FROM <regfieldall_ცხრილი>
 WHERE mox_create_date >= DATE '2026-01-01';

CREATE ROLE qc_reader LOGIN PASSWORD '…';
GRANT USAGE  ON SCHEMA qc_export TO qc_reader;
GRANT SELECT ON qc_export.v_regfield_for_qc TO qc_reader;   -- სხვა არაფერი
```

მარტივად: პირველი ბრძანება ქმნის „ფანჯარას“, სადაც მხოლოდ 7 სვეტი ჩანს, და მხოლოდ 2026 წლიდან. დანარჩენი ორი ქმნის ცალკე მომხმარებელს, რომელსაც მხოლოდ ამ ფანჯრის **წაკითხვის** უფლება აქვს. ის ვერაფერს შეცვლის და სხვა ცხრილს ვერ დაინახავს.

4. რატომ. უსაფრთხოებისა და სტაბილურობისთვის საუკეთესო ბალანსია: IT აკონტროლებს, რას ვხედავთ, ჩვენ კი ყოველთვის ახალ მონაცემს ვიღებთ.

5. კულისებში. view-ზე შეკითხვისას ბაზა view-ის განმარტებას ჩვენს შეკითხვასთან აერთიანებს და ძირითადი ცხრილის ინდექსებს იყენებს.

6. რა შეიძლება აირიოს.

- view-ზე დიდი შეკითხვა წყარო სისტემას ტვირთავს. ამიტომ ღამით ვკითხულობთ და ჩვენთან ვინახავთ (C.9);
- IT view-ის სტრუქტურას ცვლის და ჩვენი იმპორტი ტყდება. საჭიროა შეთანხმებული „კონტრაქტი“.

7. IT იკითხავს. „რატომ view და არა API?“

8. პასუხი. „view ყველაზე ნაკლებ სამუშაოს მოითხოვს თქვენგან: ერთი `CREATE VIEW` და ერთი read-only მომხმარებელი. API ასევე გამოდგება, თუ უკვე გაქვთ. ფაილიც მისაღებია, თუ ბაზაზე წვდომა თქვენი პოლიტიკით აკრძალულია.“

B.10 მიგრაცია (სქემის ცვლილება)

1. ტექნიკური განმარტება. სქემის მიგრაცია არის ბაზის სტრუქტურის ვერსიირებული, თანმიმდევრული ცვლილება: სვეტის დამატება, ინდექსი, ცხრილის გაყოფა. ინსტრუმენტი (Python-ში **Alembic**) ინახავს მიგრაციის ფაილებს (`upgrade()` / `downgrade()`) და ბაზაში წერს, რომელი ვერსიაა ბოლოს გამოყენებული (`alembic_version` ცხრილი). **მონაცემის მიგრაცია** სხვა რამეა: ძველი სისტემიდან მონაცემის გადატანა (თავი F).

2. მარტივად რომ ავხსნათ. მიგრაცია ბაზის „რემონტის გეგმაა“, ნაბიჯ-ნაბიჯ ჩაწერილი: „ვერსია 3: ოქმს დაემატოს ველი ‚ვადა‘“. ყოველ სერვერზე ეს გეგმა ერთნაირად სრულდება, და ყოველთვის ცნობილია, რომელ ვერსიაზეა ბაზა.

3. ჩვენს პროექტში.

- ⚠️ **G2:** მიგრაციის ინსტრუმენტი არ არის. ცხრილები `Base.metadata.create_all(engine)` -ით იქმნება აპლიკაციის ჩართვისას (`app/main.py`). `create_all` ახალ ცხრილს ქმნის, მაგრამ არსებულს **არ ცვლის**: ახალი სვეტი არსებულ ცხრილს არ დაემატება;
- ✅ ჩვენს სხვა პროექტს (Pre-QC hosted) უკვე აქვს Alembic (`hosted/alembic.ini`). გამოცდილება გვაქვს. იქ აღმოჩენილი ხაფანგი: `alembic.ini` მხოლოდ ASCII უნდა იყოს.

4. რატომ. საწარმოოში ბაზას ვერ წავშლით და ვერ ავაწყობთ თავიდან. ყოველი ცვლილება უსაფრთხოდ, ბექაპით და დაბრუნების შესაძლებლობით უნდა მოხდეს.

5. კულისებში. Alembic-ის მიგრაციის მაგალითი:

```
def upgrade():
    op.add_column("qc_items", sa.Column("due_at", sa.DateTime(), nullable=True))
    op.create_index("ix_item_due", "qc_items", ["due_at"])

def downgrade():
    op.drop_index("ix_item_due", table_name="qc_items")
    op.drop_column("qc_items", "due_at")
```

გაშვება: `alembic upgrade head` .

6. რა შეიძლება აირიოს.

- დიდ ცხრილზე `NOT NULL` სვეტის დამატება ნაგულისხმევი მნიშვნელობის გარეშე → შეცდომა;
- გრძელი ბლოკირება;
- მიგრაცია კოდის ვერსიას არ ემთხვევა.

7. IT იკითხავს. „სქემის ცვლილებას როგორ გააკეთებთ საწარმოოში?“

8. პასუხი. „ვერსიირებული მიგრაციებით (Alembic): ჯერ ბექაპი, მერე სატესტო გარემოზე გაშვება, მერე საწარმოოში. ყოველ მიგრაციას დაბრუნების ნაბიჯი აქვს. მიგრაციის გასაშვებად ცალკე მომხმარებელი გვჭირდება, რომელსაც სქემის შეცვლა შეუძლია; აპლიკაციის ჩვეულებრივ მომხმარებელს ეს უფლება არ აქვს.“

B.11 PostgreSQL, SQL Server, SQLite და Excel — შედარება ჩვენს პროექტზე

1. ტექნიკური განმარტება.

კრიტერიუმი	PostgreSQL	Microsoft SQL Server	SQLite	Excel / Google Sheets
ტიპი	სერვერული RDBMS, ღია კოდი	სერვერული RDBMS, კომერციული	ჩაშენებული (ფაილი)	ელცხრილი, არა ბაზა
ლიცენზია / ფასი	უფასო (PostgreSQL License)	ლიცენზია (Express უფასოა, ბაზის ლიმიტით 10 GB)	უფასო	Office / Google ლიცენზია
ერთდროული ჩაწერა	კი (MVCC, მწკრივის დონის ბლოკი)	კი	ერთი მწერალი ერთდროულად	არასანდო, „ბოლო წერს“
ტრანზაქციები, FK, CHECK	სრული	სრული	სრული (FK ჩასართავია)	არა
JSON	`JSON` / `JSONB` + ინდექსები	`NVARCHAR` + JSON ფუნქციები	JSON ფუნქციები	—
უფლებები	როლები, სქემები, მწკრივის დონის უსაფრთხოება (RLS)	როლები, სქემები, RLS, AD ინტეგრაცია	ფაილის უფლებები	გაზიარება
ბექაპი / PITR	`pg_dump`, `pg_basebackup` + WAL, pgBackRest	სრული / დიფერენციალური / ლოგის ბექაპი	ფაილის ასლი	ვერსიების ისტორია
SQLAlchemy	სრული მხარდაჭერა (`psycopg`)	მხარდაჭერა (`pyodbc` , ODBC დრაივერი)	სრული (ჩაშენებული)	—
GIS	PostGIS (საჭიროებისას)	Spatial ტიპები	SpatiaLite	—
OS	Linux, Windows	Windows, Linux	ნებისმიერი	—
ადმინისტრირების ხელსაწყო	pgAdmin, psql, DBeaver	SSMS, Azure Data Studio	DB Browser, CLI	—

2. მარტივად რომ ავხსნათ.

- **Excel** რვეულია: კარგია ერთი ადამიანისთვის, მაგრამ 300 ადამიანისთვის, წესებით და ისტორიით — არა.
- **SQLite** პირადი სეიფია: საიმედოა, მაგრამ ერთი გასაღებით.
- **PostgreSQL** და **SQL Server** ბანკის სეიფებია: ბევრი ადამიანი, უფლებები, ბექაპი. ერთი უფასოა, მეორე ფასიანი. ორივე ჩვენს საქმეს შეასრულებს.

3. ჩვენს პროექტში.

- ✅ პროტოტიპი SQLite-ზეა, რადგან ინსტალაციას არ საჭიროებს და ჩვენებისთვის იდეალურია;
- 💡 საწარმოოსთვის PostgreSQL გვირჩევნია, რადგან:
  - **(ა)** უფასოა;
  - **(ბ)** Pre-QC hosted უკვე PostgreSQL-ზეა (JSONB, ტრიგერები, Alembic);
  - **(გ)** ძველი სტატისტიკის პანელიც PostgreSQL-ს (Neon) იყენებდა;
  - **(დ)** კოდი ტიპებს პორტატულად ინახავს (`JSON` , `String`);
- ✅ კოდი სპეციალურად SQLite-ზე არ არის მიბმული: SQLite-ის სპეციფიკა (`PRAGMA` -ები) მხოლოდ `if _is_sqlite:` ბლოკშია (`app/db.py`).

4. რატომ. IT-ს შეიძლება SQL Server ჰქონდეს სტანდარტად, ლიცენზიითა და DBA-ით. ასეთ შემთხვევაში ჯიუტობა არ გვჭირდება.

5. კულისებში. SQL Server-ზე გადასვლისას შესამოწმებელია:

- `JSON` სვეტები (SQL Server-ში JSON ცალკე ტიპი ძველ ვერსიებში არ არის, ტექსტად ინახება);
- სტრიქონების შედარება ქართულზე (collation);
- `ILIKE`-ის მსგავსი ძებნა;
- თარიღების ფუნქციები სტატისტიკაში;
- შეზღუდვის სახელების სიგრძე. ეს სამუშაოა, მაგრამ შესაძლებელია.

6. რა შეიძლება აირიოს. ⚠️ „მხოლოდ URL-ის შეცვლა“ შეუმოწმებელი მტკიცებაა (G1, C3). ნებისმიერ ახალ ბაზაზე ტესტები უნდა გავიდეს.

7. IT იკითხავს. „რატომ PostgreSQL და არა SQL Server?“

8. პასუხი. „PostgreSQL უფასოა, ჩვენი სხვა პროექტი უკვე მასზეა და ორივე ხელსაწყოში გამოცდილება გვაქვს. მაგრამ აპლიკაცია ORM-ზეა აწყობილი და SQL Server-ზე გადატანა შესაძლებელია. თუ თქვენი სტანდარტი და DBA SQL Server-ისაა, ტესტებს მასზე გავუშვებთ და განსხვავებებს გავასწორებთ. ორივე შემთხვევაში ეს ჩვენი სამუშაოა პილოტამდე.“

B.12 რა ინახება ბაზაში და რა — არა

1. ტექნიკური განმარტება. ბაზაში სტრუქტურირებული, ტრანზაქციული მონაცემი ინახება. დიდი ბინარული ფაილები, საიდუმლოებები, ლოგები და აპლიკაციისთვის ზედმეტი პირადი მონაცემი სხვა ადგილას ინახება, ან საერთოდ არ ინახება (მონაცემის მინიმიზაციის პრინციპი).

2. მარტივად რომ ავხსნათ. ბაზა საქაღალდეების კარადაა. ფოტოები ცალკე ალბომშია (დისკზე), კარადაში მხოლოდ წერია, „ფოტო №5 ალბომის ამა და ამ გვერდზეა“. გასაღებები (პაროლები) კარადაში არ დევს. ის კი, რაც სამუშაოსთვის არ გვჭირდება, საერთოდ არ შემოგვაქვს.

3. ჩვენს პროექტში.

ინახება ბაზაში ✅	არ ინახება ბაზაში ✅	არ ინახება საერთოდ ✅
ოქმი, კოდი, სტატუსი, კომენტარები, ისტორია	ფოტოები და PDF — დისკზე (`data/uploads/<item_id>/<uuid>`), ბაზაში მხოლოდ მეტამონაცემი (`attachments`)	განმცხადებლების პირადი ნომრები, ტელეფონები, მისამართები (სპორადულის ფაილიდან)
მომხმარებელი, როლი, გუნდი, ელფოსტა	საიდუმლოებები (OAuth secret, SMTP პაროლი, სესიის გასაღები) — გარემოს ცვლადებში	Apps Script-ის token-ები (`data/` -ში არ იწერება)
პირადი ნომერი 🔐 (გადაწყვეტილებით, ვინაობის შესადარებლად; ინტერფეისში ნიღბით)	სერვერის ლოგები	REGFIELDALL-ის გეომეტრია
სესიები, აუდიტი, მეილის ასლი	—	ატვირთული სპორადულის ფაილი (დამუშავების შემდეგ იშლება)

4. რატომ. ნაკლები მონაცემი ნაკლებ რისკს ნიშნავს. ბაზაში ფაილების შენახვა ბექაპს ამძიმებს.

5. კულისებში. ფოტოს ატვირთვისას (`workflow.save_files`):

1. მოწმდება ზომა (≤ 10 MB) და ტიპი;
2. ფაილი ინახება შემთხვევითი სახელით (`uuid4().hex`);
3. ბაზაში იწერება ორიგინალი სახელი, ტიპი და ზომა.

გაცემისას (`/attachments/{id}`) ჯერ მოწმდება, უჩანს თუ არა ეს ოქმი ამ მომხმარებელს.

6. რა შეიძლება აირიოს. ⚠️ ბაზისა და ფოტოების ბექაპი სხვადასხვა დროს კეთდება, და აღდგენის შემდეგ ერთმანეთს არ ემთხვევა. 💡 ორივე ერთ განრიგში უნდა იყოს.

7. IT იკითხავს. „პირად მონაცემს ინახავთ?“

8. პასუხი. „თანამშრომლების სახელებს, ელფოსტებს და, გადაწყვეტილებით, პირად ნომრებს ვინახავთ ვინაობის შესადარებლად: ინტერფეისში ნიღბით ჩანს და ლოგებში არ იწერება. მოქალაქეების პირად მონაცემს არ ვინახავთ. QC კომენტარში შეიძლება მესაკუთრის სახელი შეგვხვდეს. პირადი მონაცემის დამუშავების წესს თქვენს და იურიდიულ სამსახურთან შევათანხმებთ.“

B.13 ბექაპი და აღდგენა

1. ტექნიკური განმარტება.

- **ლოგიკური ბექაპი** (`pg_dump`) — SQL ან custom ფორმატის ასლი. მოსახერხებელია, მაგრამ აღდგება მხოლოდ ასლის მომენტამდე;
- **ფიზიკური ბექაპი** (`pg_basebackup`) + **WAL არქივი** = **PITR** (point-in-time recovery). აღდგენა ნებისმიერ წამზეა შესაძლებელი არქივის ფარგლებში. ინსტრუმენტები: pgBackRest, Barman;
- **3-2-1 წესი:** 3 ასლი, 2 სხვადასხვა მედია, 1 სხვა ლოკაციაზე;
- ბექაპი **დაშიფრული** და **გამოცდილი** უნდა იყოს.

2. მარტივად რომ ავხსნათ.

- ღამის ბექაპი ფოტოა, რომელიც ყოველ საღამოს კეთდება: დაზიანებისას გუშინდელ ფოტოზე ვბრუნდებით და დღის სამუშაო იკარგება.
- PITR ვიდეოჩანაწერია: ნებისმიერ წუთზე შეგვიძლია „გადავახვიოთ“, მაგალითად შეცდომამდე ერთი წუთით ადრე.

3. ჩვენს პროექტში.

- ✅ დღეს ბექაპი არ კეთდება;
- 💡 საჭიროა:
  - **(ა)** PostgreSQL-ის PITR ან მინიმუმ ღამის `pg_dump` ;
  - **(ბ)** `data/uploads/` -ის ბექაპი;
  - **(გ)** კონფიგურაციის (საიდუმლოებების გარეშე) ბექაპი;
  - **(დ)** კვარტალში ერთხელ აღდგენის ტესტი.

4. რატომ. QC-ის ისტორია, გადაწყვეტილებები და ფოტოები ორგანიზაციის ჩანაწერებია.

5. კულისებში. ღამის `pg_dump` -ის მაგალითი (ილუსტრაცია):

```
pg_dump -h db-host -U qc_backup -Fc -f /backup/qc_$(date +%F).dump qc
pg_restore -h db-host -U qc_admin -d qc_restore_test /backup/qc_2026-10-22.dump
```

მარტივად: პირველი ბრძანება ბაზის სრულ ასლს ფაილში ინახავს თარიღიანი სახელით. მეორე ამ ასლს ცალკე, სატესტო ბაზაში აღადგენს, რომ შევამოწმოთ, ასლი ნამდვილად მუშაობს.

6. რა შეიძლება აირიოს.

- ბექაპი იმავე დისკზეა, რომელიც დაზიანდა;
- ბექაპის ფაილს ყველა კითხულობს. ის მთელ ბაზას შეიცავს, პირადი ნომრების ჩათვლით, ამიტომ უფლებები და დაშიფვრა სავალდებულოა.

7. IT იკითხავს. „ბექაპს ვინ აკეთებს და სად ინახება?“

8. პასუხი. „სასურველია, თქვენი სტანდარტული ბექაპის სისტემით, ჩვენი ბაზისა და ფაილების საქაღალდის ჩათვლით. აღდგენის ტესტში ჩვენ მივიღებთ მონაწილეობას და შევამოწმებთ, რომ აპლიკაცია აღდგენილ ბაზაზე მუშაობს.“

B.14 უფლებები და მინიმალური უფლება (least privilege) ბაზაში

1. ტექნიკური განმარტება. თითო კომპონენტი ბაზას საკუთარი როლით უკავშირდება და მხოლოდ იმ უფლებებს იღებს, რაც სჭირდება:

- **აპლიკაციის როლი** — `SELECT/INSERT/UPDATE/DELETE` საკუთარ ცხრილებზე. `DROP` / `ALTER` არა, superuser არა;
- **მიგრაციის როლი** — სქემის შეცვლა;
- **ბექაპის როლი** — წაკითხვა;
- **ანალიტიკის როლი** (საჭიროებისას) — მხოლოდ `SELECT` view-ებზე.

წყარო სისტემებზე ჩვენი როლი მხოლოდ `SELECT`-ს იღებს, მხოლოდ view-ზე.

2. მარტივად რომ ავხსნათ. თითოეულს იმდენი გასაღები აქვს, რამდენიც სჭირდება. საიტს „კარადის“ გახსნა და ქაღალდის ჩადება შეუძლია, მაგრამ კარადის დანგრევა — არა. ჩვენს გასაღებს IT-ის სისტემაში მხოლოდ ერთი ფანჯრის გაღება შეუძლია, და მხოლოდ საყურებლად.

3. ჩვენს პროექტში. 💡 შეთავაზება (PostgreSQL):

```
CREATE ROLE qc_migrator LOGIN PASSWORD '…';          -- სქემის მფლობელი, მხოლოდ deployment-ისას
CREATE ROLE qc_app      LOGIN PASSWORD '…';          -- აპლიკაცია
GRANT CONNECT ON DATABASE qc TO qc_app;
GRANT USAGE ON SCHEMA public TO qc_app;
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public TO qc_app;
GRANT USAGE ON ALL SEQUENCES IN SCHEMA public TO qc_app;
-- ისტორია მხოლოდ ემატება: შეცვლა და წაშლა აპლიკაციასაც ეკრძალება
REVOKE UPDATE, DELETE ON qc_events FROM qc_app;
```

მარტივად: ვქმნით ორ „ანგარიშს“ ბაზაში: ერთი მხოლოდ განახლებისას გამოიყენება და სტრუქტურას ცვლის, მეორეს საიტი იყენებს ყოველდღიურად. ბოლო ბრძანება ისტორიის ცხრილს „ბეტონში ასხამს“: საიტს მასში მხოლოდ ახლის დამატება შეუძლია, შეცვლა და წაშლა კი არა.

⚠️ ბოლო ხაზამდე უნდა შევამოწმოთ, რომ საწარმოოს არცერთი კოდის გზა `qc_events`-ს არ ცვლის. ძველი მონაცემის ხელახალი იმპორტი (`legacy_import`) მას შლის, ამიტომ ის მიგრაციის როლით უნდა გაეშვას. Pre-QC hosted-ში append-only ბაზის ტრიგერითაც არის დაცული. 💡 იგივე ხერხი ჩვენთვისაც გამოდგება.

4. რატომ. თუ აპლიკაცია გატყდა (მაგ. SQL injection-ის მცდელობა), ზიანი შეზღუდულია.

5. კულისებში. ჩვენი კოდი SQL-ს ORM-ით აგენერირებს, პარამეტრებით, ამიტომ SQL injection-ის რისკი დაბალია. ✅ ერთ ადგილას SQL ტექსტად იწყობა (`source.py`, `IN (?,?,…)` და ფიქსირებული ფრაგმენტები), მაგრამ მნიშვნელობები `?` პარამეტრებად გადაეცემა.

6. რა შეიძლება აირიოს. აპლიკაცია superuser-ით უკავშირდება „მოსახერხებლობისთვის“. ასე არ უნდა მოხდეს.

7. IT იკითხავს. „რა უფლებები გჭირდებათ ჩვენს ბაზებზე?“

8. პასუხი. „თქვენს ბაზებზე მხოლოდ `SELECT`, მხოლოდ ჩვენთვის შექმნილ view-ებზე, ცალკე მომხმარებლით და მხოლოდ ჩვენი სერვერის IP-დან. ჩვენს ბაზაზე: ორი როლი — აპლიკაციის და მიგრაციის. superuser არ გვჭირდება.“

B.15 მონაცემის შენახვის ვადა (retention)

1. ტექნიკური განმარტება. Retention policy განსაზღვრავს, რამდენ ხანს ინახება მონაცემის თითოეული კატეგორია და როგორ იშლება ან არქივდება. ამას სამართლებრივი მოთხოვნები (პერსონალური მონაცემის

დაცვა, საარქივო წესები), ბიზნეს-საჭიროება და ტექნიკური ხარჯი განსაზღვრავს.

2. მარტივად რომ ავხსნათ. ყველაფერი სამუდამოდ არ უნდა ინახებოდეს. სესიის ჩანაწერი ერთ დღეში აღარ გვჭირდება; QC-ის ისტორია კი, შეიძლება, წლობით გვჭირდებოდეს. რამდენ ხანს — წესი უნდა ითქვას.

3. ჩვენს პროექტში.

მონაცემი	დღეს ✅	შეთავაზება 💡 / ❓
`presence`	10 წუთზე ძველი იშლება	იგივე
`sessions`	ვადაგასული რჩება ბაზაში	30 დღის შემდეგ წაშლა
`qc_items`, `qc_events`, `attachments`	სამუდამოდ	❓ სააგენტოს საარქივო წესით
`audit_log`	სამუდამოდ	❓ მაგ. 3 წელი
`emails`	სამუდამოდ	1 წელი
სპორადულის ატვირთული ფაილი	მაშინვე იშლება	იგივე
გადახედვა (მეხსიერებაში)	30 წუთი	იგივე (ბაზაში გადასვლის შემდეგ — გასუფთავება)

4. რატომ. პირადი მონაცემის დაცვის კანონი მონაცემის საჭიროზე მეტ ხანს შენახვას არ უშვებს. გარდა ამისა, ბაზა უსასრულოდ არ უნდა იზრდებოდეს.

5. კულისებში. ფონური ამოცანა (A.8) ღამით შლის ან არქივში გადაიტანს ვადაგასულს.

6. რა შეიძლება აირიოს. აუდიტის ჩანაწერის წაშლა გამოძიებამდე.

7. IT იკითხავს. „რამდენ ხანს ინახება ლოგები და პირადი მონაცემი?“

8. პასუხი. „ტექნიკური ჩანაწერებისთვის ვადებს ჩვენ ვთავაზობთ. QC ჩანაწერებისა და აუდიტის ვადა სააგენტოს საარქივო და პერსონალური მონაცემის წესით უნდა განისაზღვროს. ამაზე თქვენი და იურიდიული სამსახურის პოზიცია გვჭირდება.“

B.16 PostgreSQL Server, pgAdmin, SSMS და VS Code — რა რა არის

1. ტექნიკური განმარტება.

სახელი	რა არის	ანალოგია
PostgreSQL Server	თავად DBMS: სერვისი (`postgres`), რომელიც მონაცემს ინახავს და 5432-ზე უსმენს	ბანკის სეიფი
psql	PostgreSQL-ის ბრძანების ხაზის კლიენტი	ტელეფონი, რომლითაც სეიფთან „ლაპარაკობ“
pgAdmin	PostgreSQL-ის გრაფიკული ადმინისტრირების კლიენტი (ვებ/დესკტოპი)	ბანკის ოპერატორის ეკრანი
SQL Server	Microsoft-ის DBMS	სხვა ბანკის სეიფი
SSMS (SQL Server Management Studio)	SQL Server-ის გრაფიკული კლიენტი (მხოლოდ Windows)	იმ ბანკის ოპერატორის ეკრანი
DBeaver	უნივერსალური კლიენტი ბევრ ბაზაზე	უნივერსალური პულტი
VS Code	კოდის რედაქტორი; გაფართოებით შეუძლია ბაზასთან დაკავშირება და შეკითხვის გაშვება	სამუშაო მაგიდა, რომელზეც პულტიც შეგიძლია დადო

2. მარტივად რომ ავხსნათ. PostgreSQL თავად ბაზაა, რომელიც სერვერზე ცხოვრობს. pgAdmin, SSMS და VS Code „ფანჯრებია“, საიდანაც ადამიანი ბაზას უყურებს. ფანჯრის დახურვა ბაზას არ აჩერებს. ბაზის გაჩერება კი ყველა ფანჯარას აცარიელებს.

3. ჩვენს პროექტში.

- ✅ დღეს SQLite ფაილია და მას VS Code-ის გაფართოებით ან DB Browser-ით ვხსნით;
- 💡 საწარმოოში ჩვენ, დეველოპერებს, საწარმოო ბაზაზე პირდაპირი ჩაწერის წვდომა არ უნდა გვქონდეს. საკმარისია წაკითხვის წვდომა დიაგნოსტიკისთვის, ან წვდომა მხოლოდ სატესტო ბაზაზე.

4. რატომ. შეხვედრაზე შეიძლება იკითხონ: „pgAdmin-ით შეხვალთ?“ ეს ნიშნავს: „ბაზაზე პირდაპირი წვდომა გჭირდებათ?“

5. კულისებში. ყველა კლიენტი ერთი და იგივე პროტოკოლით (PostgreSQL wire protocol) უკავშირდება 5432 პორტს, მომხმარებლითა და პაროლით.

6. რა შეიძლება აირიოს. ვინმე pgAdmin-იდან საწარმოო ცხრილს ხელით ასწორებს. ასეთი ცვლილება ისტორიაში არ ჩაიწერება (`qc_events` -ს გვერდს უვლის). 💡 ხელით ცვლილება მხოლოდ პროცედურით და აუდიტით უნდა ხდებოდეს.

7. IT იკითხავს. „ბაზაზე პირდაპირი წვდომა თქვენ გჭირდებათ?“

8. პასუხი. „საწარმოოში ჩვეულებრივ მუშაობაში არა: ყველაფერი აპლიკაციით ხდება. ინციდენტის დიაგნოსტიკისთვის წაკითხვის წვდომა გამოგვადგება, თქვენი წესით. ჩაწერა მხოლოდ მიგრაციით.“

B.17 როგორ ესაუბრება ჩვენი საიტი ბაზას

1. ტექნიკური განმარტება. აპლიკაცია ბაზას **ORM**-ით (SQLAlchemy 2) უკავშირდება:

- **Engine** კავშირების აუზს მართავს;
- **Session** ერთი სამუშაო ერთეულია (unit of work), ჩვეულებრივ ერთი HTTP მოთხოვნა;
- **მოდელები** Python კლასებია, რომლებიც ცხრილებს შეესაბამება;
- შეკითხვები Python-ში იწერება, და SQLAlchemy მათ კონკრეტული ბაზის SQL დიალექტად თარგმნის.

2. მარტივად რომ ავხსნათ. ჩვენი კოდი ბაზას პირდაპირ SQL-ით, როგორც წესი, არ ელაპარაკება. ის „თარჯიმანს“ (SQLAlchemy-ს) ეუბნება, მაგალითად: „მომეცი ამ ოპერატორის გასავლელი ოქმები“. თარჯიმანი ამას SQL-ად თარგმნის, იმ ბაზის ენაზე, რომელიც ახლა გვაქვს. ამიტომ SQLite-დან PostgreSQL-ზე გადასვლისას კოდის უმეტესობა არ იცვლება.

3. ჩვენს პროექტში. ✅

```
# app/db.py — კავშირი
engine = create_engine(DATABASE_URL, ...)
SessionLocal = sessionmaker(bind=engine, autoflush=False, expire_on_commit=False)

def get_db():                # FastAPI dependency — ერთი სესია ერთ მოთხოვნაზე
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

# app/scope.py — ოპერატორის ხილვადობა
select(QcItem).where(QcItem.operator_id == ctx.user.id)
```

რასაც SQLAlchemy აგენერირებს:

```
SELECT qc_items.id, qc_items.protocol_number, … FROM qc_items
WHERE qc_items.operator_id = %(operator_id_1)s
```

მარტივად: Python-ის ერთი ხაზი („მომეცი ოქმები, სადაც ოპერატორი მე ვარ“) SQL-ად იქცევა. მნიშვნელობა (`17`) ცალკე, „პარამეტრად“ გადაეცემა და ტექსტში არ ჩაიწერება. ესაა SQL injection-ისგან დაცვის მთავარი ხერხი.

4. რატომ. ერთი კოდი სხვადასხვა ბაზაზე, უსაფრთხო პარამეტრები, ნაკლები შეცდომა.

5. კულისებში. connection string:

```
sqlite:///C:/…/data/platform.db                          # დღეს
postgresql+psycopg://qc_app:<პაროლი>@db-host:5432/qc     # საწარმოოში (💡, დრაივერი დასამატებელია)
```

`QCP_DATABASE_URL` გარემოს ცვლადიდან იკითხება (`app/config.py`).

6. რა შეიძლება აირიოს.

- N+1 შეკითხვა (თითო ოქმზე ცალკე შეკითხვა ამზომველებისთვის) ✅ თავიდან აცილებულია `selectinload`-ით;
- პაროლი connection string-ში კოდშია ჩაწერილი. ✅ ჩვენთან გარემოს ცვლადშია.

7. IT იკითხავს. „connection pool-ის ზომა? რამდენ კავშირს გახსნით?“

8. პასუხი. „SQLAlchemy-ის ნაგულისხმევია 5 + 10 დამატებითი თითო worker-ზე. 3 worker-ზე მაქსიმუმ ≈ 45 კავშირი, რაც PostgreSQL-ის ნაგულისხმევ 100-იან ლიმიტში ეტევა. საჭიროებისას შევამცირებთ.“

B.18 SQL-ის პრაქტიკული მაგალითები ჩვენს სქემაზე

ყველა მაგალითი ჩვენს რეალურ ცხრილებზეა (`app/models.py`). ისინი მხოლოდ კითხულობენ (`SELECT`).

მაგალითი 1 — რამდენი შემთხვევაა თითო მდგომარეობაში

```
SELECT state, COUNT(*) AS n
  FROM qc_items
 GROUP BY state
 ORDER BY n DESC;
```

რას აკეთებს: ოქმებს მდგომარეობის მიხედვით აჯგუფებს და თითოს ითვლის. ✅ დღევანდელი შედეგი: closed 34 502, awaiting_field 3 774, awaiting_qc 3 030, pending 1 456, returned_to_manager 1 107, awaiting_qcm 290. **რატომ გამოვიყენებდით:** ერთ შეხედვაში ჩანს, სად „დგას“ სამუშაო.

მაგალითი 2 — ოპერატორის დღევანდელი რაოდენობა

```
SELECT u.full_name, COUNT(*) AS today
  FROM qc_items i
  JOIN users u ON u.id = i.operator_id
 WHERE i.first_sent_at >= DATE '2026-10-22'
   AND i.first_sent_at <  DATE '2026-10-23'
 GROUP BY u.full_name
 ORDER BY today DESC;
```

რას აკეთებს: ითვლის, ვინ რამდენი ოქმი გაგზავნა დღეს პირველად, დაზუსტების კითხვის ჩათვლით. `JOIN` ოქმს მისი ოპერატორის სახელს უერთებს. **რატომ:** ეს ზუსტად ის „დღის რაოდენობაა“, რასაც ოპერატორი ეკრანზე ხედავს. თარიღის ფილტრი `>= დღე AND < მეორე დღე` სწორი ხერხია, რადგან ინდექსს იყენებს და დღის ბოლო წამსაც მოიცავს.

მაგალითი 3 — ხარვეზის პროცენტი მენეჯერებზე

```
SELECT m.full_name                                              AS manager,
       COUNT(*)                                                 AS checked,
       SUM(CASE WHEN i.verdict = 'negative' THEN 1 ELSE 0 END)  AS negative,
       ROUND(100.0 * SUM(CASE WHEN i.verdict = 'negative' THEN 1 ELSE 0 END) / COUNT(*), 1)
                                                                AS defect_pct
  FROM qc_items i
  JOIN item_people p ON p.item_id = i.id AND p.kind = 'manager'
  JOIN users m       ON m.id = p.user_id
 WHERE i.checked_at IS NOT NULL
 GROUP BY m.full_name
 ORDER BY defect_pct DESC;
```

რას აკეთებს: თითო მენეჯერზე ითვლის შემოწმებულ და უარყოფით ოქმებს და მათ პროცენტს. ოქმი **ყველა** მიკუთვნებულ მენეჯერს ეთვლება, ამიტომ `item_people`-ით ვაერთებთ და არა `field_manager_id`-ით. **რატომ:** ასე ითვლება ჩვენი წესი: „მენეჯერზე — ოქმი ყველა მიკუთვნებულ მენეჯერს ეთვლება“. ეს გამარტივებული ვერსიაა; ზუსტი წესები (დაზუსტება, პერიოდი, როლის ფილტრი) `services/stats.py`-შია.

მაგალითი 4 — ყველაზე ხშირი ხარვეზის კოდები

```
SELECT c.code, d.name, d.severity, COUNT(*) AS n
  FROM qc_item_codes c
  JOIN qc_items     i ON i.id = c.item_id
  JOIN defect_codes d ON d.code = c.code
 WHERE i.verdict = 'negative'
 GROUP BY c.code, d.name, d.severity
 ORDER BY n DESC
 LIMIT 10;
```

რას აკეთებს: უარყოფით ოქმებში ყველაზე ხშირ 10 კოდს პოულობს, სახელითა და სიმძიმით. **რატომ:** ხელმძღვანელობას აინტერესებს, „რა ხარვეზი ყველაზე ხშირია“. ეს სწავლების საფუძველია.

მაგალითი 5 — ერთი ოქმის სრული ისტორია

```
SELECT e.at, u.full_name, e.actor_role, e.action, e.from_state, e.to_state, e.comment, e.approx
  FROM qc_events e
  LEFT JOIN users u ON u.id = e.actor_id
 WHERE e.item_id = 4512
 ORDER BY e.id;
```

რას აკეთებს: ქრონოლოგიურად აჩვენებს, ვინ, რა როლით, რა გააკეთა ამ ოქმზე და საიდან სად გადაიყვანა. `LEFT JOIN` იმიტომ არის, რომ სისტემურმა ჩანაწერმა (მომხმარებლის გარეშე) ისტორიიდან არ „ამოვარდეს“. `approx = true` ნიშნავს, რომ დრო ძველი ცხრილიდან გამოყვანილია და მიახლოებითია. **რატომ:** დავის ან საჩივრის დროს ესაა მტკიცებულება.

მაგალითი 6 — „ცრუ გასწორება“ ამზომველებზე

```
SELECT s.full_name, SUM(i.false_fix_count) AS false_fixes, COUNT(*) AS items
  FROM qc_items i
  JOIN item_people p ON p.item_id = i.id AND p.kind = 'surveyor'
  JOIN users s       ON s.id = p.user_id
 WHERE i.false_fix_count > 0
 GROUP BY s.full_name
 ORDER BY false_fixes DESC;
```

რას აკეთებს: პოულობს, ვისთან დაბრუნდა ოქმი „შესწორდა“-ს შემდეგ ისე, რომ სინამდვილეში არ იყო გასწორებული. **რატომ:** ეს მენეჯერებისთვის მნიშვნელოვანი სიგნალია, და ამიტომ შეზღუდულ სტატისტიკაშია.

მაგალითი 7 — დუბლიკატების ძებნა იმპორტამდე

```
-- staging ცხრილზე, სადაც ძველი ცხრილის მწკრივები წედლად ჩაიტვირთა
SELECT protocol_number, cadcode, COUNT(*) AS copies,
       STRING_AGG(source_tab, ', ') AS tabs
  FROM staging_legacy_rows
 GROUP BY protocol_number, cadcode
HAVING COUNT(*) > 1
 ORDER BY copies DESC;
```

რას აკეთებს: პოულობს ერთსა და იმავე ოქმს + კოდს, რომელიც რამდენიმე მწკრივად ან ჩანართად გვხვდება, და აჩვენებს, რომელ ჩანართებში. `HAVING` ფილტრავს უკვე დაჯგუფებულ შედეგს. **რატომ:** ასე დადგინდა 379 დუბლიკატი. `staging_legacy_rows` ილუსტრაციაა: პროტოტიპში ეს ლოგიკა Python-შია (`legacy_import.py`).

მაგალითი 8 — ოპტიმისტური განახლება (ჩაწერის მაგალითი, მხოლოდ ასახსნელად)

```
UPDATE qc_items
   SET field_response = 'corrected', state = 'awaiting_qc', version = version + 1
 WHERE id = 4512 AND version = 7;
-- შედეგი: "UPDATE 1" → ჩვენი ცვლილება შევიდა; "UPDATE 0" → სხვამ დაგვასწრო
```

რას აკეთებს: ოქმს მხოლოდ მაშინ ცვლის, თუ მისი ვერსია ისევ 7-ია, ანუ ამასობაში არავის შეუცვლია. **რატომ:** ასე ბაზის დონეზე გამოირიცხება ორი ერთდროული პასუხი (G5-ის გამოსავალი).

C. API და გარე მონაცემი

C.1 HTTP — როგორ ესაუბრება ბრაუზერი სერვერს

1. ტექნიკური განმარტება. HTTP არის მოთხოვნა-პასუხის (request/response) პროტოკოლი TCP-ზე. HTTPS იგივეა, ოღონდ TLS-ით დაშიფრული. მოთხოვნა შედგება:

- **მეთოდისგან:**
  - `GET` — წაკითხვა, მდგომარეობას არ ცვლის;
  - `POST` — შექმნა ან მოქმედება;
  - `PUT` / `PATCH` — განახლება;
  - `DELETE` ;
- **ბილიკისგან** (`/items/4512`) და query string-ისგან (`?tab=todo`);
- **თავსართებისგან** (headers: `Cookie` , `Origin` , `Content-Type` , `Authorization`);
- **სხეულისგან** (body: ფორმა ან JSON).

პასუხი შედგება სტატუს-კოდის, თავსართებისა და სხეულისგან.

კოდი	მნიშვნელობა	ჩვენთან ✅
200	OK	გვერდი / JSON
303	See Other — გადამისამართება	ყოველი მოქმედების შემდეგ (POST → GET), რომ გვერდის განახლებამ მოქმედება არ გაიმეოროს
401	ავთენტიფიკაცია საჭიროა	`/api/*` შესვლის გარეშე → `{"error": "login_required"}`
403	აკრძალულია	როლს ეს გვერდი არ ეკუთვნის (მაგ. ოპერატორი → `/admin`)
404	ვერ მოიძებნა	**სხვისი ჩანაწერიც** (განზრახ, D.9)
413	ძალიან დიდი მოთხოვნა	proxy-ს ლიმიტი (საწარმოოში)

კოდი	მნიშვნელობა	ჩვენთან ✅
500	სერვერის შეცდომა	კოდის შეცდომა
502 / 503 / 504	proxy ვერ უკავშირდება აპლიკაციას / გადატვირთულია / timeout	აპლიკაცია გაჩერებულია ან ნელია

2. მარტივად რომ ავხსნათ. HTTP წერილების გაცვლაა. ბრაუზერი სერვერს წერს: „მომეცი ოქმი 4512“ (GET) ან „ამ ოქმზე ვაგზავნი პასუხს“ (POST). სერვერი პასუხობს ნომრით, რომელიც გვეუბნება, რა მოხდა: 200 — „აი, ინებე“, 404 — „ასეთი არ არის“, 403 — „შენ არ შეგიძლია“, 500 — „ჩემთან რაღაც გაფუჭდა“.

3. ჩვენს პროექტში. ✅

- მოქმედებები POST-ით ხდება და პასუხად 303 მოდის (`routes/common.redirect`);
- `Cache-Control: no-store` ედება `/api/*` -სა და `/items/*` -ს (`app/main.py`), რომ ბრაუზერმა ან proxy-მ ოქმის მონაცემი ქეშში არ შეინახოს.

4. რატომ. IT-ის ლოგებსა და შეცდომის აღწერებში სწორედ ეს კოდები ჩანს. „502“ პრობლემას proxy-სა და აპლიკაციას შორის აჩვენებს, „500“ — ჩვენს კოდში.

5. კულისებში. მოქმედების ფორმა გაიგზავნება `POST /items/4512/action` -ზე, `Content-Type: multipart/form-data` -თი (ფოტოს ჩათვლით), cookie-თი და `Origin` თავსართით.

6. რა შეიძლება აირიოს. GET-ით მდგომარეობის შეცვლა: ბმულზე დაჭერამ ან სხვა საიტის `<img>` -მა მოქმედება გამოიწვია. ✅ ჩვენთან ყველა ცვლილება POST-ით ხდება, `logout` -ის ჩათვლით.

7. IT იკითხავს. „რომელი მეთოდები და ბილიკები უნდა დაუშვას proxy-მ / WAF-მა?“

8. პასუხი. „GET და POST, ერთი დომენის ყველა ბილიკზე; multipart ატვირთვა 10 MB-მდე. სრული სია OpenAPI-შია (`/docs`).“

C.2 JSON

1. ტექნიკური განმარტება. JSON (JavaScript Object Notation) არის ტექსტური ფორმატი სტრუქტურირებული მონაცემისთვის. ელემენტებია ობიექტი `{"key": value}` , მასივი `[…]` , სტრიქონი, რიცხვი, `true/false` , `null` . კოდირება UTF-8-ია, ამიტომ ქართული ტექსტი პირდაპირ იწერება.

2. მარტივად რომ ავხსნათ. JSON არის „ანკეტა“, რომელსაც კომპიუტერები ერთმანეთს უგზავნიან: ველის სახელი და მნიშვნელობა, მკაცრი წესით, რომ ორივე მხარემ ერთნაირად წაიკითხოს.

3. ჩვენს პროექტში. ✅

- ჩვენი API JSON-ს აბრუნებს;
- Apps Script-იდან მონაცემი JSON-ად მოდის (`data/raw/*.json`);
- WFS პასუხს GeoJSON-ად ვითხოვთ (`outputFormat=application/json`);
- ბაზაშიც რამდენიმე სვეტი JSON-ია (`settings.value` , `distribution_runs.params` , `legacy_extra`).

4–8. იხ. C.3, სადაც JSON-ის რეალური მაგალითია.

C.3 API და REST — ჩვენი API-ს მაგალითები

1. ტექნიკური განმარტება. API (Application Programming Interface) არის შეთანხმებული წესი, რომლითაც ერთი პროგრამა მეორეს მიმართავს. **REST** HTTP-ზე დაფუძნებული სტილია:

- რესურსები ბილიკებია (`/api/items/{id}`);
- მოქმედება HTTP მეთოდით გამოიხატება;
- მდგომარეობა სერვერზე არ ინახება მოთხოვნებს შორის (stateless), ავთენტიფიკაციის გარდა;

- პასუხი ჩვეულებრივ JSON-ია.

OpenAPI API-ს მანქანურად წაკითხვადი აღწერაა. FastAPI მას ავტომატურად ქმნის (`/openapi.json` , `/docs`).

2. მარტივად რომ ავხსნათ. API არის „სარკმელი“ პროგრამებისთვის. ადამიანი საიტს ღილაკებით იყენებს; სხვა პროგრამა, მაგალითად Pre-QC ან სააგენტოს სხვა სისტემა, იგივე ინფორმაციას API-ით იღებს: ზუსტად განსაზღვრული კითხვით და ზუსტად განსაზღვრული ფორმის პასუხით.

3. ჩვენს პროექტში. ✅ `app/routes/api.py` . ყველა endpoint იმავე ხილვადობის ფილტრს გადის, რასაც გვერდები (`scope.items_query` , `scope.can_view`).

მოთხოვნა (ბრაუზერიდან, შესული მომხმარებლის cookie-თი):

```
GET /api/items?state=awaiting_field&limit=2 HTTP/1.1
Host: 127.0.0.1:8200
Cookie: qcp_session=<ხელმოწერილი sid>
```

პასუხი (ველების სტრუქტურა რეალურია — `_item_json()` ; მნიშვნელობები გამოგონილია):

```
{
  "items": [
    {
      "id": 4512,
      "protocol": "6150001",
      "cadcode": "202.43.12.42.095.042",
      "kind": "systematic",
      "zone": "43",
      "state": "awaiting_field",
      "state_label": "ველზე — პასუხის მოლოდინში",
      "verdict": "negative",
      "codes": ["GEO-02"],
      "checked_at": "2026-10-22T11:04:31",
      "operator": "ოპერატორი ა",
      "field_manager": "მენეჯერი ბ",
      "surveyors": ["ამზომველი გ", "ამზომველი დ"],
      "managers": ["მენეჯერი ბ"]
    }
  ],
  "limit": 2,
  "offset": 0
}
```

შესვლის გარეშე:

```
HTTP/1.1 401 Unauthorized
{"error": "login_required"}
```

სხვისი ოქმი:

```
GET /api/items/9999        → HTTP/1.1 404 Not Found   {"error": "ვერ მოიძებნა"}
```

მარტივად: პროგრამა ეკითხება: „მომეცი ველზე მყოფი ოქმები, მაქსიმუმ 2“. სერვერი აბრუნებს მხოლოდ იმ ოქმებს, რომლებიც **ამ მომხმარებელს** ეკუთვნის, თავისი ნომრით, კოდით, სტატუსით, ოპერატორით და ამზომველებით. შესვლის გარეშე პასუხი არის „შედი ჯერ“, სხვისი ოქმის მოთხოვნაზე კი „ასეთი არ არსებობს“.

4. რატომ. მომავალში:

- Pre-QC შედეგებს API-ით მოგვაწვდის;
- სააგენტოს სხვა სისტემას შეუძლია QC სტატუსის წაკითხვა;
- შესაძლოა, მობილური აპლიკაციაც გაჩნდეს.

5. კულისებში. FastAPI ფუნქციის პარამეტრებს (`state` , `limit` , `offset`) ავტომატურად კითხულობს და ამოწმებს. `limit` შეზღუდულია 200-ით (`min(limit, 200)`), რომ ვინმემ მთელი ბაზა ერთ მოთხოვნაში არ „ჩამოქაჩოს“.

6. რა შეიძლება აირიოს.

- ⚠️ **C1:** კოორდინატორი `/api/items` -ზე 403-ს იღებს, თუმცა გვერდზე გავლილ ოქმებს ხედავს. გასასწორებელი შეუსაბამობაა;
- 💡 მანქანა-მანქანასთან (machine-to-machine) კავშირისთვის cookie არ გამოდგება. საჭიროა API გასაღები (`Authorization: Bearer …`). ასეა Pre-QC hosted-ში: გასაღებს მხოლოდ ერთი ნებართვა აქვს (`preqc:write`), ბაზაში hash-ად ინახება, და cookie ingest-ზე განზრახ არ მუშაობს.

7. IT იკითხავს. „API დოკუმენტირებულია? ვერსიირებულია? ავთენტიფიკაცია?“

8. პასუხი. „OpenAPI ავტომატურად გენერირდება (`/docs`). დღეს API ადამიანის სესიით მუშაობს. სისტემებს შორის ინტეგრაციისთვის ცალკე API გასაღებებს დავამატებთ, მინიმალური უფლებით, როგორც Pre-QC-ის hosted ნაწილშია. ვერსიირება (`/api/v1`) საწარმოომდე დაემატება.“

C.4 WFS — რუკის ობიექტების წაკითხვა

1. ტექნიკური განმარტება. WFS (Web Feature Service) არის OGC-ის სტანდარტი გეოგრაფიული ობიექტების (feature) HTTP-ით მისაღებად, გეომეტრიითა და ატრიბუტებით. მთავარი ოპერაციებია `GetCapabilities` , `DescribeFeatureType` და `GetFeature` . ფილტრაციისთვის გამოიყენება OGC Filter ან (GeoServer-ზე) CQL. **WMS** სხვა რამეა: ის რუკის **სურათს** აბრუნებს და არა ობიექტებს.

2. მარტივად რომ ავხსნათ. WFS რუკის „API-ა“. ვეკითხებით: „მომეცი ნაკვეთი, რომლის საკადასტრო კოდია ესა და ეს“, და ვიღებთ მის ზუსტ საზღვრებს (კოორდინატებს) და მონაცემებს. ამით პროგრამა ამოწმებს, მაგალითად, ნაკვეთი მეზობელს ხომ არ ეფარება.

3. ჩვენს პროექტში. ✅ WFS-ს Pre-QC Validator იყენებს (`pre-qc-validator-master/app/wfs.py`); პლატფორმა დღეს WFS-ს **არ** იყენებს. მოთხოვნის სტრუქტურა (რეალური პარამეტრები, ფილტრის მნიშვნელობა ილუსტრაციაა):

```
GET https://wblr.napr.gov.ge/data/SLR/ows
    ?service=WFS&version=2.0.0&request=GetFeature
    &typeNames=SLR:GFLD_PARCELS
    &srsName=EPSG:32638
    &outputFormat=application/json
    &propertyName=SHAPE,TAG,ZONE,SECTOR,CADCODE,FUNCTION,CATEGORY,GFLD_PARCEL_ID,...
    &cql_filter=CADCODE='202.43.12.42.095.042'
Authorization: Basic <მომხმარებელი:პაროლი, base64>
```

პასუხი GeoJSON-ია:

```
{"type": "FeatureCollection",
 "features": [{"type": "Feature",
               "geometry": {"type": "Polygon", "coordinates": [[[480123.4, 4620456.7], …]]},
               "properties": {"CADCODE": "202.43.12.42.095.042", "ZONE": "43", …}}]}
```

მარტივად: ვეკითხებით SLR-ის რუკის სერვისს: „მომეცი ეს ნაკვეთი, მეტრებში (`EPSG:32638`), JSON-ად და მხოლოდ ეს ველები“. ვიღებთ პოლიგონს, ანუ საზღვრის წერტილებს მეტრებში, და ატრიბუტებს.

4. რატომ. Pre-QC-ის GIS-შემოწმებები (ფართობი, გადაფარვა, მეზობლები) ამაზე დგას.

5. კულისებში.

- ✅ Validator პასუხში მხოლოდ საჭირო ატრიბუტებს ითხოვს (`PROPERTIES`). წინათ ყველა ატრიბუტი მოდიოდა, მისამართისა და პირის იდენტიფიკატორის ჩათვლით, და ეს მონაცემის მინიმიზაციისთვის შეიცვალა;
- ✅ წარმატებული პასუხები 300 წამით ქეშდება (`WFS_CACHE_TTL_S`);
- ✅ ქსელის დროებით შეცდომაზე ერთხელ მეორდება;

- ✅ წაკითხვის timeout 90 წამია და არ მეორდება.

6. რა შეიძლება აირიოს.

- ⚠️ ავთენტიფიკაცია HTTP Basic-ია, `.env`-ში შენახული მომხმარებლით და პაროლით. ❓ ვისი ანგარიშია ეს, და დასაშვებია თუ არა სერვერზე — IT-ის საკითხია;
- სერვერის IP შეიძლება WFS-ზე დაშვებული არ იყოს;
- მძიმე CQL ფილტრი WFS სერვერს ტვირთავს.

7. IT იკითხავს. „WFS-ზე რა წვდომა გჭირდებათ?“

8. პასუხი. „Pre-QC-ისთვის: მხოლოდ GetFeature, მხოლოდ კითხვა, SLR-ის რამდენიმე ფენაზე (ნაკვეთები, შენობები, ხაზები, სერვიტუტები). სერვისის ანგარიში გვჭირდება და არა ადამიანის პირადი, და სერვერის IP-ის დაშვება. პლატფორმის პირველ ეტაპს WFS არ სჭირდება.“

C.5 SQL view, როგორც ინტეგრაციის გზა

იხ. B.9. მოკლედ: IT ქმნის view-ს მხოლოდ საჭირო ველებით და გვაძლევს მომხმარებელს, რომელსაც მხოლოდ `SELECT`-ის უფლება აქვს ამ view-ზე. ჩვენი ფონური worker ღამით კითხულობს და ჩვენს ბაზაში ინახავს.

✅ დადებითი	⚠️ უარყოფითი
ყოველთვის ახალი მონაცემი; IT აკონტროლებს, რა ჩანს; ერთი SQL-ის სამუშაოა IT-ისთვის	ქსელური წვდომა DB სერვერზე (firewall, 5432/1433); დატვირთვა წყაროზე; სტრუქტურის ცვლილება ჩვენს იმპორტს ტეხს

მარტივად რომ ავხსნათ: IT თავის არქივში პატარა „ფანჯარას“ გვიჭრის. ჩვენ ღამით ამ ფანჯარაში ვიყურებით და საჭირო სვეტებს ვიწერთ. ფანჯრიდან სხვა არაფერი ჩანს და ვერაფერს შევცვლით. ერთადერთი პირობა: ფანჯარამდე „გზა“ უნდა გაიხსნას (firewall), და ფანჯრის ფორმა წინასწარ შეთანხმებული არ უნდა შეიცვალოს.

C.6 ფაილის ექსპორტი: ქსელური საქაღალდე (SMB) და SFTP

1. ტექნიკური განმარტება.

- **SMB/CIFS** — Windows-ის ქსელური საქაღალდე (`\\server\share`, ან მიერთებული დისკი, მაგ. `Z:`), პორტი 445. წვდომა Windows/AD ანგარიშით ხდება;
- **SFTP** — ფაილის გადაცემა SSH-ით (პორტი 22), გასაღებით ან პაროლით;
- ექსპორტის ფორმატები: GDB (Esri File Geodatabase), CSV, XLSX, JSON.

ფაილზე დაფუძნებული ინტეგრაცია „batch“ ტიპისაა: მონაცემი პერიოდულად, მზა პაკეტად მოდის.

2. მარტივად რომ ავხსნათ. IT ყოველ ღამე „ამანათს“ (ფაილს) დებს ერთ თაროზე (საქაღალდეში). ჩვენი სისტემა დილით იღებს და ხსნის. მარტივია და IT-ს სისტემას ვერ ვეხებით, მაგრამ მონაცემი მხოლოდ ამანათის დადების დროისაა.

3. ჩვენს პროექტში.

- ✅ REGFIELDALL დღეს სწორედ ასე მოდის: ღამით `Z:\REGFIELDALL\REGFIELDALL_<თარიღი>_01-00_<N>.gdb` (ბოლო: `REGFIELDALL_2026-10-19_01-00_93.gdb`). Distributor ბოლოს ავტომატურად პოულობს;
- ✅ სპორადული დღეს xlsx-ად მეილით მოდის;
- ⚠️ `Z:` დისკი **ჩემს კომპიუტერზეა მიერთებული**, ჩემი Windows ანგარიშით. სერვერს ეს დისკი არ აქვს.

4. რატომ. თუ IT ბაზაზე წვდომას არ მოგვცემს, ფაილი ყველაზე რეალისტური გზაა. ის უკვე არსებობს და მუშაობს.

5. კულისებში. GDB-ის წასაკითხად GDAL ბიბლიოთეკა სჭირდება (Distributor მას QGIS-ის Python-ით იყენებს). ✅ მხოლოდ საჭირო ველებს კითხულობს, გეომეტრიის გარეშე: ≈ 2.27–2.33 მლნ ჩანაწერი 40–60 წამში. სერვერზე GDAL-ის დაყენება ცალკე ამოცანაა 💡. ალტერნატივაა, IT-მ CSV ექსპორტი გააკეთოს.

6. რა შეიძლება აირიოს.

- ფაილი არ გაჩნდა ან ნახევრად ჩაწერილი წავიკითხეთ. ამიტომ სასურველია „მზადაა“ ნიშანი (ფაილის სახელის წესი ან ცალკე `.done` ფაილი);
- ფორმატი ან სვეტების სახელები შეიცვალა;
- ძველი ფაილები საქაღალდეს ავსებს.

7. IT იკითხავს. „Z:-ის საქაღალდეზე სერვერს წვდომას მოგცემთ — საკმარისია?“

8. პასუხი. „კი, პირველ ეტაპზე საკმარისია. ამისთვის სერვისის ანგარიში გვჭირდება, რომელსაც ამ საქაღალდის მხოლოდ წაკითხვა შეუძლია, და GDAL სერვერზე. თუ CSV ექსპორტი შესაძლებელია, ის უფრო მარტივია. გრძელვადიანად view გვირჩევნია.“

C.7 პირდაპირი წვდომა სხვა სისტემის ბაზაზე

1. ტექნიკური განმარტება. აპლიკაცია წყარო სისტემის ცხრილებს პირდაპირ კითხულობს, view-ის გარეშე.

2. მარტივად რომ ავხსნათ. ეს ნიშნავს, რომ ჩვენ თვითონ შევდივართ სხვის არქივში და თვითონ ვეძებთ, არქივარიუსის „ფანჯრის“ გარეშე. სწრაფია, მაგრამ ვინც არქივს მართავს, მას ეს არ მოსწონს, და სწორადაც.

3. ჩვენს პროექტში. ✅ პლატფორმა Distributor-ის SQLite ფაილს პირდაპირ კითხულობს, `mode=ro`-თი (`source.connect()`). ეს ჩვენი საკუთარი, ლოკალური ასლია და არა სააგენტოს ბაზა.

4. რატომ ვიცნობთ. IT შეიძლება შემოგვთავაზოს: „მოგცემთ წვდომას ცხრილზე“.

5. კულისებში. პირდაპირი წვდომა ჩვენ შიდა სტრუქტურაზე გვაბამს. მცირე ცვლილებაც კი წყაროში ჩვენს კოდს გატეხს, და უფლებების მართვაც რთულდება.

6. რა შეიძლება აირიოს. მძიმე შეკითხვა საწარმოო ბაზას ანელებს. ამიტომ ან რეპლიკა, ან view, ან ღამის დრო.

7–8. პასუხი IT-ს: „პირდაპირ წვდომას view გვირჩევნია, ან რეპლიკაზე წვდომა. ასე თქვენი სისტემა ჩვენი შეკითხვებისგან დაცულია.“

C.8 Apps Script Web App — დღევანდელი ხიდი

1. ტექნიკური განმარტება. Google Apps Script-ის Web App არის `doGet` / `doPost` ფუნქცია, რომელიც Google-ის სერვერზე, სკრიპტის **მფლობელის** უფლებით სრულდება ("Execute as: Me") და საჯარო URL-ზეა გამოქვეყნებული ("Who has access: Anyone"). ავთენტიფიკაცია მხოლოდ query string-ში გადაცემული საერთო token-ით ხდება.

2. მარტივად რომ ავხსნათ. ეს არის კარი Google ცხრილში, რომელიც ინტერნეტიდან იღება, ვისაც „პაროლიანი ბმული“ აქვს. კარის მიღმა სკრიპტი ცხრილის პატრონის სახელით მოქმედებს.

3. ჩვენს პროექტში. ✅

- `fetch_sheets.py` ორ ასეთ Web App-ს იყენებს: QCStatRead (კონტროლის ჩანართები, მხოლოდ კითხვა) და Field Group (ამზომველი → მენეჯერი). მხოლოდ GET-ით;
- token-ები სხვა პროექტების კონფიგურაციიდან იკითხება და არსად იბეჭდება;
- ✅ Field Group-ის სკრიპტი მხოლოდ სახელსა და მენეჯერს აბრუნებს. პირადი ნომერი, ანაზღაურება და ტელეფონი გარეთ არ გადის (`NAPR_Distributor/README.md`);
- ⚠️ ძველი Distributor `ControlWebApp.gs`-ით **წერს** კონტროლის ცხრილში, ჩვენი პლატფორმა კი — არა.

4. რატომ ვიცნობთ. IT-ის უსაფრთხოების თანამშრომელი ამაზე აუცილებლად იკითხავს: „საჯარო URL, რომელიც token-ით შიდა ცხრილს აბრუნებს?“

5. კულისებში.

- ✅ Apps Script-ს 6-წუთიანი შესრულების ლიმიტი აქვს ჩვეულებრივ ანგარიშზე (Workspace-ზე 30 წუთი), ამიტომ წაკითხვა 8 000-მწკრივიანი გვერდებით ხდება;
- ✅ დიდი ცხრილის წაკითხვას ~85 წამი სჭირდება და ხანდახან ვარდება. ამიტომ `fetch_sheets.py` ოთხჯერ ცდილობს, მზარდი პაუზით.

6. რა შეიძლება აირიოს.

- token გაჟონა → ნებისმიერს შეუძლია ცხრილის ნაწილის წაკითხვა;
- სკრიპტის მფლობელმა დატოვა ორგანიზაცია → ხიდი ქრება.

7. IT იკითხავს. „ეს გამოსავალი საწარმოოში დარჩება?“

8. პასუხი. „არა. ეს დროებითი ხიდია, მხოლოდ კითხვისთვის: ძველი მონაცემის ერთჯერადი გადმოტანისთვის და Field Group-ისთვის. საწარმოოში გუნდებს ადმინი პლატფორმაში მართავს, ძველი ცხრილი კი მიგრაციის შემდეგ მხოლოდ საკითხავი არქივი ხდება. სანამ ხიდი არსებობს, token-ის როტაცია შეგვიძლია.“

C.9 fetch, სინქრონიზაცია, იმპორტი და მუდმივი შენახვა — რა განსხვავებაა

1. ტექნიკური განმარტება.

ტერმინი	რას ნიშნავს	მაგალითი ჩვენთან
ცოცხალი წაკითხვა (live read / fetch on demand)	მოთხოვნისას პირდაპირ წყაროს ვეკითხებით, და არაფერი ინახება	Pre-QC-ის WFS მოთხოვნა ოქმის შემოწმებისას
ქეში (cache)	ცოცხალი წაკითხვის შედეგს დროებით ვინახავთ (TTL), რომ განმეორებით არ ვიკითხოთ	WFS ქეში 300 წმ
იმპორტი (one-off import)	ერთჯერადი გადმოტანა	ძველი კონტროლის ცხრილი → `qc_items`
სინქრონიზაცია (sync, periodic)	პერიოდულად ვიღებთ ცვლილებებს და ჩვენს ასლს ვანახლებთ: upsert, წაშლილის მონიშვნა	🗓 REGFIELDALL ღამით; Field Group ყოველდღე
მუდმივი შენახვა (system of record)	მონაცემი, რომლის **მთავარი წყარო** ჩვენი სისტემაა	QC ვერდიქტები, ისტორია, ველის პასუხები

2. მარტივად რომ ავხსნათ.

- ცოცხალი წაკითხვა ტელეფონით კითხვაა: „ახლა რა ხდება?“
- ქეში იგივე კითხვაა, ოღონდ პასუხს 5 წუთით ვიმახსოვრებთ.
- იმპორტი ერთხელ გადმოტანილი არქივია.
- სინქრონიზაცია ყოველდღიური „განახლებული სიაა“.
- მუდმივი შენახვა ის მონაცემია, რომლის „დედანიც“ ჩვენთანაა. ჩვენი სისტემის გარეთ ის არსად არსებობს.

3. ჩვენს პროექტში — რა სად უნდა იყოს 💡**:**

მონაცემი	მთავარი წყარო	ჩვენთან	რატომ
ოქმის ბაზისური ველები (ნომერი, კოდი, თარიღი, შემდგენი, სტატუსი)	REGFIELDALL	**სინქრონი ღამით** + ასლი QC შემთხვევაზე განაწილების მომენტისთვის	განაწილებას 2.3 მლნ ჩანაწერზე ანალიზი სჭირდება; ცოცხალი წაკითხვა წყაროს გადატვირთავდა
საველე სტატუსი	REGFIELDALL	სინქრონი	დღეში ერთხელ საკმარისია
გუნდები	Field Group → 💡 პლატფორმა	**პლატფორმა ხდება მთავარი წყარო**	ადმინი მართავს ისტორიით
QC სამუშაო	**პლატფორმა**	მუდმივი შენახვა	სხვაგან არ არსებობს
Pre-QC შედეგი	Validator	იმპორტი API-ით	Validator ცალკე სისტემაა

მონაცემი	მთავარი წყარო	ჩვენთან	რატომ
ნაკვეთის გეომეტრია	SLR WFS	ცოცხლად + ქეში (მხოლოდ Pre-QC-ში)	ცვალებადია და დიდი
ოქმის შინაარსი (შემსრულებლები)	REDI / 🅰 API	ცოცხლად ან განაწილებისას ასლი	❓ API-ზეა დამოკიდებული

4. რატომ. ეს გადაწყვეტილება განსაზღვრავს დატვირთვას წყაროზე, მონაცემის „სიახლეს“ და იმას, რა მოხდება, როცა წყარო მიუწვდომელია.

5. კულისებში. სინქრონის upsert (PostgreSQL):

```
INSERT INTO src_regfield (fld_paper_id, cadcode, mox_create_date, fld_status_name,
                          oqmi_creator, moxaz_creator, is_last, synced_at)
VALUES (…)
ON CONFLICT (fld_paper_id, cadcode) DO UPDATE
   SET fld_status_name = EXCLUDED.fld_status_name,
       is_last         = EXCLUDED.is_last,
       synced_at       = EXCLUDED.synced_at;
```

მარტივად: „თუ ეს ოქმი + კოდი უკვე გვაქვს, მხოლოდ სტატუსი განვაახლოთ; თუ არ გვაქვს, დავამატოთ“. ასე ყოველ ღამე სრული თავიდან აწყობა აღარ არის საჭირო. `src_regfield` შემოთავაზებული ცხრილის სახელია და ჯერ არ არსებობს.

6. რა შეიძლება აირიოს.

- სინქრონი ჩუმად ჩავარდა, და ყველა ძველ მონაცემზე მუშაობს. ამიტომ ეკრანზე უნდა ჩანდეს „ბოლო განახლება: …“. ✅ განაწილების გვერდი დღესაც აჩვენებს წყაროს GDB-ს სახელს და თარიღს (`source.source_info()`);
- სინქრონმა წყაროში გამქრალი ჩანაწერი წაშალა, თუმცა QC მასზე უკვე მუშაობდა. 💡 არასოდეს წავშალოთ, მხოლოდ მოვნიშნოთ (Pre-QC hosted-ის `is_stale` პრინციპი).

7. IT იკითხავს. „ცოცხლად კითხულობთ თუ ასლს ინახავთ? რამდენად ხშირად?“

8. პასუხი. „საჭირო ველების ასლს ვინახავთ და ღამით ვანახლებთ: ასე თქვენს სისტემას დღისით არ ვტვირთავთ, და მისი მიუწვდომლობისას ჩვენ ვმუშაობთ. QC-ის საკუთარი მონაცემის დედანი ჩვენთანაა. ცოცხლად მხოლოდ ის უნდა წავიკითხოთ, რაც სწრაფად იცვლება და მცირეა, მაგალითად Pre-QC-ის რუკის მონაცემი.“

C.10 რატომ მუშაობს დეველოპერის კომპიუტერზე და არა სერვერზე

1. ტექნიკური განმარტება. დეველოპერის სამუშაო სადგურს აქვს:

- ქსელური ადგილი (შიდა ქსელი, VPN, ნებადართული IP);
- მიერთებული ქსელური დისკები;
- ბრაუზერის ავთენტიფიცირებული სესიები (cookie-ები REDI-ში, SLR-ში);
- AD/Windows ანგარიშის უფლებები;
- კორპორატიული სერტიფიკატი (root CA) Windows-ის საცავში;
- proxy-ს პარამეტრები;
- ლოკალური `.env` / `config.json` საიდუმლოებებით;
- დაყენებული პროგრამები (QGIS, GDAL).

სერვერს არცერთი ეს ავტომატურად არ აქვს.

2. მარტივად რომ ავხსნათ. ჩემს კომპიუტერს „გასაღებების აცმა“ აქვს: ქსელურ დისკზეც შედის, REDI-შიც შესულია, სერტიფიკატიც სცნობს. სერვერი ახალი თანამშრომელივითაა, რომელსაც ჯერ არცერთი გასაღები არ მიუღია. თითო გასაღები ოფიციალურად უნდა გასცეს IT-მ.

3. ჩვენს პროექტში — კონკრეტული სია ✅:

რა მუშაობს ჩემთან	რატომ	სერვერზე რა გვჭირდება
REGFIELDALL `Z:\`-დან	ჩემს ანგარიშზე მიერთებული ქსელური დისკი	სერვისის ანგარიში + SMB წვდომა, ან view/SFTP
Distributor-ის SQLite	ფაილი ჩემს დისკზე	⇒ ზემოთ (წყარო პირდაპირ)
Apps Script-ის token-ები	სხვა პროექტების `config.json` ჩემს კომპიუტერზე	საწარმოოში — ჩანაცვლება (C.8)
Pre-QC REDI-დან	ჩემს ბრაუზერში REDI-ში შესული ვარ (გაფართოება `api.napr.gov.ge`)	API ან ექსპორტი (სერვერს ბრაუზერის სესია არ აქვს)
SLR WFS	`.env`-ში მომხმარებელი/პაროლი; ჩემი IP ნებადართულია	სერვისის ანგარიში + სერვერის IP ❓
Google OAuth / SMTP TLS	`truststore` Windows-ის საცავიდან კორპორატიულ სერტიფიკატს იღებს	სერვერის OS-ის საცავში სააგენტოს CA + გამავალი წვდომა 443/587-ზე
`pip install`	ჩემს კომპიუტერს ინტერნეტი აქვს	შიდა სარკე ან მზა image (A.11)

4. რატომ. IT-ის კითხვა „თქვენთან ხომ მუშაობს?“ ხშირია. პასუხად ზუსტი სია უნდა გვქონდეს.

5. კულისებში.

- სერვერი ჩვეულებრივ ცალკე ქსელურ სეგმენტშია, საკუთარი firewall-ის წესებით;
- „გამავალი“ კავშირი (სერვერიდან ინტერნეტში) ხშირად სრულად დაკეტილია, ნებართვების სიის (allowlist) გარდა.

6. რა შეიძლება აირიოს. პირველი გაშვება სერვერზე „ყველაფერი კარგად“, მაგრამ ღამის იმპორტი ჩავარდა, რადგან `Z:` არ არსებობს. ✅ ამიტომ ყველა ბილიკი გარემოს ცვლადშია (`QCP_DISTRIBUTOR_DB` და სხვ.).

7–8. პასუხი IT-ს: „ჩემთან მუშაობს იმიტომ, რომ ჩემი ანგარიშით ქსელურ დისკზე, REDI-სა და WFS-ზე ვარ შესული. სერვერისთვის თითოეულის ოფიციალური ეკვივალენტი გვჭირდება: სერვისის ანგარიში, firewall-ის წესი და, სადაც შესაძლებელია, view ან API. სია დოკუმენტ 04-შია.“

C.11 რა ხდება, როცა გარე წყარო მიუწვდომელია

1. ტექნიკური განმარტება. გამძლეობის (resilience) პრინციპები:

- **graceful degradation** — წყაროს ჩავარდნა მთელ სისტემას არ აჩერებს;
- **timeout**-ები, რომ ლოდინი უსასრულო არ იყოს;
- **retry** exponential backoff-ით, მხოლოდ გარდამავალ შეცდომებზე;
- **circuit breaker** — ხშირი ჩავარდნისას დროებით ვწყვეტთ მცდელობას;
- **stale-while-error** — ძველი მონაცემის ჩვენება, „მოძველების“ ნიშნით;
- იმპორტის **ჟურნალი** და **განგაში**;
- **იდემპოტენტურობა** — გამეორებული იმპორტი დუბლიკატს არ ქმნის.

2. მარტივად რომ ავხსნათ. თუ ღამის „ამანათი“ არ მოვიდა, სისტემა არ ჩერდება. ის გუშინდელი სიით აგრძელებს მუშაობას, ადმინს ეუბნება „დღეს ახალი მონაცემი არ მოსულა“, და ხვალ ისევ ცდის.

3. ჩვენს პროექტში.

- ✅ QC-ის მთელი სამუშაო პროცესი (შემოწმება, პასუხები, სტატისტიკა) **მხოლოდ ჩვენს ბაზაზეა დამოკიდებული**. წყაროების ჩავარდნა მას არ აჩერებს, მხოლოდ ახალი განაწილება ფერხდება;
- ✅ `fetch_sheets.py`: 4 ცდა, 5/10/15/20 წმ პაუზით, timeout 330 წმ;

- ✅ განაწილების გადახედვა წყაროს ფაილის არარსებობისას გასაგებ შეცდომას აბრუნებს (`FileNotFoundError` → „განაწილების ბაზა ვერ მოიძებნა“);
- ✅ ძველ პანელში „ამომრთველი“ იყო (30 წმ). Pre-QC-ში — ერთი retry და ქეში, რომელიც მხოლოდ წარმატებულ პასუხს ინახავს;
- 💡 საწარმოოში: `import_runs` -ში ჩაწერა, განგაში ადმინთან, „ბოლო განახლების“ ნიშანი განაწილების გვერდზე.

4. რატომ. სააგენტოს სისტემები ხანდახან ახლდება ან ტექნიკურ სამუშაოზეა გაჩერებული.

5. კულისებში. იდემპოტენტური იმპორტი = upsert (C.9) + უნიკალური გასაღები. იგივე ფაილის ორჯერ წაკითხვა შედეგს არ ცვლის.

6. რა შეიძლება აირიოს. რამდენიმე დღე ჩუმი ჩავარდნა, და განაწილება ძველ მონაცემზე.

7. IT იკითხავს. „ჩვენი სისტემა რომ გაჩერდეს, თქვენი იმუშავებს?“

8. პასუხი. „კი. QC სამუშაო მხოლოდ ჩვენს ბაზაზეა. წყაროს გაჩერება ახალ განაწილებას დააყოვნებს, მაგრამ მიმდინარე სამუშაოს — არა. იმპორტი ხელახლა ცდის და ადმინს ატყობინებს.“

C.12 მონაცემის წყაროები — რა რა არის და რა ვიცით

წყარო	რა არის (ჩვენი გაგებით)	რას ვიყენებთ	წვდომის დადასტურებული გზა	უცნობი ❓
REGFIELDALL	საველე ოქმების სრული ამონაწერი (ფენა `NAPR_VEXP_FLD_GPARCELS_4GDB`), ღამის GDB ექსპორტით	11 ველი: `objectid` , `cadcode` , `fld_paper_id` , `app_id` , `regnumber` , `is_last` , `fld_status_name` , `moxaz_creator` , `mox_create_date` , `oqmi_creator` , `moxazulobis_avtori` ; პლატფორმა 7-ს იყენებს	✅ `Z:\REGFIELDALL\*.gdb` → Distributor → SQLite	საიდან იქმნება ექსპორტი (რომელი ბაზიდან), view შესაძლებელია თუ არა, ექსპორტის დრო (სახელში `01-00`)
REDI	ოქმების/რეგისტრაციის ვებ-სისტემა (`https://api.napr.gov.ge/lr/redi…`): ოქმის გვერდი (`REDI.VIEW.PAGE`), ბარათები (`DYNAMIC.ABSTRACT.PAGE` , `REGCARD`), ფოტოები (`bs.napr.gov.ge/GetBlob`)	Pre-QC	✅ თანამშრომლის ბრაუზერის სესია + გაფართოება	სერვერისთვის API არსებობს? ოქმის შემსრულებლების API?
SLR	მიწის სისტემური რეგისტრაციის GIS სერვისი: WFS `https://wblr.napr.gov.ge/data/SLR/ows` , ფენები `SLR:GFLD_PARCELS` , `GFLD_BUILDINGS` , `GFLD_TOPOLINES` , `GFLD_EASEMENTS`	Pre-QC GIS	✅ HTTP Basic	სერვისის ანგარიში, IP-ის შეზღუდვა, SLR განაცხადების API (`გეგმა.docx` : „ახლა SLR-ზე და REDI-ზე ავტორიზაცია გვიწევს“)
LR_BUILDINGS	რეგისტრირებული შენობების WMS	Pre-QC (არჩევითი)	✅ `LR_WMS_URL` `.env` -ში	—

წყარო	რა არის (ჩვენი გაგებით)	რას ვიყენებთ	წვდომის დადასტურებული გზა	უცნობი ❓
სპორადულის სია	დათვალიერებების ყოველდღიური ექსპორტი (20.10-ის ფაილი: 50 320 ჩანაწერი)	10 სვეტი	✅ xlsx მეილით	საიდან იქმნება, view შესაძლებელია?
Field Group	Google ცხრილი: FIELD_GROUP_272 (დღევანდელი) + სრული (წასულებით)	სახელი, მენეჯერი	✅ Apps Script GET	— (საწარმოოში ჩანაცვლდება)
კონტროლის ცხრილი	Google ცხრილი — დღევანდელი QC	ყველაფერი (მიგრაცია)	✅ Apps Script GET	—
QC სამუშაო.xlsx	ძველი პერიოდი + სახელების რუკა (Reference / TblPlannerMap, 591 ალიასი)	ალიასები	✅ ლოკალური ფაილი	—

მარტივად რომ ავხსნათ:

- **REGFIELDALL** ყველა აზომილი ნაკვეთის „დიდი სიაა“: რა აიზომა, როდის და ვინ;
- **REDI** ის სისტემაა, სადაც თავად ოქმი „ცხოვრობს“, ბარათებითა და ფოტოებით;
- **SLR** რუკაა, სადაც ნაკვეთის საზღვრები ჩანს;
- **Field Group** გვეუბნება, ვინ ვისი გუნდიდანაა;
- **სპორადულის სია** — ვისთან იყო დათვალიერება გუშინ.

პლატფორმას დღეს პირველი, მეოთხე და მეხუთე სჭირდება. REDI და SLR მხოლოდ Pre-QC-ისთვისაა საჭირო.

C.13 მონაცემის გზა — წყაროდან ეკრანამდე

```
1. სააგენტოს სისტემა (REGFIELDALL)      → ღამის ექსპორტი / 🅸 view
2. ჩვენი იმპორტი (ფონური worker)          → upsert ჩვენს ბაზაში (src ცხრილი), ჟურნალი
3. განაწილება (QC მენეჯერი)              → qc_items + item_people + qc_events
4. მოთხოვნა ბრაუზერიდან                    → HTTPS → proxy → uvicorn → FastAPI
5. ავთენტიფიკაცია                          → cookie → sessions → users (აქტიურია?)
6. ავტორიზაცია                             → scope.items_query(ctx): მხოლოდ ჩემი ჩანაწერები
7. შეკითხვა ბაზაში                         → SQLAlchemy → SQL → PostgreSQL → მწკრივები
8. HTML-ის აწყობა                          → Jinja2 შაბლონი (templates/qc/list.html)
9. პასუხი                                  → ბრაუზერი ხატავს; HTMX ცალკეულ ნაწილებს ანახლებს
```

მარტივად რომ ავხსნათ: ოქმი ჯერ სააგენტოს სისტემაში ჩნდება, ღამით ჩვენთან „ჩამოდის“, QC მენეჯერი მას ოპერატორს უწერს. როცა ოპერატორი საიტს ხსნის, სერვერი ამოწმებს, ვინ არის, პოულობს მხოლოდ მის ოქმებს, აწყობს გვერდს და უგზავნის. ეს ყველაფერი წამის ნაწილში ხდება.

D. უსაფრთხოება და ავთენტიფიკაცია

D.1 დომენი და DNS

1. ტექნიკური განმარტება. დომენი ადამიანისთვის წაკითხვადი სახელია (`qc.napr.gov.ge` — ეს მაგალითია ❓). DNS (Domain Name System) სახელს IP მისამართად თარგმნის (A/AAAA ჩანაწერი) ან სხვა სახელზე მიუთითებს (CNAME). შიდა DNS სახელს მხოლოდ შიდა ქსელში ხსნის, საჯარო DNS — ყველგან. „split-horizon“ DNS-ში ერთი სახელი შიგნით და გარეთ სხვადასხვა IP-ზე მიდის.

2. მარტივად რომ ავხსნათ. დომენი საიტის „მისამართის სახელია“. ადამიანი წერს `qc.napr.gov.ge` და არა `10.20.30.40`-ს. DNS „ცნობარია“, რომელიც სახელს ნომრად თარგმნის.

3. ჩვენს პროექტში.

- ✅ დღეს დომენი არ არის: `127.0.0.1:8200`, ან LAN-ზე კომპიუტერის IP;
- 💡 საწარმოოში სააგენტოს ქვედომენი გვჭირდება;
- ✅ დომენი კოდში არ წერია: ის `QCP_BASE_URL`-ით მიეთითება, რომელიც მეილის ბმულებსა და Google-ის redirect URI-ს აწყობს.

4. რატომ.

- Google OAuth-ს სტაბილური HTTPS მისამართი სჭირდება (redirect URI);
- მეილის ბმულები ამ მისამართზე მიდის;
- თანამშრომლებმა ერთი მისამართი უნდა დაიმახსოვრონ.

5. კულისებში. ბრაუზერი ჯერ DNS-ს ეკითხება, შემდეგ IP-ს უკავშირდება, და TLS-ის დროს სერტიფიკატს ამოწმებს: ემთხვევა თუ არა დომენს.

6. რა შეიძლება აირიოს.

- შიდა დომენი ტელეფონიდან, VPN-ის გარეშე, არ იხსნება;
- დომენის შეცვლა Google client-ში redirect URI-ის განახლებას მოითხოვს.

7. IT იკითხავს. „რა სახელი გინდათ?“

8. პასუხი. „თქვენი სახელების წესით, მაგალითად `qc.<სააგენტოს დომენი>`. მთავარია, ერთი სახელი იყოს, შიგნითაც და (თუ გარე წვდომა იქნება) გარეთაც.“

D.2 HTTPS, TLS და სერტიფიკატი

1. ტექნიკური განმარტება. TLS (Transport Layer Security) არის შიფრაციის პროტოკოლი. HTTPS = HTTP TLS-ზე. TLS უზრუნველყოფს:

- **კონფიდენციალურობას** — დაშიფვრა;
- **მთლიანობას** — ცვლილების აღმოჩენა;
- **სერვერის ავთენტიფიკაციას** — სერტიფიკატით.

სერტიფიკატს სანდო ცენტრი (CA) გასცემს: საჯარო CA (Let's Encrypt, კომერციული) ან ორგანიზაციის შიდა CA. შიდა CA-ს სერტიფიკატს მხოლოდ ის მოწყობილობები ენდობა, რომლებზეც შიდა root CA დაყენებულია. ეს ტელეფონებზე ხშირად პრობლემაა. რეკომენდებულია TLS 1.2+, სასურველია 1.3.

2. მარტივად რომ ავხსნათ. HTTPS „დალუქული კონვერტია“. გზაში ვერავინ წაიკითხავს და ვერ შეცვლის. სერტიფიკატი კი „პირადობის მოწმობაა“, რომელიც ამტკიცებს: ეს ნამდვილად სააგენტოს საიტია და არა ყალბი.

3. ჩვენს პროექტში.

- ✅ დღეს HTTPS არ არის (localhost / LAN);
- ✅ cookie-ს `Secure` ნიშანი HTTPS-ზე ჩაირთვება (Google-ით შესვლისას), მაგრამ proxy-ს გამო იხ. G4;
- 💡 საწარმოოში HTTPS სავალდებულოა.

4. რატომ.

- Google OAuth redirect URI-ს HTTPS-ით მოითხოვს (localhost-ის გარდა);
- სესიის cookie, პაროლები და QC მონაცემი ღია ტექსტით ქსელში არ უნდა მოძრაობდეს, განსაკუთრებით ტელეფონიდან, საჯარო ქსელით.

5. კულისებში. TLS-ის დაწყებისას (handshake) სერვერი სერტიფიკატს აგზავნის, ბრაუზერი ამოწმებს სახელს, ვადას და ხელმოწერის ჯაჭვს, შემდეგ კი სესიის გასაღები იქმნება. 💡 TLS proxy-ზე სრულდება (A.6).

6. რა შეიძლება აირიოს.

- სერტიფიკატის ვადა ამოიწურა (საჭიროა ავტომატური განახლება ან კალენდარი);
- ტელეფონი შიდა CA-ს არ ენდობა და „საიტი სახიფათოა“-ს წერს;
- სერვერიდან გამავალი კავშირი Google-თან ვერ მყარდება კორპორატიული proxy-ს TLS შემოწმების (TLS inspection) გამო. ✅ `truststore` ამისთვისაა: ის OS-ის სერტიფიკატების საცავს იყენებს.

7. IT იკითხავს. „სერტიფიკატს ჩვენ მოგცემთ, თუ Let's Encrypt?“

8. პასუხი. „თქვენი სტანდარტით. თუ ველიდან, ტელეფონით, საჯარო წვდომა იქნება, საჯარო CA-ს სერტიფიკატი სჯობს, რომ ტელეფონებზე დამატებითი დაყენება არ დაგვჭირდეს.“

D.3 ავთენტიფიკაცია და ავტორიზაცია — განსხვავება

1. ტექნიკური განმარტება.

- **ავთენტიფიკაცია** (authN) — „ვინ ხარ?“: ვინაობის დადასტურება;
- **ავტორიზაცია** (authZ) — „რა შეგიძლია?“: უფლებების შემოწმება კონკრეტულ რესურსზე და მოქმედებაზე.

2. მარტივად რომ ავხსნათ. ავთენტიფიკაცია შენობის შესასვლელშია: „პირადობა მაჩვენე“. ავტორიზაცია ყოველ კართანაა: „ამ ოთახში შესვლის უფლება გაქვს?“

3. ჩვენს პროექტში. ✅

- ავთენტიფიკაციას Google აკეთებს (🅘) ან დემო შესვლა. ჩვენ ვამოწმებთ, რომ ელფოსტა ჩვენთან რეგისტრირებული და აქტიურია;
- ავტორიზაციას **მხოლოდ ჩვენ** ვაკეთებთ:
  - როლი: `security.require()`;
  - საკუთრება: `scope.py`;
  - მოქმედება და მდგომარეობა: `workflow.available_actions()`.

4. რატომ. Google-მა იცის, რომ ეს „ნამდვილად გიორგია“, მაგრამ არ იცის, რომ გიორგი QC მენეჯერია. ამას ჩვენი ბაზა განსაზღვრავს.

5–8. იხ. D.4–D.9.

D.4 OAuth 2.0 და OpenID Connect — როგორ იმუშავებს Google-ით შესვლა

1. ტექნიკური განმარტება.

- **OAuth 2.0** არის ავტორიზაციის ჩარჩო: ერთი აპლიკაცია მეორის რესურსზე წვდომას იღებს მომხმარებლის თანხმობით.
- **OpenID Connect (OIDC)** OAuth 2.0-ის ზედა ფენაა **ავთენტიფიკაციისთვის**: მისი შედეგი **ID token-ია** (JWT), რომელიც შეიცავს claim-ებს — `iss` (გამცემი), `aud` (ვისთვის), `exp` (ვადა), `sub` (მომხმარებლის ID), `email`, `email_verified`, `hd` (Workspace დომენი), `nonce`.
- ჩვენ ვიყენებთ **Authorization Code Flow + PKCE**-ს, რომელიც ვებ-აპლიკაციისთვის ყველაზე უსაფრთხო სტანდარტული ნაკადია.

2. მარტივად რომ ავხსნათ. პაროლს ჩვენ საერთოდ არ ვხედავთ და არ ვინახავთ. „Google-ით შესვლაზე“ დაჭერისას თანამშრომელი Google-ის საკუთარ გვერდზე გადადის და იქ შედის. Google ჩვენს საიტს უბრუნებს დალუქულ „ცნობას“: „ეს ადამიანი არის ესა და ეს ელფოსტა, დადასტურებულია“. ჩვენ ვამოწმებთ, რომ ცნობა ნამდვილია და ჩვენთვისაა განკუთვნილი, ვპოულობთ ამ ელფოსტას ჩვენს სიაში და ვუშვებთ, მისი როლით.

3. ჩვენს პროექტში — ნაბიჯ-ნაბიჯ ✅ (`app/routes/google_auth.py`; ჩაირთვება, როცა `QCP_GOOGLE_CLIENT_ID` და `QCP_GOOGLE_CLIENT_SECRET` მიეთითება):

```
1. მომხმარებელი → GET /auth/google
2. სერვერი ქმნის შემთხვევით state-ს, nonce-ს და PKCE verifier-ს;
   ინახავს ხელმოწერილ, 10-წუთიან cookie-ში (qcp_oidc, HttpOnly).
3. გადამისამართება → https://accounts.google.com/o/oauth2/v2/auth
     ?client_id=…&redirect_uri=https://<დომენი>/auth/google/callback
     &response_type=code&scope=openid email profile
     &state=…&nonce=…&code_challenge=<SHA256(verifier)>&code_challenge_method=S256
     &prompt=select_account[&hd=<დომენი>, თუ მითითებულია]
4. მომხმარებელი Google-ში შედის (პაროლი / 2FA — Google-ის მხარეს).
5. Google → GET /auth/google/callback?code=…&state=…
6. სერვერი: state == cookie-ში შენახულს? (თუ არა → უარი)
7. სერვერი → POST https://oauth2.googleapis.com/token
     (code + client_id + client_secret + redirect_uri + code_verifier)  — სერვერიდან, TLS-ით
8. Google → { id_token: … }
9. სერვერი ამოწმებს claim-ებს: iss ∈ {accounts.google.com}, aud == ჩვენი client_id,
   exp > ახლა, nonce == შენახული, email_verified == true, [hd == დომენი]
10. users-ში ეძებს email-ს, active = true. ვერ იპოვა → audit "login_denied" → „არ არის რეგისტრირებული"
11. იქმნება სესია (sessions), cookie qcp_session → მთავარი გვერდი
```

✅ ID token-ის ხელმოწერა ცალკე არ მოწმდება. მიზეზი: token პირდაპირ Google-ის token endpoint-იდან, TLS-ით მოდის, და ეს OpenID Connect Core-ის §3.1.3.7-ით დასაშვებია. ეს კოდის კომენტარშიც წერია.

4. რატომ.

- პაროლების შენახვის პასუხისმგებლობა ჩვენზე არ არის;
- Google-ის დაცვა მოქმედებს (2FA, საეჭვო შესვლის აღმოჩენა);
- თანამშრომელს ახალი პაროლი არ სჭირდება;
- ✅ ძველ სტატისტიკის პანელში პაროლების hash-ები ჩვენ გვქონდა შენახული (`users.json` , scrypt). ამას ვტოვებთ.

5. კულისებში.

- **state** იცავს CSRF-ისგან შესვლის ნაკადში: სხვისმა ბმულმა შენი სესია არ უნდა შექმნას;
- **nonce** იცავს ID token-ის განმეორებისგან (replay);
- **PKCE** იცავს მოპარული `code` -ისგან: verifier-ის გარეშე `code` უსარგებლოა.

6. რა შეიძლება აირიოს.

შეცდომა	მიზეზი
`redirect_uri_mismatch`	Google Console-ში ჩაწერილი URI ზუსტად არ ემთხვევა (`http` ↔ `https` , ბოლო `/` , პორტი)
„არ არის რეგისტრირებული“	✅ ელფოსტა ბაზაში არ არის. დღეს **0 ელფოსტაა** (G15)
`google_state`	cookie დაიკარგა: 10 წუთზე მეტი გავიდა, ან ბრაუზერი cookie-ს ბლოკავს
`google_token`	სერვერი `oauth2.googleapis.com` -ს ვერ უკავშირდება (firewall / proxy / სერტიფიკატი)
`google_domain`	`QCP_GOOGLE_ALLOWED_DOMAIN` მითითებულია, თანამშრომელი კი პირადი Gmail-ით შედის

7. IT იკითხავს. „Google-ის token-ებს ინახავთ? Google-ის მონაცემზე რა წვდომას ითხოვთ?“

8. პასუხი. „მხოლოდ ვინაობას ვითხოვთ: `openid email profile` . Gmail-ზე, Drive-ზე ან ცხრილებზე წვდომას არ ვითხოვთ. Google-ის token-ს არ ვინახავთ: ერთხელ ვიყენებთ ელფოსტის დასადგენად, შემდეგ ჩვენი სესია იქმნება. ვინ შედის, ჩვენი მომხმარებლების სია განსაზღვრავს, და არა Google.“

D.5 Client ID, Client Secret, Redirect URI, scope და consent screen

1. ტექნიკური განმარტება.

ტერმინი	რა არის	საიდუმლოა?
Google Cloud project	კონტეინერი, რომელშიც OAuth client იქმნება; ეკუთვნის ორგანიზაციას ან ანგარიშს	—
OAuth consent screen	რას ხედავს მომხმარებელი შესვლისას: აპლიკაციის სახელი, დომენი, კონტაქტი. ტიპი: **Internal** (მხოლოდ Workspace ორგანიზაციის მომხმარებლები) ან **External** (ნებისმიერი Google ანგარიში); External-ს აქვს სტატუსი Testing (მხოლოდ სატესტო მომხმარებლები) ან In production	არა
Client ID	აპლიკაციის საჯარო იდენტიფიკატორი (`….apps.googleusercontent.com`)	არა (ბრაუზერშიც ჩანს)
Client Secret	აპლიკაციის პაროლი Google-თან; მხოლოდ სერვერზე	**დიახ**
Redirect URI	სად აბრუნებს Google მომხმარებელს. უნდა ემთხვეოდეს **ზუსტად**	არა
Scope	რა წვდომას ითხოვს აპლიკაცია	—
`hd`	Workspace დომენის მინიშნება / claim	—

2. მარტივად რომ ავხსნათ.

- Client ID ჩვენი საიტის „სახელია“ Google-თან;
- Client Secret მისი „პაროლია“, რომელიც მხოლოდ სერვერმა იცის;
- Redirect URI ის ერთადერთი მისამართია, სადაც Google მომხმარებელს დააბრუნებს. სხვა საიტი ამ ნაკადს ვერ „მოიპარავს“;
- Scope არის „რა გვჭირდება“: ჩვენ მხოლოდ სახელი და ელფოსტა.

3. ჩვენს პროექტში.

- ✅ `QCP_GOOGLE_CLIENT_ID` , `QCP_GOOGLE_CLIENT_SECRET` , `QCP_GOOGLE_ALLOWED_DOMAIN` (არჩევითი);
- ✅ redirect URI: `<QCP_BASE_URL>/auth/google/callback` ;
- ✅ scope: `openid email profile` ;
- ✅ client-ის ტიპი: **Web application**.

თანამშრომლები პირადი Gmail-ით შედიან (გადაწყვეტილება, `IT_REQUEST.md` §6; მენეჯერები აგროვებენ ელფოსტებს). ❓ თუ სააგენტოს Google Workspace აქვს, შესაძლებელია ერთი დომენით შეზღუდვაც.

4. რატომ. IT-მ client სააგენტოს სახელზე უნდა შექმნას, რომ ის ორგანიზაციას ეკუთვნოდეს და არა ჩემს პირად ანგარიშს.

5. კულისებში — რას აკეთებს IT Google Console-ში (❓ ზუსტი ნაბიჯები Google-ის ინტერფეისის მიმდინარე ვერსიით უნდა გადამოწმდეს):

1. ქმნის პროექტს სააგენტოს ანგარიშზე;
2. აწყობს consent screen-ს: სახელი (მაგ. „NAPR QC“), მხარდაჭერის ელფოსტა, დომენი. ტიპი: Workspace-ზე Internal, პირადი Gmail-ებისთვის External → **In production** (Testing სტატუსში მხოლოდ ჩამოთვლილ სატესტო მომხმარებლებს შეუძლიათ შესვლა). `openid email profile` მგრძნობიარე scope-ები არ არის;
3. Credentials → OAuth client ID → **Web application** → Authorized redirect URIs: `https://<დომენი>/auth/google/callback` (პილოტისთვის, საჭიროებისას, სატესტო მისამართიც);
4. Client ID-სა და Secret-ს **უსაფრთხო არხით** გვაწვდის, ან პირდაპირ სერვერის საიდუმლოების საცავში წერს.

6. რა შეიძლება აირიოს.

- client ჩემს პირად Google ანგარიშზე შეიქმნა და ჩემთან ერთად „წავიდა“;
- secret მეილით გაიგზავნა;

- ⚠️ ერთი Cloud პროექტი სხვადასხვა client-ს აერთიანებს: Pre-QC hosted-ის README აფრთხილებს, რომ Google ნებართვებს პროექტის დონეზე აჯგუფებს. 💡 QC პლატფორმას ცალკე პროექტი სჭირდება;
- ✅ `include_granted_scopes`-ს ჩვენი კოდი არ იყენებს. სწორედ ამის გამო მიიღო Pre-QC hosted-მა ზედმეტი Sheets scope.

7. IT იკითხავს. „Workspace გვაქვს / არ გვაქვს — რა განსხვავებაა?“

8. პასუხი. „Workspace-ით შეგვიძლია შესვლა მხოლოდ სააგენტოს დომენზე შევზღუდოთ (Internal + `hd` შემოწმება, რომელიც კოდში უკვე არის). პირადი Gmail-ებით External client გვჭირდება. ორივე შემთხვევაში შედის მხოლოდ ის, ვისი ელფოსტაც ჩვენმა ადმინმა დაარეგისტრირა.“

D.6 სესია და cookie

1. ტექნიკური განმარტება. შესვლის შემდეგ სერვერი **სესიას** ქმნის და ბრაუზერს cookie-ს აძლევს. ორი მიდგომა არსებობს:

- **(ა) სერვერის მხარის სესია** — cookie-ში მხოლოდ იდენტიფიკატორია, მონაცემი ბაზაში ინახება. მყისიერი გაუქმება შესაძლებელია;
- **(ბ) თვითკმარი token** (JWT) — ყველაფერი token-შია. გაუქმება რთულია.

cookie-ს ნიშნები:

- `HttpOnly` — JavaScript ვერ კითხულობს (XSS-ისგან დაცვა);
- `Secure` — მხოლოდ HTTPS-ით იგზავნება;
- `SameSite=Lax` — სხვა საიტიდან POST-ზე არ იგზავნება (CSRF-ის შემცირება);
- `Max-Age` — ვადა.

2. მარტივად რომ ავხსნათ. შესვლის შემდეგ ბრაუზერი იღებს „სამაჯურს“ ნომრით, როგორც სასტუმროში. თვითონ სამაჯურზე არაფერი წერია, მხოლოდ ნომერი; ვინ ხარ და რა გეკუთვნის, მიმღების წიგნშია (ბაზაში). სამაჯურის გაყალბება არ შეიძლება (ხელმოწერილია), და თუ წიგნში გაუქმდა, სამაჯური მაშინვე უსარგებლოა.

3. ჩვენს პროექტში. ✅ `app/security.py`:

- `sessions` ცხრილი:
  - `sid` — 32 შემთხვევითი ბაიტი (`secrets.token_urlsafe(32)`);
  - `user_id`, `active_role`;
  - `expires_at` — 12 სთ;
  - `revoked_at`, `ip`, `user_agent`;
- cookie `qcp_session` = ხელმოწერილი `sid` (`itsdangerous`, საიდუმლო: `QCP_SESSION_SECRET`), `HttpOnly`, `SameSite=Lax`, `Secure` HTTPS-ზე;
- ყოველ მოთხოვნაზე მოწმდება: ხელმოწერა, სესიის არსებობა, `revoked_at`, ვადა, **მომხმარებლის** `active`;
- გასვლა (`POST /logout`) სესიას აუქმებს.

4. რატომ.

- თანამშრომლის გათიშვა მაშინვე მოქმედებს და არა 12 საათში;
- როლი cookie-ში საერთოდ არ წერია, ამიტომ მისი გაყალბება შეუძლებელია.

5. კულისებში. `read_session()` ბაზიდან სესიასა და მომხმარებელს კითხულობს. თუ `u.active == False`, მომხმარებელი ითვლება შეუსვლელად და გადამისამართდება `/login`-ზე.

6. რა შეიძლება აირიოს.

- `QCP_SESSION_SECRET` შეიცვალა → ყველა სესია გაუქმდება. ეს დასაშვებია, ოღონდ ყველას თავიდან მოუწევს შესვლა;
- secret გაჟონა → cookie-ის ხელმოწერის გაყალბება შესაძლებელი ხდება. ⚠️ დღეს secret `data/.session_secret` ფაილშია. 💡 საწარმოოში საიდუმლოების საცავში უნდა იყოს;

- ⚠️ G3: დემო შესვლის cookie-ს `Secure` ნიშანი არ აქვს, და დემო შესვლა ნაგულისხმევად ჩართულია.

7. IT იკითხავს. „სესია რამდენ ხანს გრძელდება? იძულებით გამოყვანა შესაძლებელია?“

8. პასუხი. „12 საათი. სესია სერვერზე ინახება, ამიტომ ნებისმიერი თანამშრომლის გამოყვანა მყისიერად შეგვიძლია: გათიშვით ან სესიის გაუქმებით. ვადა პარამეტრია.“

D.7 RBAC — როლებზე დაფუძნებული წვდომა

1. ტექნიკური განმარტება. RBAC (Role-Based Access Control): უფლებები როლებს ენიჭება, როლები კი მომხმარებლებს. ჩვენთან RBAC **საკუთრების** წესებით არის გამდიდრებული (ownership / relationship-based access): ოპერატორს მხოლოდ საკუთარი ოქმები უჩანს, მენეჯერს — თავისი გუნდის. ერთ მომხმარებელს რამდენიმე როლი შეიძლება ჰქონდეს, და **აქტიური როლი** სესიაშია.

2. მარტივად რომ ავხსნათ. თითო ადამიანს „ბეჯი“ აქვს: ოპერატორი, მენეჯერი, ამზომველი. ბეჯი განსაზღვრავს, რომელ ოთახებში შედის. ამას ემატება წესი: „შენი კაბინეტის საქაღალდეები“. ოპერატორი QC-ის ოთახში შედის, მაგრამ მხოლოდ საკუთარ საქაღალდეებს ხსნის.

3. ჩვენს პროექტში. ✅ 7 როლი (`domain.ROLES`), `user_roles` ცხრილი. როლების მატრიცა — Handoff §4.5. სამი დონე:

- **(ა)** გვერდი / მარშრუტი — `require(D.QC_MANAGER, D.ADMIN)` ;
- **(ბ)** ჩანაწერი — `scope.item_filter()` , `scope.can_view()` ;
- **(გ)** მოქმედება — `workflow.available_actions()` : მაგალითად, „არ ვეთანხმები“-ს დადასტურება მხოლოდ **პასუხისმგებელ** მენეჯერს შეუძლია, CC მენეჯერს — არა.

4. რატომ.

- QC შედეგები მგრძნობიარეა: ოპერატორის შეცდომები და შეფასებები მხოლოდ QC მენეჯერმა და ადმინმა უნდა ნახოს;
- სამუშაო პროცესის წესები ხილვადობით არის დაცული: ველი გაუგზავნელს და დადებითს ვერ ხედავს.

5. კულისებში. როლის ცვლა: `POST /role` → `switch_role()` ამოწმებს, რომ როლი მომხმარებელს ნამდვილად აქვს, და აუდიტში წერს.

6. რა შეიძლება აირიოს.

- როლის მინიჭების შეცდომა (ადამიანს ადმინის ბეჯი მიეცა). ✅ ამიტომ როლების ცვლილება აუდიტში იწერება (`user_updated` : before/after) და ✅ ადმინი საკუთარ ადმინის როლს ვერ მოიხსნის;
- 💡 ადმინი ორზე მეტი არ უნდა იყოს.

7. IT იკითხავს. „როლებს ვინ ანიჭებს? AD-ის ჯგუფებიდან შეიძლება?“

8. პასუხი. „დღეს ჩვენი ორი ადმინი ანიჭებს, პლატფორმის ადმინისტრირების გვერდიდან, ყოველი ცვლილება აუდიტში იწერება. AD/Entra ჯგუფებიდან ავტომატური როლები მომავალში შესაძლებელია, თუ სააგენტოს ასეთი ჯგუფები აქვს. ამ ეტაპზე ეს არ დაგვიგეგმავს.“

D.8 როგორ სრულდება უფლებები backend-ზე და რატომ არა მხოლოდ ინტერფეისში

1. ტექნიკური განმარტება. ინტერფეისში ღილაკის დამალვა უსაფრთხოება არ არის. კლიენტს ნებისმიერი HTTP მოთხოვნის გაგზავნა შეუძლია (DevTools, curl). ყოველი შემოწმება **სერვერზე** უნდა სრულდებოდეს, ყოველ მოთხოვნაზე. „არასწორი ობიექტის“ მოთხოვნის კლასიკური სისუსტეა **IDOR** (Insecure Direct Object Reference): `/items/4513` -ის გახსნა მხოლოდ ნომრის შეცვლით.

2. მარტივად რომ ავხსნათ. კარზე „არ შეხვიდე“-ს დაწერა კარს არ კეტავს. ცნობისმოყვარე ადამიანი მისამართის ზოლში ნომერს შეცვლის: 4512-ის ნაცვლად 4513. სერვერი ამ დროს უნდა შეამოწმოს: „4513 შენია? არა? მაშინ

ასეთი არ არსებობს.“

3. ჩვენს პროექტში. ✅

- `items._load()` ყოველ ბარათზე, მოქმედებაზე და ფაილზე `scope.can_view()`-ს იძახებს. თუ ჩანაწერი არ უჩანს — 404;
- `/attachments/{id}` ჯერ ოქმის ხილვადობას ამოწმებს და მერე ფაილს გასცემს;
- `/api/presence?ids=…` მხოლოდ იმ ოქმებზე აბრუნებს ინფორმაციას, რომლებიც ამ მომხმარებელს უჩანს;
- ✅ ტესტები: `test_operator_sees_only_own`, `test_field_sees_nothing_before_send_and_nothing_positive`, `test_field_does_not_see_qc_internal_history`.

4. რატომ — 404 და არა 403? 403 დაადასტურებდა, რომ ჩანაწერი არსებობს. ნომრების გადარჩევით ვინმე ოქმების სიას ააგებდა. ეს პრინციპი Pre-QC hosted-შიცაა.

5. კულისებში. სიები `scope.items_query(ctx)`-ით იწყება: ფილტრი SQL-შივე ემატება (`WHERE operator_id = …`). ზედმეტი მონაცემი ბაზიდან საერთოდ არ ამოდის.

6. რა შეიძლება აირიოს. ახალი endpoint-ი, რომელსაც scope-ის გამოძახება დაავიწყდა. 💡 Pre-QC hosted-ში ეს ინვარიანტი ტესტით არის დაცული (grep: მარშრუტი პირდაპირ `db.query`-ს ვერ აკეთებს). ჩვენთვისაც სასარგებლო იქნება.

7. IT იკითხავს. „როგორ ვიცით, რომ ოპერატორი სხვის ოქმს ვერ ნახავს?“

8. პასუხი. „ყოველი სია და ბარათი სერვერზე იფილტრება, აქტიური როლისა და საკუთრების მიხედვით. სხვისი ოქმი 404-ს აბრუნებს, და ეს ავტომატური ტესტებით მოწმდება. პენტესტი სასურველია; თუ ასეთი პროცედურა გაქვთ, სიამოვნებით გავივლით.“

D.9 CSRF

1. ტექნიკური განმარტება. CSRF (Cross-Site Request Forgery): მავნე საიტი მომხმარებლის ბრაუზერს აიძულებს, ჩვენს საიტზე მოთხოვნა გააგზავნოს. ბრაუზერი cookie-ს ავტომატურად ურთავს, ამიტომ მოთხოვნა „ავთენტიფიცირებულად“ გამოიყურება. დაცვის ხერხები:

- `SameSite` cookie;
- `Origin` / `Referer` თავსართის შემოწმება;
- CSRF token ფორმაში.

2. მარტივად რომ ავხსნათ. სხვა საიტს შეუძლია, შენი ბრაუზერის სახელით ჩვენს საიტზე „ღილაკი დააჭიროს“, რადგან ბრაუზერი შენს სამაჯურს ავტომატურად აჩვენებს. დაცვა: სერვერი ამოწმებს, საიდან მოვიდა მოთხოვნა, და თუ სხვა საიტიდან, უარყოფს.

3. ჩვენს პროექტში.

- ✅ `security.same_origin()`: ყველა მდგომარეობის შემცვლელ POST-ზე `Origin` (ან `Referer`) უნდა ემთხვეოდეს `Host`-ს. ტესტი: `test_cross_site_post_is_rejected`;
- ✅ `SameSite=Lax`;
- ⚠️ G8: თუ ორივე თავსართი აკლია, მოთხოვნა გადის. 💡 CSRF token-ის დამატება საწარმოომდე (უკვე წერია `ARCHITECTURE.md` §11-ში).

4–8. პასუხი IT-ს: „SameSite cookie და Origin-ის შემოწმება გვაქვს. საწარმოომდე CSRF token-საც დავამატებთ.“

D.10 საიდუმლოებების (secrets) მართვა

1. ტექნიკური განმარტება. საიდუმლოებებია:

- ბაზის პაროლი;
- OAuth client secret;

- SMTP პაროლი;
- სესიის ხელმოწერის გასაღები;
- API გასაღებები;
- WFS-ის მომხმარებელი და პაროლი.

ისინი არ უნდა იყოს კოდში, git-ში, ლოგში ან მეილში. ინახება გარემოს ცვლადებში, `600` უფლებიან ფაილში, ან საიდუმლოების საცავში (HashiCorp Vault, Windows Credential Manager, Docker/Kubernetes secrets). ასევე საჭიროა როტაცია: პერიოდული შეცვლა, და მყისიერი შეცვლა გაჟონვისას.

2. მარტივად რომ ავხსნათ. პაროლები სეიფში ინახება და არა წერილში ან ფურცელზე მონიტორთან. კოდი ყველამ შეიძლება ნახოს, ამიტომ პაროლი იქ არასოდეს იწერება.

3. ჩვენს პროექტში.

- ✅ ყველა საიდუმლოება `QCP_*` გარემოს ცვლადებიდან იკითხება (`app/config.py`). კოდში არცერთი არ წერია;
- ✅ `fetch_sheets.py` token-ებს არ ბეჭდავს და `data/` -ში არ ინახავს;
- ✅ Pre-QC Validator-ში WFS-ის პაროლი და AI გასაღები კოდიდან `.env` -ში გადავიდა. ⚠️ ადრე AI გასაღები `.exe` -ში იყო ჩაკერებული (`app/README.md` ამას ჯერ კიდევ აღწერს, `.env.example` კი ამბობს, რომ ეს აღარ ხდება). ეს შეუსაბამობაა დოკუმენტაციაში;
- ⚠️ ძველ პროექტებში token-ები `config.json` / `staff_config.json` -ში და ერთ handoff დოკუმენტში ღია ტექსტით წერია. 💡 მიგრაციის შემდეგ ეს token-ები უნდა შეიცვალოს.

4. რატომ. ერთი გაჟონილი secret შეიძლება მთელი სისტემის კომპრომეტაციას ნიშნავდეს.

5. კულისებში. სერვერზე: `EnvironmentFile=/etc/qc-platforma/env` , უფლებები `root:qcapp 640` (A.11).

6. რა შეიძლება აირიოს. secret-ი ჩატში, მეილში ან ეკრანის სურათზე.

7. IT იკითხავს. „საიდუმლოებებს სად ინახავთ?“

8. პასუხი. „მხოლოდ სერვერზე, გარემოს ცვლადებში ან თქვენს საიდუმლოების საცავში. კოდში არა. client secret-ი და ბაზის პაროლი პირდაპირ სერვერზე ჩაწერეთ, ჩვენ მათი ნახვაც არ გვჭირდება.“

D.11 აუდიტის ჟურნალი

1. ტექნიკური განმარტება. აუდიტის ჟურნალი უცვლელი (append-only) ჩანაწერებია, რომლებიც ამბობენ, ვინ, როდის, რა მოქმედება გააკეთა, რომელ ობიექტზე და რა შეიცვალა (before/after). ის ცალკეა ტექნიკური ლოგისგან და ანგარიშვალდებულებას და გამოძიებას ემსახურება.

2. მარტივად რომ ავხსნათ. „ვინ, რა, როდის“ — წიგნი, რომელშიც მხოლოდ იწერება და არაფერი იშლება.

3. ჩვენს პროექტში. ✅ ორი ფენა:

- `qc_events` — ყოველი ოქმის სრული ისტორია: მომხმარებელი, **აქტიური როლი**, მოქმედება, `from_state → to_state` , კომენტარი, მონაცემი (მაგ. ვერდიქტის „ადრე / შემდეგ“), დრო. ✅ კოდი მას არასოდეს ცვლის;
- `audit_log` — ადმინისტრაციული მოქმედებები:
  - შესვლა (`login`) და უარყოფილი შესვლა (`login_denied` + ელფოსტა);
  - როლის შეცვლა;
  - მომხმარებლის შექმნა და ცვლილება (before/after);
  - კოდის ცვლილება;
  - პარამეტრები;
  - განაწილება. IP-ითაც.

4. რატომ.

- დავის დროს, მაგალითად „მე ეს არ შემიცვლია“, ისტორია მტკიცებულებაა;

- ⚠️ ძველი ცხრილი სტატუსს ზედ წერდა, და ისტორია ვერ აღდგება.

5. კულისებში. ცვლილება და მისი ისტორიის ჩანაწერი ერთ ტრანზაქციაშია (B.7).

6. რა შეიძლება აირიოს.

- ბაზაში ხელით ცვლილება ისტორიას გვერდს უვლის. 💡 ამიტომ B.14-ის `REVOKE` და პროცედურა;
- proxy-ს უკან IP proxy-ისაა (G4).

7–8. პასუხი IT-ს: „ყოველი სამუშაო მოქმედება და ადმინის ყოველი ცვლილება ბაზაში იწერება, ვინ, რა როლით და როდის. ისტორია მხოლოდ ემატება. საწარმოოში ბაზის დონეზეც ავკრძალავთ მის შეცვლას.“

D.12 მინიმალური უფლება — მოკლე შეჯამება

სად	მინიმალური უფლება
წყარო სისტემები	მხოლოდ `SELECT`, მხოლოდ view-ზე / მხოლოდ საქაღალდის წაკითხვა; ცალკე ანგარიში თითო წყაროზე; მხოლოდ ჩვენი სერვერის IP-დან
ჩვენი ბაზა	აპლიკაციის როლი სქემას ვერ ცვლის; ისტორიას ვერ შლის; superuser არა
Google	მხოლოდ `openid email profile`
აპლიკაციის მომხმარებლები	როლი + საკუთრება; ადმინი მხოლოდ 2
OS	აპლიკაცია ცალკე, არაპრივილეგირებული მომხმარებლით (`qcapp`), არა root/Administrator
დეველოპერები	საწარმოო ბაზაზე ჩაწერა არა; ცვლილება მხოლოდ deployment-ით

მარტივად რომ ავხსნათ: თითოეულს ზუსტად იმდენი უფლება, რამდენიც სამუშაოს სჭირდება. თუ რომელიმე ნაწილი გატყდება ან გაჟონავს, ზიანი პატარა იქნება.

D.13 თანამშრომელი გუნდს იცვლის ან ტოვებს სააგენტოს

1. ტექნიკური განმარტება. იდენტობის სასიცოცხლო ციკლი (joiner → mover → leaver):

- **ახალი** — ანგარიშის შექმნა, როლი, გუნდი;
- **გადაყვანა** — როლის ან გუნდის ცვლილება, ძველი წვდომის მოხსნა;
- **წასვლა** — წვდომის მყისიერი გაუქმება, ისტორიის შენარჩუნება.

2. მარტივად რომ ავხსნათ.

- ახალს ბეჯს ვაძლევთ;
- გადაყვანილს ბეჯს ვუცვლით;
- წასულს ბეჯს ვართმევთ, მაგრამ მისი სახელი ძველ საქაღალდეებში რჩება, რადგან ის ნამდვილად მუშაობდა.

3. ჩვენს პროექტში — რეალურად როგორ მუშაობს ✅ (`routes/admin.user_save`, `services/seed.manager_map`, `scope.py`):

შემთხვევა	რა ხდება სისტემაში
ამზომველი სხვა გუნდში გადავიდა	ადმინი უცვლის გუნდს: ძველი წევრობა იხურება (`valid_to = დღეს`), ახალი იხსნება. **ახალი** ოქმები განაწილებისას ახალ მენეჯერს ეკუთვნის (`manager_map` მიმდინარე წევრობას კითხულობს). **უკვე განაწილებული** ოქმები ძველ მენეჯერთან რჩება (`item_people`, `field_manager_id` განაწილების მომენტში ფიქსირდება). საჭიროებისას QC მენეჯერს ოქმზე მენეჯერის ხელით შეცვლა შეუძლია (`assign_manager`). ამზომველი საკუთარ ოქმებს ორივე შემთხვევაში ხედავს

შემთხვევა	რა ხდება სისტემაში
ამზომველი წავიდა	ადმინი რთავს `active = false`-ს → სესიები მაშინვე უქმდება, Google-ით შესვლა უარყოფილია. მისი ოქმები და ისტორია რჩება. ⚠️ `active = false` გუნდის წევრობას **თავისით არ ხურავს**, და `manager_map` წევრის აქტიურობას არ ამოწმებს (მხოლოდ მენეჯერისას). ამიტომ მისი ახალი ოქმები ისევ ძველ მენეჯერს მიეწერება, სანამ წევრობა არ დაიხურება (Field Group-იდან ამოღებით ან ადმინის მიერ). 💡 გათიშვისას წევრობის ავტომატური დახურვა გასაკეთებელია. როცა ადამიანი მიმდინარე გუნდში აღარ არის, მისი ოქმი მეორე ამზომველის მენეჯერს ერგება, ან, თუ ასეთი არ არის, „დაუჯგუფებელში“ ხვდება
მენეჯერი წავიდა	`active = false`. ✅ `manager_map` არააქტიური მენეჯერის გუნდს აღარ იყენებს, ამიტომ მისი ამზომველების ახალი ოქმები „დაუჯგუფებელში“ მოხვდება, სანამ გუნდს ახალი მენეჯერი არ ეყოლება. მისი **ღია** ოქმების „არ ვეთანხმები“-ს დადასტურება აღარავის შეუძლია, ადმინის გარდა. 💡 QC მენეჯერმა ოქმებს ახალი მენეჯერი უნდა მიუწეროს (`assign_manager`). ❓ მასობრივი გადაწერის ინსტრუმენტი ჯერ არ არის
QC ოპერატორი წავიდა	`active = false`. მისი გასავლელი ოქმები QC მენეჯერმა სხვას უნდა გადაუნაწილოს (`reassign`). 💡 მასობრივი ვარიანტი სასურველია
როლის შეცვლა	ადმინი ცვლის `user_roles`-ს, აუდიტში იწერება before/after. თუ ადამიანს აქტიური სესიის როლი მოეხსნა, შემდეგ მოთხოვნაზე ნაგულისხმევ როლზე გადადის (`current()` ამოწმებს, რომ `active_role` ისევ აქვს)

4. რატომ. წასულ თანამშრომელს წვდომა არ უნდა დარჩეს. ✅ Pre-QC hosted-იც ამავე პრინციპით მუშაობს: „საფუძვლის გაქრობაზე წვდომაც ქრება“.

5. კულისებში. `read_session()` ყოველ მოთხოვნაზე ამოწმებს `u.active`-ს, ამიტომ გათიშვა მყისიერია. სესიის ვადის ამოწურვის ლოდინი საჭირო არ არის.

6. რა შეიძლება აირიოს.

- HR-მა წასვლა არ გვაცნობა. 💡 საჭიროა პროცესი: HR/IT → ჩვენი ადმინი, ან სინქრონი HR/AD-თან;
- Google ანგარიში პირადია, და ადამიანი მას წასვლის შემდეგაც ფლობს. ამიტომ წვდომას **ჩვენი** `active` ნიშანი განსაზღვრავს.

7. IT იკითხავს. „თანამშრომლის წასვლისას ვინ და როგორ უხსნის წვდომას?“

8. პასუხი. „ჩვენი ადმინი, ერთი ღილაკით (`active = false`). ის მყისიერად მოქმედებს და აუდიტში იწერება. პირადი Gmail-ის შემთხვევაში ეს ერთადერთი გზაა, ამიტომ HR-ისგან შეტყობინების პროცესი გვჭირდება. თუ Workspace იქნება, ანგარიშის გათიშვა Google-ის მხარესაც იმუშავებს.“

D.14 პირადი მონაცემი

1. ტექნიკური განმარტება. პერსონალური მონაცემის დამუშავების პრინციპებია:

- კანონიერი საფუძველი;
- **მინიმიზაცია**;
- მიზნობრიობა;
- შენახვის ვადის შეზღუდვა;
- უსაფრთხოება;
- წვდომის კონტროლი;
- აუდიტი.

ტექნიკური ზომებია ნიღბვა (masking), ფსევდონიმიზაცია, დაშიფვრა (გადაცემისას — TLS; შენახვისას — დისკის ან ბაზის დონეზე) და ლოგების გასუფთავება.

2. მარტივად რომ ავხსნათ. მხოლოდ ის შევაგროვოთ, რაც სამუშაოს სჭირდება; მხოლოდ იმას ვაჩვენოთ, ვისაც სჭირდება; და დავიცვათ, რომ არ გაჟონოს.

3. ჩვენს პროექტში. ✅

- თანამშრომლის პირადი ნომერი ინახება (გადაწყვეტილებით, ვინაობის შესადარებლად) და ინტერფეისში ნიღბით ჩანს. ის `__repr__`-ში არ იბეჭდება;
- სპორადულის ფაილიდან განმცხადებლების პირადი ნომრები, ტელეფონები და მისამართები არ იკითხება, ფაილი იშლება;
- `fetch_sheets.py` „პირადი ნომრების“ ჩანართს არ კითხულობს;
- `data/` git-ში არ შედის;
- ❓ QC კომენტარებში შეიძლება მოქალაქეების სახელები იყოს. Pre-QC hosted-ის შემოწმებით, `qc_comment` 2 311 სიმბოლომდე აღწევს და სახელებს შეიცავს.

4. რატომ. პერსონალურ მონაცემთა დაცვის კანონი და სააგენტოს რეპუტაცია.

5. კულისებში. ✅ Pre-QC hosted-ში პირადი ნომრები ჩაწერისასვე იცვლება `[ID]`-ით (9+ ციფრი). 💡 ჩვენი ლოგებისთვის იგივე წესი გამოგვადგება.

6. რა შეიძლება აირიოს.

- ბექაპის ფაილი ან Excel ექსპორტი პირადი მონაცემით ვრცელდება. 💡 ექსპორტი როლით შეზღუდულია, ბექაპი დაშიფრულია;
- ⚠️ `data\`-ის კოპირება „ტესტისთვის“.

7. IT იკითხავს. „რა პირად მონაცემს ამუშავებთ და რის საფუძველზე?“

8. პასუხი. „თანამშრომლების: სახელი, ელფოსტა, გუნდი, პირადი ნომერი ვინაობის შესადარებლად. მოქალაქეების მონაცემი სისტემაში მხოლოდ QC კომენტარის ტექსტში შეიძლება მოხვდეს. სამართლებრივი საფუძველი და შენახვის ვადა თქვენს და იურიდიულ სამსახურთან უნდა შევათანხმოთ. ამისთვის მზად ვართ, მონაცემის აღწერა მოგაწოდოთ.“

D.15 ფაილების ატვირთვა

1. ტექნიკური განმარტება. ფაილის ატვირთვის რისკებია:

- მავნე ფაილი;
- ზედმეტად დიდი ფაილი (DoS);
- ბილიკის მანიპულაცია (path traversal);
- ფაილის შიგთავსის ტიპის გაყალბება;
- სხვისი ფაილის ნახვა.

დაცვის ხერხები:

- ზომისა და ტიპის შემოწმება;
- შემთხვევითი სახელი;
- ცალკე საქაღალდე, შესრულების უფლების გარეშე;
- გაცემისას წვდომის შემოწმება;
- `X-Content-Type-Options: nosniff`;
- არჩევით — ანტივირუსული შემოწმება.

2. მარტივად რომ ავხსნათ. ფოტოს ატვირთვა „ამანათის მიღებაა“. ვამოწმებთ ზომას და იმას, ფოტოა თუ არა. ვინახავთ ჩვენი ნომრით და არა გამგზავნის დაწერილი სახელით. ვაძლევთ მხოლოდ მას, ვისაც ამ ოქმის ნახვა შეუძლია.

3. ჩვენს პროექტში. ✅ `workflow.save_files()`:

- ტვირთავს მხოლოდ კონტროლი (QC როლები);
- JPEG / PNG / WEBP / GIF / PDF;
- ≤ 10 MB;

- სახელი `uuid4().hex`;
- საქაღალდე `data/uploads/<item_id>/`;
- გაცემისას — ოქმის ხილვადობის შემოწმება;
- `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY` (`app/main.py`).

⚠️ ტიპი ბრაუზერის მიერ გამოგზავნილი `content_type`-ით მოწმდება და არა ფაილის რეალური შიგთავსით. 💡 შიგთავსის შემოწმება (magic bytes) და ანტივირუსი გასათვალისწინებელია.

4–8. პასუხი IT-ს: „ატვირთვა მხოლოდ კონტროლის თანამშრომლებს შეუძლიათ, მხოლოდ სურათი ან PDF, 10 MB-მდე, შემთხვევითი სახელით. გაცემისას წვდომა მოწმდება. თუ ანტივირუსული შემოწმების სერვისი გაქვთ, ჩავრთავთ.“

D.16 უსაფრთხოების თავსართები

✅ `app/main.py` ყველა პასუხს ამატებს:

- `X-Content-Type-Options: nosniff` — ბრაუზერი ფაილის ტიპს არ „გამოიცნობს“;
- `X-Frame-Options: DENY` — ჩვენს საიტს სხვა საიტი ჩარჩოში (iframe) ვერ ჩასვამს. ეს clickjacking-ისგან იცავს;
- `Referrer-Policy: same-origin` — სხვა საიტზე გადასვლისას ჩვენი მისამართი არ გადაეცემა;
- `Cache-Control: no-store` — `/api/*` და `/items/*`.

💡 დასამატებელია:

- `Strict-Transport-Security` (HSTS), proxy-ზე;
- `Content-Security-Policy` მთელ საიტზე (მეილის ნახვის გვერდს CSP უკვე აქვს: `/admin/emails/{id}`).

მარტივად რომ ავხსნათ: ესაა ბრაუზერისთვის მიცემული „ინსტრუქციები“: სხვა საიტში არ ჩაგვსვა, ფაილის ტიპი არ გამოიცნო, ჩვენი მისამართი სხვას არ უთხრა.

E. აპლიკაციის არქიტექტურა

E.1 Frontend, backend, ბაზა — ვინ რას აკეთებს

1. ტექნიკური განმარტება.

- **Frontend** — ის, რაც ბრაუზერში სრულდება: HTML, CSS, JavaScript.
- **Backend** — სერვერის კოდი: ბიზნეს-ლოგიკა, ავტორიზაცია, ბაზასთან მუშაობა.
- **ბაზა** — მდგომარეობის შენახვა.

ჩვენი არქიტექტურა **სერვერზე აწყობილი HTML**-ია (server-side rendering, SSR), „monolith“ სტილით: ერთი აპლიკაცია ემსახურება გვერდებსაც და API-საც. ცალკე SPA (React/Vue) და ცალკე API სერვერი არ გვაქვს.

2. მარტივად რომ ავხსნათ. ბრაუზერი „ეკრანია“, რომელიც მზა გვერდს აჩვენებს. ყველა გადაწყვეტილება — ვინ ხარ, რა გეკუთვნის, რა შეგიძლია — სერვერზე მიიღება. ბრაუზერში ბევრი „ჭკვიანი“ კოდი არ არის, ამიტომ არც მისი გატეხვაა ადვილი.

3. ჩვენს პროექტში. ✅

```
ბრაუზერი
  HTML (Jinja2-ით სერვერზე აწყობილი) + CSS (app/static/css/app.css)
  HTMX (ფორმები / ნაწილობრივი განახლება) + მცირე JS:
     app.js, presence.js („ახლა ხსნის"), stats.js (ECharts), qc-rules.js (კოდების მინიშნება)
  ECharts (გრაფიკები), Leaflet + zones.geojson (რუკა) — ლოკალური ასლები
        │ HTTP(S): GET გვერდები, POST ფორმები, JSON (/api/*)
        ▼
FastAPI (backend)
  routes/  →  security (ვინ ხარ)  →  scope (რა გიჩანს)  →  workflow / services (რა ხდება)
        │ SQLAlchemy
        ▼
ბაზა (SQLite → PostgreSQL)  +  data/uploads (ფაილები)
```

4. რატომ ეს მიდგომა.

- **ერთი ენა და ერთი პროექტი.** Python-ის გარდა build-ის ხელსაწყოები (Node, webpack) არ გვჭირდება;
- **უსაფრთხოება.** ლოგიკა სერვერზეა, ამიტომ ავტორიზაციის „გამოტოვება" ბრაუზერიდან შეუძლებელია;
- **სისწრაფე სუსტ ტელეფონზე.** მზა HTML მოდის, და ბრაუზერს დიდი JavaScript-ის შესრულება არ სჭირდება;
- **ინტერნეტის გარეშე CDN.** ✅ ყველა ბიბლიოთეკა `app/static/vendor/` -შია.

5. კულისებში. გვერდის ჩატვირთვა:

1. `GET /qc?tab=todo` ;
2. FastAPI ამოწმებს სესიას და როლს;
3. `scope` -ით კითხულობს ოქმებს;
4. Jinja2 აწყობს `templates/qc/list.html` -ს;
5. ბრაუზერი ხატავს.

ფორმის გაგზავნისას: `POST` → `workflow.apply` → `303` → ახალი GET.

6. რა შეიძლება აირიოს. ძალიან „ცოცხალი" ინტერფეისისთვის (ჩატი, რეალურ დროში განახლება) SSR-ს მეტი ხრიკი სჭირდება. ✅ „ახლა ხსნის" ნიშანი ამიტომ polling-ით მუშაობს (20 წმ) და არა WebSocket-ით.

7. IT იკითხავს. „SPA არის? ცალკე frontend-ის განთავსება გჭირდებათ?"

8. პასუხი. „არა. ერთი აპლიკაციაა, რომელიც გვერდებსაც და API-საც ემსახურება. ერთი პროცესი, ერთი პორტი, ერთი დომენი. ცალკე frontend სერვერი საჭირო არ არის."

E.2 ტექნოლოგიები: რატომ ეს და რა შეზღუდვები აქვს

ტექნოლოგია	რატომ ავირჩიეთ	უპირატესობა	შეზღუდვა
Python	Distributor, Dashboard და Pre-QC Python-ზეა; ძველი ალგორითმები და წესები გადმოვიტანეთ	ერთი ენა, დიდი ეკოსისტემა (openpyxl, GDAL, requests)	ერთ პროცესში CPU-ს ინტენსიური სამუშაო შეზღუდულია (GIL); რამდენიმე worker საჭიროა
FastAPI	თანამედროვე, სწრაფი, ავტომატური OpenAPI; Pre-QC-შიც იგივეა	ტიპები, dependency injection (`Depends(require(...))`), ტესტირება `TestClient` -ით	✅ ჩვენ ძირითადად სინქრონულ ფუნქციებს ვიყენებთ — ეს მისაღებია (thread pool)
Jinja2 + HTMX	ჩარჩოს გარეშე, build-ის გარეშე	მარტივი, სწრაფი, ცოტა JS	რთული ინტერაქტიული ეკრანები (drag-and-drop და მსგავსი) უფრო ძნელად კეთდება
SQLAlchemy 2	ORM, რომელიც SQLite-საც, PostgreSQL-საც და SQL Server-საც უჭერს მხარს	ბაზის შეცვლა კოდის უმეტესობის შეცვლის გარეშე; პარამეტრიზებული შეკითხვები	ORM ზოგჯერ არაოპტიმალურ SQL-ს აგენერირებს — `EXPLAIN` საჭიროა

ტექნოლოგია	რატომ ავირჩიეთ	უპირატესობა	შეზღუდვა
SQLite (პროტოტიპი)	ინსტალაციის გარეშე, ერთი ფაილი	დემოსთვის იდეალური	ერთი მწერალი; რამდენიმე სერვერი ვერ იზიარებს
ECharts / Leaflet	ძველი პანელიდან და Pre-QC-იდან; ცნობილი ინსტრუმენტები	მდიდარი გრაფიკები, რუკა	ECharts ≈ 1 MB-ია (ერთხელ იტვირთება, ბრაუზერი ქეშავს)
openpyxl	Excel ექსპორტი და იმპორტი	—	დიდი ფაილები ნელია; ✅ `read_only=True` გამოიყენება
truststore	კორპორატიული სერტიფიკატი	Windows-ზე „უბრალოდ მუშაობს“	Linux-ზე სისტემის საცავში CA უნდა იყოს

მარტივად რომ ავხსნათ: ყველა ხელსაწყო გავრცელებული, უფასო და ღია კოდისაა. ისეთი ავირჩიეთ, რაც ჩვენს სხვა პროექტებშიც გამოგვიყენებია, რომ მხარდაჭერა მარტივი იყოს.

IT იკითხავს: „ვინ დაუჭერს მხარს, თუ წახვალთ?“ **პასუხი:** „სტეკი სტანდარტულია: Python, FastAPI, PostgreSQL. ნებისმიერ Python დეველოპერს ესმის. კოდს კომენტარები და ტესტები აქვს, დოკუმენტაცია კი ქართულადაა. მაინც, ეს რეალური რისკია, და ამიტომ ვამზადებთ handoff დოკუმენტებს და ვთავაზობთ, რომ თქვენი ერთი თანამშრომელი ჩაერთოს.“

E.3 სამუშაო პროცესი კოდში — state machine

1. ტექნიკური განმარტება. Finite state machine (FSM) შედგება:

- სასრული მდგომარეობების სიმრავლისგან;
- დაშვებული გადასვლებისგან (transition);
- გადასვლის პირობებისგან (guard: როლი, საკუთრება, სავალდებულო ველი);
- გვერდითი ეფექტებისგან (ისტორია, მეილი).

ბიზნეს-წესები ერთ ადგილას, დეკლარაციულად ან ერთ მოდულშია თავმოყრილი, და არა გაბნეული გვერდებზე.

2. მარტივად რომ ავხსნათ. ეს არის ოქმის „მარშრუტის რუკა“. თითო გაჩერებაზე წერია, ვის შეუძლია შემდეგ გაჩერებაზე გადაყვანა და რა უნდა შეავსოს. სხვა გზით ოქმი ვერ წავა.

3. ჩვენს პროექტში. ✅ `app/workflow.py`:

- `available_actions(db, item, user, role)` — რა ღილაკები ჩანს;
- `apply(db, item, user, role, action, form, files)` — მოქმედების შესრულება. ჯერ ამოწმებს, რომ მოქმედება დაშვებულია, მერე ვერსიას, ფოტოს უფლებას და სავალდებულო ველებს; ბოლოს წერს `QcEvent`-ს;
- მდგომარეობები და წარწერები `app/domain.py`-შია: „ერთადერთი ადგილი, სადაც ეს მნიშვნელობები ჩამოთვლილია“.

4. რატომ.

- წესი ერთ ადგილას იცვლება;
- ტესტები ყველა გადასვლას ამოწმებს (`test_workflow.py`, 18 ტესტი);
- ინტერფეისი ღილაკებს იმავე ფუნქციიდან იღებს, ამიტომ ღილაკი და უფლება ერთმანეთს ვერ დაშორდება.

5. კულისებში. მაგალითი: `qcm_field_right` (QC მენეჯერი — „ველი მართალია“):

1. მოწმდება, რომ მონიშნულია „დადებითი“;
2. ვერდიქტი ხდება `positive`;
3. `qc_error = true`;
4. იხურება `close_reason = qc_error`-ით;
5. ისტორიაში იწერება „ადრე: negative“.

6. რა შეიძლება აირიოს. ახალი ბიზნეს-წესი, რომელიც მხოლოდ შაბლონში დაემატა, `workflow`-ში კი არა. ასეთი რამ არ უნდა მოხდეს: წესი ყოველთვის `workflow`-ში ემატება და მისთვის ტესტი იწერება.

7. IT იკითხავს. „ბიზნეს-წესები სად არის აღწერილი?“

8. პასუხი. „კოდში ერთ მოდულშია (`workflow.py`), ტექსტურად — `ARCHITECTURE.md`-ის §5-ში (გადასვლების ცხრილი), და ტესტებით არის დაცული.“

E.4 დამოკიდებულებები (dependencies) და მიწოდების ჯაჭვი

1. ტექნიკური განმარტება. აპლიკაცია მესამე მხარის ბიბლიოთეკებს იყენებს. მათი ვერსიები უნდა იყოს ფიქსირებული (lock), რეგულარულად განახლებული უსაფრთხოების პაჩებით, და შემოწმებული ცნობილ სისუსტეებზე (`pip-audit`, Dependabot და სხვ.). frontend-ის ბიბლიოთეკები შეიძლება იყოს CDN-იდან ან ლოკალური (vendored).

2. მარტივად რომ ავხსნათ. ჩვენი საიტი სხვისი „აგურებითაც“ არის აშენებული. ვიცით, რომელი აგური გამოვიყენეთ და რომელი ვერსიის. როცა მწარმოებელი იტყვის „ამ აგურს ბზარი აქვს“, ახლით ვცვლით.

3. ჩვენს პროექტში.

- ✅ `requirements.txt`: `fastapi>=0.115`, `uvicorn>=0.30`, `sqlalchemy>=2.0.36`, `jinja2>=3.1`, `python-multipart>=0.0.9`, `itsdangerous>=2.2`, `openpyxl>=3.1`, `requests>=2.32`, `truststore>=0.10`, `pytest>=8`, `httpx>=0.27`;
- ⚠️ ვერსიები **მინიმალურია (`>=`)** და არა ზუსტი. სხვა სერვერზე სხვა ვერსია დადგება. 💡 lock ფაილი (`uv pip compile` / `pip freeze`) საწარმოოსთვის;
- ⚠️ ტესტის გაშვებისას გაფრთხილება: `starlette.testclient` + `httpx` მოძველებულად არის მონიშნული. ეს მხოლოდ ტესტებს ეხება;
- ✅ frontend-ის ბიბლიოთეკები ლოკალურია: HTMX 2.0.4, ECharts 5.5.1, Leaflet, Noto Sans Georgian შრიფტი (OFL ლიცენზია).

4. რატომ. IT-ის უსაფრთხოების გუნდს სიისა და ლიცენზიების ნახვა სურს.

5–8. პასუხი IT-ს: „ყველა ბიბლიოთეკა ღია კოდისაა, თავისუფალი ლიცენზიებით (MIT, BSD, Apache, OFL შრიფტზე). სიას და ზუსტ ვერსიებს lock ფაილით მოგაწვდით, და `pip-audit`-ის ანგარიშსაც.“

E.5 როგორ ესაუბრება frontend backend-ს და backend გარე სისტემებს

მიმართულება	როგორ ✅	მაგალითი
ბრაუზერი → backend	HTML ფორმები (POST), HTMX მოთხოვნები, `fetch()` JSON-ისთვის	`POST /items/4512/action`; `POST /api/presence/4512` ყოველ ~20 წმ-ში
backend → ბაზა	SQLAlchemy ORM, ერთი სესია ერთ მოთხოვნაზე	`scope.items_query(ctx)`
backend → Distributor-ის SQLite	`sqlite3`, `mode=ro`, პარამეტრიზებული SQL	`source.query_candidates()`
სკრიპტი → Google Apps Script	`requests.get`, token-ით, გვერდებად	`fetch_sheets.py`
backend → Google OAuth	ბრაუზერის გადამისამართება + `requests.post` token endpoint-ზე	`google_auth.py`
backend → SMTP	`smtplib`, STARTTLS	`notify._send()`
🅿 Pre-QC → backend	API გასაღებით, JSON	კონტრაქტი შესათანხმებელია (Handoff C4)

მარტივად რომ ავხსნათ:

- ბრაუზერი სერვერს ფორმებით და პატარა „კითხვებით“ ელაპარაკება;
- სერვერი ბაზას „თარჯიმნით“ (ORM);
- ძველ წყაროებს მხოლოდ კითხვით;
- Google-ს მხოლოდ შესვლისას;
- ფოსტას მხოლოდ შეტყობინებისას.

ყოველ ამ „ხაზს“ თავისი წესი აქვს, და არცერთი ხაზი არ წერს სააგენტოს სისტემებში.

E.6 დღევანდელი და საწარმოო არქიტექტურა — რა იცვლება და რატომ

კომპონენტი	დღეს ✅	საწარმოოში 💡	ცვლილება გამართლებულია?
ბაზა	SQLite	PostgreSQL	**კი:** ერთდროული ჩაწერა, ბექაპი, რამდენიმე worker
სქემის მართვა	`create_all`	Alembic	**კი:** სქემის უსაფრთხო ცვლილება
პროცესი	1 uvicorn	N worker + სერვისი	**კი:** ხელმისაწვდომობა
გადახედვის შენახვა	მეხსიერება	ბაზის ცხრილი	**კი:** worker-ებს შორის
ფონური ამოცანები	ხელით, სკრიპტებით	worker + განრიგი	**კი:** ღამის იმპორტი, მეილი
მეილი	სინქრონული / preview	outbox + worker + SMTP	**კი**
შესვლა	დემო	მხოლოდ Google	**კი:** დემო ფიზიკურად გათიშული
TLS	არა	proxy-ზე	**კი**
წყაროები	ლოკალური ფაილები / Apps Script	view / ექსპორტი / API	**კი**
frontend-ის ჩარჩო	Jinja2 + HTMX	იგივე	**არა** — ცვლილება საჭირო არ არის
ენა / ჩარჩო	Python / FastAPI	იგივე	**არა**
მონოლითი → მიკროსერვისები	—	**არ ვთავაზობთ**	**არა** — ამ მასშტაბზე ზედმეტი სირთულეა

მარტივად რომ ავხსნათ: სახლის „საძირკველს“ (ენა, ჩარჩო, ეკრანები) არ ვცვლით. ვცვლით იმას, რაც ერთი ადამიანისთვის იყო აწყობილი და ბევრისთვის არ გამოდგება: ბაზა, ფონური სამუშაო, შესვლა, დაცული მისამართი.

E.7 რა არ ვიცით არქიტექტურაზე (და როგორ გავარკვევთ)

კითხვა	როგორ გავარკვევთ
რამდენ ერთდროულ მომხმარებელს გაუძლებს?	დატვირთვის ტესტი PostgreSQL-ზე, პილოტამდე
სტატისტიკა რამდენ ხანში ჩაიტვირთება PostgreSQL-ზე?	`EXPLAIN ANALYZE` + გაზომვა
დიდი განაწილების ტრანზაქცია რამდენ ხანს ბლოკავს?	გაზომვა 3 000+ ოქმიან განაწილებაზე
GDB-ის წაკითხვა სერვერზე (GDAL)?	IT-ის გადაწყვეტილება ფორმატზე (CSV თუ GDB)
რა ზომისაა worker-ის მეხსიერება?	გაზომვა პილოტზე

მარტივად რომ ავხსნათ: რამდენიმე კითხვაზე პასუხი ჯერ არ ვიცით: რამდენ ადამიანს გაუძლებს, რამდენად სწრაფი იქნება, რამდენ მეხსიერებას დაიკავებს. ამას ვამბობთ გულწრფელად, და იმასაც, **როგორ** გავიგებთ: ყოველ კითხვას თავისი გაზომვა აქვს, და ეს გაზომვები პილოტამდე ტარდება.

F. მონაცემის მიგრაცია და არსებული სისტემები

F.1 რას ნიშნავს მიგრაცია ჩვენს შემთხვევაში

1. ტექნიკური განმარტება. მონაცემის მიგრაცია არის ETL (Extract → Transform → Load):

- **ამოღება** წყაროდან;
- **გარდაქმნა** — გასუფთავება, ნორმალიზაცია, ახალ მოდელზე გადაყვანა, დედუპლიკაცია;
- **ჩატვირთვა** ახალ სისტემაში.

ამას ემატება **შეჯერება** (reconciliation), რომელიც ამტკიცებს, რომ არაფერი დაიკარგა, და გადასვლის გეგმა (cutover). ის შეიძლება იყოს „დიდი აფეთქება“ (big bang) ან პარალელური მუშაობა.

2. მარტივად რომ ავხსნათ. მიგრაცია „სახლიდან სახლში გადასვლაა“: ნივთები ჩავალაგეთ (ამოღება), დავახარისხეთ და ზედმეტი გადავაგდეთ (გარდაქმნა), ახალ სახლში დავალაგეთ (ჩატვირთვა), და ჩამონათვალით შევამოწმეთ, ყველაფერი მოვიდა თუ არა (შეჯერება).

3. ჩვენს პროექტში.

- ✅ პროტოტიპში ერთჯერადი გადმოტანა უკვე გაკეთდა: 44 538 მწკრივი → 44 159 შემთხვევა (Handoff §4.9);
- ⚠️ ეს **დემონსტრაციაა და არა დამტკიცებული საწარმოო მიგრაცია**. მიგრაციის დაწყებამდე ხელმძღვანელობის თანხმობა სჭირდება: რა პერიოდი, რა ჩანართები, ვინ ამოწმებს. ის, რომ ტექნიკურად შესაძლებელია, ნებართვას არ ნიშნავს.

4. რატომ.

- ახალი სისტემა ძველი ციფრების გარეშე ნდობას ვერ მოიპოვებს;
- დაუხურავ შემთხვევებზე სამუშაო უნდა გაგრძელდეს: 9 657 (გასავლელი 1 456, ველზე 3 774, გადასამოწმებელი 3 030, მენეჯერთან 1 107, QC მენეჯერთან 290).

5–8. იხ. F.2–F.9.

F.2 Mapping — სად რა გადადის

1. ტექნიკური განმარტება. Mapping-ის სპეციფიკაცია წყაროს ყოველ ველს სამიზნე ველთან აკავშირებს, გარდაქმნის წესით, და ამბობს, რა ხდება ცარიელ, არასწორ ან უცნობ მნიშვნელობაზე.

2. მარტივად რომ ავხსნათ. ეს არის „თარგმანის ლექსიკონი“: Excel-ის რომელი სვეტი რომელ ველში ჩაიწეროს და როგორ.

3. ჩვენს პროექტში. ✅ სვეტების რუკა — Handoff §3.1. მნიშვნელოვანი გარდაქმნები (`legacy_import.py`):

წყარო	გარდაქმნა	სამიზნე
E „შემოწმების სტატუსი“ (ტექსტი)	`VERDICT_FROM_SHEET` : დადებითი → `positive` …	`verdict`
F, G (სახელი ± პირადი ნომერი)	`clean_name` + `name_key` + ალიასები (591) → `users.id`	`item_people`
H (კოდები ტექსტად)	`parse_codes`	`qc_item_codes` (`source = legacy`)
J (ველის სტატუსი)	`RESPONSE_FROM_SHEET` : შესწორდა / გასწორდა → `corrected` …	`field_response`
M (TRUE/FALSE)	→ `corrected_confirmed` , დახურვის მიზეზი	`state` , `close_reason`
Q, S (სინქრონის დროები) + O	„ვისთანაა ოქმი“ წესი	`state`

წყარო	გარდაქმნა	სამიზნე
V (TYPE)	`LEGACY_TYPE_TO_MARK`	`qc_reviews`
W, დანარჩენი	ნედლად	`legacy_extra` (JSON)

4. რატომ. ძველ ცხრილში ერთი და იგივე რამ რამდენიმე ფორმით იწერებოდა („შესწორდა“ / „გასწორდა“, „გავითვალისწინე“ / „გავითვალისწინებთ“).

5. კულისებში. ✅ სვეტები **სათაურით** იკითხება და არა პოზიციით. „სინქრონის დრო“ ორჯერ გვხვდება: პირველი Q-ა, მეორე S.

6. რა შეიძლება აირიოს.

- ახალი ფორმის ტექსტი, რომელიც ლექსიკონში არ არის, ჩუმად ცარიელი დარჩება. 💡 საწარმოო მიგრაციაში უცნობი მნიშვნელობა ანგარიშში უნდა ჩანდეს;
- K სვეტში ზოგჯერ ახალი ოქმის ნომერი წერია, და ეს ცალკე უნდა დამუშავდეს (`legacy_import.py` -ის კომენტარი).

7–8. IT იკითხავს: „როგორ ადარებთ ძველ და ახალ ველებს?“ **პასუხი:** „წერილობითი რუკა გვაქვს, სვეტიდან ველამდე, გარდაქმნის წესით. უცნობი მნიშვნელობები ანგარიშში ჩანს და ჩუმად არ იკარგება.“

F.3 დუბლიკატები და კონფლიქტური ჩანაწერები

1. ტექნიკური განმარტება. დუბლიკატი იდენტიფიცირდება ბიზნეს-გასაღებით (ოქმი + საკადასტრო კოდი). **კონფლიქტი** არის დუბლიკატი, რომლის ასლები ერთმანეთს ეწინააღმდეგება (სხვადასხვა ვერდიქტი, ოპერატორი). საჭიროა გადაწყვეტის წესი (survivorship rule) და კონფლიქტების ანგარიში.

2. მარტივად რომ ავხსნათ. ერთი ოქმი ორ ჩანართში წერია. თუ ორივე ერთნაირია, ერთს ვტოვებთ. თუ განსხვავდება, წესით ვირჩევთ, რომელია სწორი, და დანარჩენს ანგარიშში ვაჩვენებთ, რომ ადამიანმა გადახედოს.

3. ჩვენს პროექტში. ✅ წესი (მომხმარებლის გადაწყვეტილება, 2026-10-20):

- ერთი ჩანაწერი რჩება;
- თუ კოდი **სპორადულშიცაა**, სპორადული რჩება;
- სხვა შემთხვევაში — ყველაზე სრული.

შედეგი: 379 დუბლიკატი, ყველა სისტემურ ჩანართებს შორის.

4. რატომ. დუბლიკატი სტატისტიკას ზრდის: ერთი ოქმი ორჯერ ითვლება.

5. კულისებში. SQL-ის მაგალითი — B.18, მაგალითი 7. ✅ ძველ პანელში დედუპლიკაცია „ოქმის N + CADCODE“-ით ხდებოდა: „შემოწმებულია — 0 წინააღმდეგობა“ (`HANDOFF.md`).

6. რა შეიძლება აირიოს.

- „ყველაზე სრული“ არასწორი აღმოჩნდა;
- ⚠️ ცარიელი ან არასტანდარტული საკადასტრო კოდი.

7–8. პასუხი IT-ს: „დუბლიკატს ოქმის ნომრით და საკადასტრო კოდით ვპოულობთ. წესი ხელმძღვანელობასთან შეთანხმებულია, და ყოველი მოცილებული ჩანაწერი ანგარიშშია.“

F.4 ისტორიის შენარჩუნება

1. ტექნიკური განმარტება. თუ წყარო ისტორიას არ ინახავს (in-place update), ის ვერ აღდგება. შესაძლებელია ბოლო მდგომარეობის შენარჩუნება, ნედლი ანაბეჭდის (raw snapshot) შენახვა და დროების გამოყვანა არსებული ველებიდან, „მიახლოებითის“ ნიშნით.

2. მარტივად რომ ავხსნათ. ძველ ცხრილში სტატუსი „იშლებოდა და თავიდან იწერებოდა“. რა იყო ადრე, აღარსად ჩანს. ამიტომ ვინახავთ იმას, რაც ახლაა, და ძველი მწკრივის ზუსტ ასლს, რომ არაფერი დაიკარგოს.

3. ჩვენს პროექტში. ✅

- თითო შემთხვევას ერთი `legacy_import` მოვლენა აქვს (`qc_events` = 44 159);
- `approx = true` მონიშნავს ძველი მონაცემიდან გამოყვანილ დროებს;
- ნედლი მწკრივი ინახება `legacy_extra` -ში, და `legacy_source` / `legacy_row` მიუთითებს, რომელი ჩანართის რომელი მწკრივიდან მოვიდა.

4. რატომ. აუდიტისა და დავისთვის, „საიდან მოვიდა ეს ციფრი“ ყოველთვის უნდა ჩანდეს.

5–8. პასუხი IT-ს: „ძველ ცხრილს ისტორია არ ჰქონდა, ამიტომ ვინახავთ ბოლო მდგომარეობას და ნედლ ასლს, წყაროს მითითებით. ძველი ცხრილი თავად არქივად რჩება, მხოლოდ კითხვით.“

F.5 სიზუსტის შემოწმება (შეჯერება)

1. ტექნიკური განმარტება. Reconciliation-ის დონეებია:

- **(ა)** რაოდენობები: წყარო = ჩატვირთული + მოცილებული (დოკუმენტირებული მიზეზით);
- **(ბ)** აგრეგატები ჭრილებით: ოპერატორი, მენეჯერი, ვერდიქტი, თვე;
- **(გ)** შერჩევითი ჩანაწერების ხელით შედარება (sample audit);
- **(დ)** ინვარიანტები: მაგ. ყოველ დახურულს აქვს დახურვის მიზეზი.

დამოუკიდებელი შემოწმების პრინციპი: გადამოწმების ლოგიკა ცალკეა დაწერილი, ისე რომ ჩატვირთვის შეცდომა არ „გაიმეოროს“.

2. მარტივად რომ ავხსნათ. ჯერ ვითვლით: „იყო 44 538, გადავიდა 44 159 და 379 დუბლიკატი მოვაცილეთ — ჯამი ემთხვევა“. მერე ვამოწმებთ თითო ოპერატორზე: „ძველ შემაჯამებელში ამდენი ჰქონდა, ჩვენთან იმდენივეა“. ბოლოს რამდენიმე ოქმს ხელით ვხსნით ორივე ადგილას და ვადარებთ.

3. ჩვენს პროექტში. ✅ (დოკუმენტირებული 2026-10-20, `ARCHITECTURE.md` §9):

- „შემაჯამებელის“ „შემოწმებული“ თითოეულ ოპერატორზე = ჩვენი რიცხვი + მოცილებული დუბლიკატები, ზუსტად. გამონაკლისია 4 ჩანაწერი „პროცესშია“;
- „შემაჯამებელის“ 271 დავა = QC მენეჯერთან მყოფი სისტემური ოქმები ($Q \le S$ წესით), თითოეულ ოპერატორზე ზუსტად. ამას ემატება 11 სპორადული და 8, სადაც $Q > S$, მაგრამ შენიშვნა ცარიელია;
- ✅ ძველ პანელს `validate.py` ჰქონდა: დამოუკიდებელი გადამოწმება Excel-თან, „ლოგიკა განზრახ დუბლირებულია“. 💡 ახალ პლატფორმას მსგავსი სკრიპტი სჭირდება საწარმოო მიგრაციისთვის.

4. რატომ. ხელმძღვანელობა ახალ ციფრებს ძველს შეადარებს. განსხვავება ახსნილი უნდა იყოს, მაგალითად ATR-04-ის სიმძიმის შეცვლა (Handoff C2) ან დაზუსტების ახალი წესი.

5–8. პასუხი IT-ს: „ოთხ დონეზე ვამოწმებთ: ჯამები, ჭრილები, ხელით შერჩევა და ინვარიანტები. პროტოტიპის გადმოტანა თითოეულ ოპერატორზე ზუსტად დაემთხვა ძველ შემაჯამებელს, დოკუმენტირებული გამონაკლისებით.“

F.6 არსებული საწარმოო სისტემების დაცვა მიგრაციისას

1. ტექნიკური განმარტება. მიგრაცია წყაროში არ წერს. კითხვა ხდება read-only ანგარიშით ან ასლიდან, დაბალი დატვირთვის დროს, ლიმიტებით (გვერდები, პაუზები). ძველი სისტემის ფუნქციონირება უცვლელად გრძელდება, სანამ ახალზე ოფიციალური გადასვლა არ მოხდება.

2. მარტივად რომ ავხსნათ. ძველ სახლს გადასვლისას არ ვანგრევთ. ჯერ ასლს ვიღებთ, ახალ სახლს ვამოწმებთ, და ძველი ბოლომდე ფუნქციონირებს.

3. ჩვენს პროექტში. ✅

- „პროტოტიპი არსებულ სისტემებში არაფერს წერს“ (README, ARCHITECTURE §1);
- Distributor-ის ბაზა `mode=ro`-ით იხსნება;
- Google ცხრილები მხოლოდ GET-ით იკითხება, 8 000-მწკრივიანი გვერდებით, 1 წამის პაუზით („ცხრილს ზედმეტად არ ვტვირთავთ“);
- სარკისებრი ჩანართები არ იკითხება.

4–8. პასუხი IT-ს: „წყაროებში არაფერს ვწერთ, და ამას კოდი იცავს: read-only გახსნა, მხოლოდ GET. დიდ წაკითხვას გვერდებად ვაკეთებთ, პაუზებით.“

F.7 ძველი სისტემები მიგრაციისას უნდა მუშაობდეს?

1. ტექნიკური განმარტება. პარალელური პერიოდის (parallel run) მიზანია, რომ ორივე სისტემა ერთსა და იმავე პერიოდზე ერთნაირ შედეგს იძლეოდეს. ამის შემდეგ ხდება cutover: ძველი იკეტება ჩაწერაზე (read-only) და არქივდება. ხელახალი სინქრონიზაცია ორივე მიმართულებით (dual-write) რთული და რისკიანია.

2. მარტივად რომ ავხსნათ. რამდენიმე კვირა ორივე სისტემა არსებობს, მაგრამ **სამუშაო ერთში** უნდა კეთდებოდეს. თორემ ერთი ოქმი ორ ადგილას სხვადასხვანაირად შეიცვლება. ძველი მხოლოდ შესადარებლად რჩება.

3. ჩვენს პროექტში — შეთავაზება 💡 (`DEMO.md`: „Google ცხრილები… პარალელური პერიოდის შემდეგ, როცა ციფრები დაემთხვევა. მანამდე ძველი ცხრილი მხოლოდ საკითხავად რჩება“):

ეტაპი	ძველი ცხრილი	Distributor	ძველი პანელი	ახალი პლატფორმა
დღეს	სამუშაო	სამუშაო	სამუშაო	პროტოტიპი
პილოტი (მცირე ჯგუფი)	სამუშაო დანარჩენებისთვის	სამუშაო დანარჩენებისთვის	სამუშაო	სამუშაო პილოტის ჯგუფისთვის; მისი ოქმები ძველში აღარ ნაწილდება
cutover	**მხოლოდ კითხვა**	**გათიშული** განაწილებაზე	შედარებისთვის	ყველასთვის
შემდეგ	არქივი	არქივი	❓ გაითიშება / ჩანაცვლდება	—

⚠️ კრიტიკულია: ერთი ოქმი ერთდროულად ორ სისტემაში არ უნდა იყოს განაწილებული. 💡 cutover-ის დღეს ძველი ცხრილის ღია შემთხვევები ბოლოჯერ გადმოგვაქვს, და ცხრილი იკეტება.

4–8. პასუხი IT-ს: „პარალელურ პერიოდს ვთავაზობთ, მაგრამ ორმხრივი სინქრონის გარეშე: სამუშაო ერთ სისტემაში ხდება. ძველი ცხრილი cutover-ის შემდეგ მხოლოდ საკითხავი ხდება.“

F.8 რა გამოვიყენოთ ხელახლა და რა აშენდა თავიდან

ნაწილი	წყარო	გადაწყვეტილება ✅
კანდიდატის წესი („პირველი ოქმი“, „სისტემური სერვისი“, ღია სტატუსი)	Distributor	**ხელახლა გამოყენებულია** უცვლელად (`source.py`)
ბლოკის წესი	Distributor	**ხელახლა** (`algo.block_key`)
მენეჯერების გათანაბრება	Distributor (`balance.py`)	**ჩანაცვლდა** პროცენტული დაფარვით (გადაწყვეტილება 2026-10-20)
REGFIELDALL-ის ლოკალური ასლი	Distributor	**დროებით ხელახლა**, `mode=ro`; საწარმოოში ჩანაცვლდება
ავტომატური კოდირება (`rules.gs`)	Apps Script	**გადმოტანილია** Python-ში (`services/codes.py`, `taxonomy.py`) და JS-ში (`static/js/qc-rules.js`)

ნაწილი	წყარო	გადაწყვეტილება ✅
ხარვეზის კოდების კატალოგი (27)	Dashboard (`code_catalog.json`)	**ხელახლა** (`app/seed/code_catalog.json`)
მივლინებები	Dashboard (`missions.json`)	**ხელახლა** (`app/seed/missions.json`)
ზონების GeoJSON, სახელები	Dashboard	**ხელახლა** (`app/static/geo/`)
სახელების რუკა (TblPlannerMap)	QC სამუშაო.xlsx	**ხელახლა** (`person_aliases` , 591)
სტატისტიკის ლოგიკა	Dashboard (`build_dashboard.py`)	**თავიდან აშენდა** ბაზაზე, იგივე წესებით (მეტრიკების ლექსიკონი)
ავთენტიფიკაცია	Dashboard (პაროლები) / Pre-QC hosted (Google)	**Pre-QC hosted-ის პრინციპები** (სერვერული სესია, რეგისტრირებული ელფოსტა)
Pre-QC ვალიდაცია	Pre-QC Validator	**ცალკე რჩება**; შედეგები API-ით 🅘
შრიფტი, Leaflet	Pre-QC	**ხელახლა**

მარტივად რომ ავხსნათ: რაც უკვე კარგად მუშაობდა და შეთანხმებული წესი იყო (რომელი ოქმი შედის, როგორ ჯგუფდება, კოდები), ავიღეთ. თავიდან ავაშენეთ ის, რაც ცხრილზე ან ფაილზე იყო მიბმული და ბაზაში უნდა გადასულიყო: სამუშაო პროცესი, ისტორია, სტატისტიკა, უფლებები.

F.9 საწარმოო მიგრაციის ნაბიჯები და რისკები 💡

#	ნაბიჯი	რას ვამოწმებთ	რისკი
1	**თანხმობა:** რა პერიოდი, რა ჩანართები, ვინ ამტკიცებს შედეგს, cutover-ის თარიღი	წერილობითი გადაწყვეტილება	„ტექნიკურად შესაძლებელი“ ნებართვად აღიქმება
2	**სატესტო მიგრაცია** სატესტო გარემოზე, ბოლო ასლით	ყველა ანგარიში	—
3	**შეჯერება** (F.5) + ხელმძღვანელობის მიერ შერჩევითი შემოწმება	ჯამები ემთხვევა; განსხვავებები ახსნილია	ახალი წესებით (ATR-04, დაზუსტება) ციფრები შეიცვლება — წინასწარ უნდა ითქვას
4	**ღია შემთხვევების განხილვა** — ვისთან რა არის (9 657 დაუხურავი)	QC მენეჯერები ადასტურებენ „ვისთანაა ბურთის“ წესს	არასწორ ადამიანთან მოხვედრა
5	**cutover-ის ფანჯარა:** ძველი ცხრილი იკეტება → ბოლო წამოღება → იმპორტი → შეჯერება → გახსნა	—	გაყინვის დროს შეტანილი ცვლილება იკარგება — საჭიროა გაფრთხილება
6	**ბექაპი** მიგრაციამდე და მის შემდეგ	—	—
7	**პარალელური შედარება** 2–4 კვირა (მხოლოდ კითხვით)	ყოველკვირეული ანგარიში	—
8	**არქივი:** ძველი ცხრილი read-only; token-ების როტაცია	—	ძველი Apps Script-ები აქტიური რჩება

⚠️ **რისკები:**

- სახელით მიბმის შეცდომა (Handoff C6);
- „დაუჯგუფებელი“ ამზომველები;
- ძველი ცხრილის ფორმატის ცვლილება მიგრაციამდე;
- ცარიელი ელფოსტები, რის გამოც მომხმარებლები ვერ შედიან cutover-ის დღეს.

მარტივად რომ ავხსნათ: ჯერ ვრეპეტირებთ, მერე ვამოწმებთ ციფრებს ხელმძღვანელობასთან ერთად, მერე ერთ დღეს ძველ ცხრილს ვკეტავთ, ბოლოჯერ გადმოგვაქვს და ყველა ახალ სისტემაში გადადის. ძველი

შესადარებლად რჩება.

დასკვნა — რა უნდა ვიცოდე ზეპირად

1. **ერთი წინადადებით:** ერთი ვებ-პლატფორმა ხუთის ნაცვლად. წყაროებს მხოლოდ კითხულობს, საკუთარ ბაზაში ინახავს სამუშაოს სრული ისტორიით, და ყველა უფლებას სერვერზე ამოწმებს.
2. **სერვერი:** ერთი VM საკმარისია; PostgreSQL, HTTPS, Google OAuth, SMTP; რესურსი შეფასებაა და პილოტზე გაიზომება.
3. **ბაზა:** PostgreSQL, რადგან ერთდროული ჩაწერა, ბექაპი და უფლებები სჭირდება. SQLite მხოლოდ პროტოტიპისთვისაა. SQL Server-იც შესაძლებელია, ოღონდ ტესტირებით.
4. **ინტეგრაცია:** view → API → ფაილი, ამ რიგით. ასლი ღამით. დღეს ყველაფერი ჩემი კომპიუტერის „გასაღებებით“ მუშაობს, და სერვერს ოფიციალური გასაღებები სჭირდება.
5. **უსაფრთხოება:** Google ვინაობას ადასტურებს, ჩვენ უფლებებს ვაძლევთ. სხვისი ჩანაწერი 404-ს აბრუნებს. ისტორია მხოლოდ ემატება. საიდუმლოებები კოდში არ არის.
6. **გულწრფელად:** საწარმოომდე ჩვენი მხრიდანაც არის სამუშაო (Handoff §8.3, G1–G18), და ამას დამალვის გარეშე ვამბობთ.

გენერირებულია 02-პირადი-ტექნიკური-სახელმძღვანელო.md-იდან — სწორდება მხოლოდ .md ფაილი.